

OpenBook · Forward Deployed Engineer — AI 应用的落地 工程学

Contents

前言	9
这本书的定位	9
写给谁	9
不写给谁	9
这本书的三个承诺	10
这本书引用的核心来源	10
读这本书的方式	10
反馈	11
阅读指南	12
按 FDE 类型选路线	12
章节难度	13
每章结构	13
出场的核心概念	14
配套资源	14
Part I: 起手 — 理解 FDE 工程的形状	15
这个 Part 要解决什么问题	15
包含章节	15
与其他 Part 的关系	15
Chapter 1: FDE 工程的工作流 — 一周 / 一季度的真实日程	16
开场	16
1.1 一周节奏 — FDE 的「钟」	16
1.2 一季度节奏 — FDE 的「四象限」	17
1.3 两条主线下的工作流差异	18
1.4 时间花在哪是对的 — 三条诊断	19
1.5 一个真实的反例	19
关键引用	20
动手清单	20
反模式清单	20
与下一章的关系	21
Chapter 2: 三条铁律 — Sell Outcomes / Fix Forward / Eval-Driven	22
开场	22
2.1 铁律 1: Sell the Outcome, Not the Product	22
2.2 铁律 2: Fix Forward	24

2.3 铁律 3: Eval-Driven	25
2.4 三条铁律的相互关系	26
2.5 怎么向团队 / 老板讲清这三条	26
关键引用	27
动手清单	27
反模式清单	27
与下一章的关系	27
Chapter 3: 两种 FDE 形态 — 数据驱动型 vs LLM 驱动型	28
开场	28
3.1 两种形态的本质区别	28
3.2 决定形态的不是公司, 是项目	29
3.3 不同形态下的 “三条铁律” 具体落法	30
3.4 工具栈的两套肌肉	30
3.5 同一个项目内的形态切换	31
3.6 一个 AWS 视角的形态对照	32
3.7 自检: 你现在该侧重哪种形态	32
关键引用	33
动手清单	33
反模式清单	33
与下一 Part 的关系	33
Part II: 客户发现 — 用工程师的方式做需求	34
这个 Part 要解决什么问题	34
包含章节	34
与其他 Part 的关系	34
Chapter 4: Discovery — 观察 > 提问	35
开场	35
4.1 为什么 “观察” 比 “提问” 重要	35
4.2 三种观察技巧	36
4.3 12 个一定要问的问题	36
4.4 Discovery 报告模板	37
4.5 Discovery 在 AWS 项目中的常见数据准入流程	38
4.6 LLM 项目的 Discovery 特殊点	39
关键引用	40
动手清单	40
反模式清单	40
与下一章的关系	40
Chapter 5: 从需求到验收 — 评估集 / 验收标准 / SOW	41
开场	41
5.1 三个合同物的关系	41
5.2 Eval 集 v0.1 — 工程师的考试卷	42
5.3 验收标准 — 客户的签字栏	45
5.4 SOW — 商务的合同物	46
5.5 三者怎么互相校验	47
5.6 一个完整的小例子 (端到端)	48

关键引用	49
动手清单	49
反模式清单	50
与下一 Part 的关系	50
Part III: 脚手架 — 6 周让 LLM workflow 跑起来	51
这个 Part 要解决什么问题	51
包含章节	51
与其他 Part 的关系	51
Chapter 6: 技术栈速决矩阵 — 模型 / 框架 / 数据库 / 编排	52
开场	52
6.1 选型的两个原则	52
6.2 模型选型矩阵	53
6.3 框架选型矩阵	55
6.4 向量库选型矩阵	56
6.5 编排 / 工作流选型	57
6.6 监控 / Trace 选型	58
6.7 一张总速决表 (30 分钟决定)	59
关键引用	59
动手清单	60
反模式清单	60
与下一章的关系	60
Chapter 7: 决策树 — RAG / Fine-tune / Prompting / Agent	61
开场	61
7.1 四种解法的本质区别	61
7.2 主决策树	62
7.3 RAG vs Fine-tune — 永恒的混淆	63
7.4 RAG 内部的子决策	64
7.5 Agent — 什么时候上	66
7.6 一张总决策表	67
7.7 切换信号	68
关键引用	68
动手清单	69
反模式清单	69
与下一章的关系	69
Chapter 8: 先 Eval 再开发 — 实操指南	70
开场	70
8.1 Eval-Driven Development 的工程定义	70
8.2 Eval 集的“金字塔结构”	71
8.3 Eval 集的三种 metric 实操	71
8.4 把 Eval 集接进 CI — 实操	73
8.5 跑分的“分布”思维	76
8.6 生产采样回流 — 闭环	77
8.7 Eval 集的反模式	78
关键引用	78

动手清单	78
反模式清单	79
与下一 Part 的关系	79
Part IV: 数据与集成 — 面对客户真实环境	80
这个 Part 要解决什么问题	80
包含章节	80
与其他 Part 的关系	80
Chapter 9: 客户数据栈 — Ontology / ETL / 实时管道	81
开场	81
9.1 客户数据栈的典型形状	81
9.2 Ontology — 把 “五湖四海” 统一	82
9.3 ETL — 让数据流起来	83
9.4 数据血缘 / Lineage — 失败定位的命脉	85
9.5 实时数据管道 — 慎用	86
9.6 数据治理的 “最低 5 件事”	86
9.7 一个真实端到端例子	87
关键引用	88
动手清单	88
反模式清单	88
与下一章的关系	88
Chapter 10: 在客户 VPC / 私有部署 / 离线机房工作	89
开场	89
10.1 三种 “非云原生” 客户场景	89
10.2 场景 A: 客户 VPC 工作流	90
10.3 场景 B: 客户私有云 / 自建机房	92
10.4 场景 C: Air-gap (完全离线)	93
10.5 私有化 LLM 部署的工程要点	94
10.6 合规清单 (私有部署 / 中国 ToB 常见)	95
关键引用	96
动手清单	96
反模式清单	96
与下一章的关系	97
Chapter 11: 与遗留系统对接 — SSO / SCIM / API / 审计	98
开场	98
11.1 四件事的关系	98
11.2 SSO — 接客户 AD / Okta / Identity Center	99
11.3 SCIM — 用户生命周期同步	101
11.4 API 集成 — 调客户的旧系统	102
11.5 审计日志 — “你能证明吗?”	103
11.6 Guardrails — Agent 的行为审计	105
11.7 一个完整的整合例子	105
关键引用	106
动手清单	106
反模式清单	107

与下一 Part 的关系	107
Part V: PoC → 生产 — 跨过最难的鸿沟	108
这个 Part 要解决什么问题	108
包含章节	108
与其他 Part 的关系	108
Chapter 12: PoC 过线条件 — 哪些能过, 哪些卡死	109
开场	109
12.1 PoC vs 生产 — 工程指标差 100 倍	109
12.2 PoC 阶段就要做的 6 件事	110
12.3 4 个 “一定卡死” 的信号	112
12.4 PoC 过线的 5 项硬指标	113
12.5 一个真实的 PoC → 生产时间表	113
12.6 一个真实反例	114
关键引用	115
动手清单	115
反模式清单	115
与下一章的关系	115
Chapter 13: 可观测性 / 成本 / 灰度 / 回滚	116
开场	116
13.1 可观测性 — 不只是 “看日志”	116
13.2 成本控制 — Token 是新的 “水电费”	118
13.3 灰度发布 — 不是 “all or nothing”	119
13.4 回滚 — 5 分钟内必须能切	121
13.5 故障演练 — Chaos Engineering for LLM	122
13.6 一个生产化 dashboard 范例	122
关键引用	123
动手清单	123
反模式清单	123
与下一 Part 的关系	124
Part VI: Agent 时代 — FDE 的新工程任务	125
这个 Part 要解决什么问题	125
包含章节	125
与其他 Part 的关系	125
Chapter 14: 在客户环境部署 Agent — 工具集 / 沙箱 / 失败恢复	126
开场	126
14.1 工具集设计 — 少即是多	127
14.2 沙箱 — Agent 的 “权限边界”	128
14.3 失败恢复 — 多步任务中断怎么办	130
14.4 Agent Eval — 不一样的 Eval	132
14.5 Human-in-the-loop — 高风险动作必有人审	133
14.6 部署清单	134
14.7 一个端到端的 Agent 部署	134
关键引用	135
动手清单	135

反模式清单	136
与下一章的关系	136
Chapter 15: MCP 与企业集成 — 把 Agent 接到客户工具上	137
开场	137
15.1 MCP 是什么	137
15.2 企业部署 MCP 的 3 种形态	138
15.3 写一个企业 MCP server — 实操	139
15.4 MCP 的 4 个工程坑	141
15.5 AWS 实操: Bedrock Agent + MCP 集成	142
15.6 多 server 协作 — 一个客户场景	143
15.7 安全清单	144
关键引用	144
动手清单	145
反模式清单	145
与下一 Part 的关系	145
Part VII: 交付与精进	146
这个 Part 要解决什么问题	146
包含章节	146
与其他 Part 的关系	146
Chapter 16: Handoff + 模式提取 — 把方案抽象成可复用资产	147
开场	147
16.1 Handoff 的工程定义	147
16.2 Handoff 的 “3 周倒计时”	148
16.3 Runbook — 给客户的 “操作手册”	149
16.4 培训 — 4 小时课程	150
16.5 模式提取 — 让下一个项目快 5 倍	150
16.6 模式提取的工程动作	151
16.7 行业模板 — 一个例子	152
16.8 公司层面的模式管理	153
关键引用	153
动手清单	154
反模式清单	154
与下一章的关系	154
Chapter 17: FDE 的 T 字成长 — 工程深度 + 行业纵深	155
开场	155
17.1 T 字 = 工程深度 × 行业纵深	155
17.2 工程横向 — 必须有的 5 块基础	156
17.3 行业纵深 — 怎么选 + 怎么深	157
17.4 怎么 “加深” 行业纵深 — 6 个动作	158
17.5 工程深度 vs 工程横向 — 深 1-2 块	158
17.6 5 年成长路径	159
17.7 反模式 — 5 年没长出 T	160
17.8 给现在的你 4 句话	160
关键引用	161

动手清单	161
反模式清单	161
全书结尾	161
附录 A: FDE 工具栈速查表 (2026 版)	163
A.1 模型 API	163
A.2 编排 / 框架	163
A.3 向量库 / Knowledge Base	164
A.4 Eval / Trace	164
A.5 数据工程	164
A.6 部署 / 运行时	165
A.7 安全 / 合规	165
A.8 IaC	165
A.9 监控 / 告警	166
A.10 客户协作	166
附录 B: 模型 / 框架 / 平台对比矩阵	167
B.1 主流大模型 — 9 维对比	167
B.2 RAG 框架对比	168
B.3 Agent 框架对比	168
B.4 向量库对比 — 6 个生产维度	168
B.5 Eval 工具对比	169
B.6 部署平台对比 — LLM 应用维度	169
B.7 编程语言 / 运行时	170
B.8 决策综合表	170
附录 C: 评估集设计模板	172
C.1 共通设计原则	172
C.2 模板 1: RAG 问答 Eval	173
C.3 模板 2: Agent 任务执行 Eval	175
C.4 模板 3: 分类 / 路由 Eval	177
C.5 模板 4: 结构化抽取 Eval	178
C.6 一个完整 Eval 项目结构	179
C.7 标注流程 — 让客户业务专家高效干活	179
C.8 Eval 集生命周期管理	180
C.9 一份 Eval 检查清单	180
附录 D: 客户启动包 — 12 份第一周可用模板	181
模板 1: Discovery 提纲 (访谈用)	181
模板 2: Discovery 总结报告	182
模板 3: PoC SOW 骨架 (6-8 周)	183
模板 4: Eval 集 v0 (jsonl 起手)	184
模板 5: 安全 / 合规问卷应答模板	184
模板 6: 架构图 (Mermaid 起手)	185
模板 7: 风险登记 (Risk Register)	185
模板 8: 周报模板	186
模板 9: Runbook 骨架 (Handoff 前 3 周)	186
3. Top 10 故障 SOP	187

4. Eval 怎么跑	187
5. 关键配置	187
6. 数据 / KB 更新 SOP	187
7. Escalation	187
模板 12: 客户 Stakeholder Map	189
用法总结	190
术语表	191
A	191
B	191
C	191
D	191
E	192
F	192
G	192
H	192
I	192
J	192
L	192
M	193
N	193
O	193
P	193
R	193
S	193
T	193
V	194
其他	194
参考文献	195
一、本书反复引用的核心来源	195
二、2026-05-04 奇点事件相关	196
三、行业博客与 Playbook	196
四、中文社区资源（Part VI 中国语境章节主参照）	196
五、《FDE Rule Book》专项标识	197
六、引用规范	197

前言

这本书的定位

你周一早上 9 点开机。
邮箱里有客户安全官发来的 27 项合规问卷。
Slack 里上一任 FDE 留下的群组 4 个未读，最新一条是
"那个 RAG 的召回怎么又掉了？"
日历上 11 点和客户 VP 同步会，13 点和模型供应商对成本，
15 点要给老板交一份 PoC 评审材料。

你打开 IDE 之前还有 1 小时。
这 1 小时该干什么？

这本书想回答的就是这种问题。

写给谁

已经在做 FDE 的工程师，或被分配了 FDE 工作但 title 不一定叫 FDE 的人。

具体一点：

- 5+ 年工程经验，能独立 own 一个完整后端服务
- 至少跑过一个客户 PoC，至少卡过一次生产化
- 用过 LLM API 写过 Demo，但没在客户环境跑过 Agent
- 自己读 OpenAPI、跑 SQL、看 Trace，不需要中间人

如果你完全没接触过客户现场、没用过 LLM API，建议先做几个小项目再来读。这本书不教基础。

不写给谁

不适合	原因
决定“要不要转 FDE”的求职者 投资人 / 政策制定者 大学生 / 应届	这本书不讲职业判断 太具体，他们要的是结论 缺前置经验，读了也用不上

不适合	原因
想读 LLM 学术原理	不讲 transformer / attention 等

这本书的三个承诺

1. 每个判断都有一个具体场景。

不写“FDE 应该重视客户访谈”这种空话。写的是“接手项目第一周，前 3 次客户会议你应该做什么、不该做什么、问哪 5 个问题”。

2. 每个技术选型都给决策维度。

不写“RAG 比 Fine-tune 好”或反之。给一张矩阵：数据量 / 更新频率 / 答案确定性 / 推理预算 / 合规约束 — 让你自己判断。

3. 每章末尾给“动手清单 + 反模式清单”。

动手清单是这一章读完可以立刻照做的工程动作；反模式清单是 FDE 真实失败案例里反复出现的错误。

这本书引用的核心来源

正文以下面这些为主要依据：

- **A. Lawrence**, Forward Deployed Engineer Rule Book (2025-10)
- **Bob McGrew @ Y Combinator** (2025) — “Sell the outcome, not the product”
- **AWS GenAI Innovation Center** — 公开的 45 天 / 73% 数据
- **Conikeec @ Substack**, The FDE Playbook
- **Nabeel Qureshi**, Reflections on Palantir

完整引用清单见 [bibliography.md](https://github.com/forward-deployed-engineer/bibliography.md)。

读这本书的方式

最有效的读法：带着一个真实的 FDE 项目读。

每读完一章，问自己：

1. 我当前的项目里有这个问题吗？
2. 如果有，我之前怎么处理的？
3. 这一章的方法跟我之前的做法差在哪？哪个更好？

读完一遍约 6-7 小时；如果配合自己的项目，2-3 周可能更划算。

反馈

GitHub Issue 是最快的反馈通道。我特别想要：

- **反例**：你做 FDE 时遇到一个本书方法跑不通的场景
- **真实数字**：你的 PoC 转化率、客户 NPS、第一周交付物清单
- **缺章**：你觉得本书漏掉的关键技术议题

下一版可能会把你的反馈整合进去。

写于 2026-05。

阅读指南

17 章，约 6-7 小时读完。两条主线并存，按你的 FDE 类型选路线。

按 FDE 类型选路线

路线 A: LLM 应用 FDE (推荐主路线)

适合：在 OpenAI / Anthropic / Cohere / 中国大模型公司或下游创业公司做 LLM 应用落地的 FDE。

路线：Part I → Part II → **Part III** → Part V → **Part VI** → Part VII。

重点章：Ch 6 技术栈速决矩阵、Ch 7 RAG/FT/Agent 决策树、Ch 8 Eval-driven、Ch 14 Agent 部署、Ch 15 MCP。

用时：约 5 小时。

路线 B: 现场交付 FDE (Palantir 风格)

适合：在 Palantir、AWS GenAI Innovation Center、AI 咨询公司做客户现场数据 + 软件交付的 FDE。

路线：Part I → Part II → **Part IV** → Part V → Part VII。

重点章：Ch 9 Ontology / ETL、Ch 10 VPC 部署、Ch 11 SSO/SCIM/审计、Ch 12 PoC 过线、Ch 16 Handoff。

用时：约 4 小时。

路线 C: 通读 (FDE 团队 lead / 想拉通两条线)

适合：FDE 团队负责人、要为团队做内部培训的 staff/principal、跨两类项目的 FDE。

路线：从 Part I 一直读到 Part VII。

用时：6-7 小时。

路线 D：当工具书

适合：有一个具体问题，比如“客户要私有部署，我怎么估算硬件”、“PoC 验收标准怎么写”。

用法：直接查附录 A-D 和对应章节。

你的问题	直接查
选模型 / 框架 / 向量库	附录 A、附录 B
写评估集	Ch 8、附录 C
写 SOW / 安全问卷 / 风险登记	附录 D
Agent 沙箱怎么搭	Ch 14
客户 VPC 部署清单	Ch 10

章节难度

- ☐ 直接照做 — 工程师拿来就能用
- ☐ 需要思考 — 需要结合自己项目判断
- ☐ 需要决策权 — 涉及组织 / 商务 / 合规判断

Part	难度
I 起手	☒
II 客户发现	☒
III 脚手架	☒
IV 数据与集成	☒
V PoC→ 生产	☒
VI Agent 时代	☒
VII 交付与精进	☒

每章结构

开场 — 一个真实场景或一个具体反例

正文 — 3-5 节，每节一个工程判断

关键引用 — 一两句来自 Lawrence / McGrew / Conikeec / AWS

动手清单 — 这一章读完可以立刻照做的事

反模式清单 — FDE 真实失败案例里反复出现的错误

时间紧时直接看动手清单和反模式清单。

出场的核心概念

完整术语见 [glossary.md](#)。最关键的：

概念	一句话
Sell the outcome	不卖工具，卖结果 — McGrew 第一定律
Fix Forward	在客户现场就地修，不带回总部 — Lawrence 工法
Eval-driven	先有评估集，再写代码 — 本书 Ch 8 主线
Ontology	Palantir 的核心抽象，本书 Ch 9 给工程师视角
Agent Toolset	给 Agent 暴露的工具集合，本书 Ch 14 主题
MCP	Model Context Protocol，把 Agent 接到企业工具的标准
PoC 过线	从概念验证到生产的鸿沟，本书 Ch 12 主题
Handoff	FDE 离场时的交付动作，本书 Ch 16 主题

配套资源

- [research/](#) — 本书所有研究素材公开可查（07 篇）
- [附录 D](#) — 12 份第一周可用模板
- [GitHub Issues](#) — 提反例 / 真实数字 / 缺章建议

下一步：前言 → [Part I 导读](#)

Part I: 起手 — 理解 FDE 工程的形状

适用范围：通用 / 两条主线起点

这个 Part 是给所有 FDE 看的“地图”。读完应该能回答三件事：1. FDE 的真实日程长什么样 2. 哪三条铁律决定你做得好不好 3. 你现在做的是“数据驱动型 FDE”还是“LLM 驱动型 FDE”，怎么切换

这个 Part 要解决什么问题

很多人接到 FDE 工作，最早的迷茫不是“技术不会”，而是“不知道时间花在哪是对的”。

跟客户开会算不算干活？写 Demo 算不算交付？写 Eval 集算不算正事？跟运维拉群对 SSO 算不算 FDE 的范围？

Part I 不教任何具体技术，目的只有一个——在你打开 IDE 之前，把脑子里关于“FDE 工程师一周做什么、怎么判断对错、当下场景属于哪一类”的坐标系建起来。

剩下 6 个 Part 都建立在这个坐标系上。

包含章节

- **Chapter 1: FDE 工程的工作流。**一周日程 + 一季度节奏，配两条主线（数据 vs LLM）的对照。
- **Chapter 2: 三条铁律。** Sell outcomes / Fix Forward / Eval-driven —— 这三条决定你对没做对的边界。
- **Chapter 3: 两种 FDE 形态。**数据驱动(Palantir / AWS GenAIIC 风格)vs LLM 驱动(OpenAI / Anthropic 风格) —— 同一个项目什么时候切换。

与其他 Part 的关系

- 没有前置：这是全书起点。
 - 后续延伸：Ch 2 的“三条铁律”会贯穿 Part II-VII；Ch 3 的“两种形态”决定你后面读 Part III（LLM）还是 Part IV（现场交付）权重更大。
-

Chapter 1: FDE 工程的工作流 — 一周 / 一季度的真实日程

开场

某周二，上海，客户某互联网金融公司会议室。

09:00 客户运维 Slack 群里 @ 你 — "昨晚 RAG 召回率从 0.82 掉到 0.61"
09:15 打开 LangFuse trace 翻昨天最后 200 条失败请求，发现 78 条都是 "近 3 天保单"类问题 — 客户上周加了一批新险种但没同步进 KB
09:40 写两行重新 ingest 的 SOP 丢回群里
10:00 客户业务部 PM 站会：要新加 4 个意图，下周老板要看 Demo
10:30 跟客户工程一起对一个 OAuth 报错（不是你写的，但你最熟）
11:00 和你公司远端的 staff 工程师拉个 30 分钟，讨论是不是该把召回拆成 retriever 集成版，回去再做
12:00 午饭，客户食堂，听对面运营吐槽 "上次那个 dashboard 慢"
— 你顺手记到本子上，未必这个项目用得到，但是个信号
13:30 写下午要 demo 的 4 个 prompt 变体，提前跑 Eval 集
— 不能在客户面前现挂
15:00 Demo。客户业务负责人当场提了 2 个新场景，你不正面接
—— "我回去拉一下 Eval，明天给您过线评估"
17:00 写 Eval 案例：4 个新场景 × 5 个边角样本 = 20 条
17:30 发版上昨晚那个 ingest fix，灰度 10%
18:00 写日报：今天进度 / 明天 3 件事 / 1 个风险

这就是 FDE 一天。

写代码的时间不到 25%，但你做的每一件事都是工程动作。

1.1 一周节奏 — FDE 的「钟」

把 FDE 的一周抽象成三种 “钟”：

客户钟 (Cadence with Customer)
- 周一站会 / 周五演示 / 月度 review
- 节奏由客户日历决定，FDE 不能错过

工程钟 (Engineering Cadence) - PR 节奏、Eval 节奏、发版节奏 - 节奏由你和你团队决定
学习钟 (Learning Cadence) - 看客户业务、读他们的 wiki、跟运营吃饭 - 节奏只能由你自己保护

新手 FDE 的常见错误是把整个一周让给 “客户钟”，每天追着客户的需求跑，没有 “工程钟” 的固定 cadence，更没有 “学习钟”。

三周后你会发现：交付质量稳定下滑，因为没有 Eval 投入；客户开始绕过你直接找你老板，因为你提不出有 insight 的建议。

推荐周模板（5 天版）

	周一	周二	周三	周四	周五
07:00					← 自己保留学习钟
09:00	客户站会 + 周计划	深耕日 (no-meeting) + Eval review (no-meeting)	客户站会	深耕日	周演示日 + 周报
午饭	跟客户运营一起	跟自己团队远端连	跟客户业务 PM 一起	自己读 wiki	和客户决策层一起
下午	Discovery 访谈	工程深做	客户工程对接日	工程深做	客户演示 + 反馈消化
17:00	日报	日报	日报	日报	周报+下周计划

关键是周二 / 周四 “深耕日 (no-meeting day)” 必须保住。少了它，你就只剩 “项目经理 + 售前”，不再是 Forward Deployed Engineer。

1.2 一季度节奏 — FDE 的「四象限」

一个季度大概 12 周。一个典型的 FDE 项目按 “四象限” 展开：

	Week 1-3	Week 4-7	Week 8-10	Week 11-12
	DISCOVERY 发现	SCAFFOLDING 脚手架	PROD-IZATION 产品化	HANDOFF 交付
Goal	搞清“真问题”	跑通最小闭环	稳定上线 +	客户接走

	和"真预算"	+ Eval 基线	回滚预案	+ 模式提取
Output	Discovery 报告 + SOW 草案	可演示 demo + Eval 集 v0.1	生产部署 + 监控仪表盘	Runbook + Eval v1.0 + Handoff 文档
铁律 关注	Sell outcome	Eval-driven	Fix Forward	全部三条
风险	需求飘 + 期待飘	Eval 缺	生产事故	Handoff 后无人 maintain

记住一件事：这四个阶段的边界很模糊。Discovery 永远不“结束”，Eval 永远不“够”。但每个阶段有一个主任务，你 70% 的时间应该花在主任务上。

季度阶段的判断信号

一个项目你接手时，先判断它在哪个阶段：

问 1：客户的"成功标准"具体到一个数字了吗？

↓

否 → 你在 DISCOVERY 阶段，先去做 Discovery，不要写代码

是 ↓

问 2：你有 Eval 集，能跑出当前版本的分数吗？

↓

否 → 你在 SCAFFOLDING 阶段，先建 Eval，再做 demo

是 ↓

问 3：有人在客户的"真生产环境"里用了至少一周吗？

↓

否 → 你在 PROD-IZATION 阶段，重点是稳定性 + 监控

是 ↓

问 4：客户内部已经有人能不依赖你独立运营了吗？

↓

否 → 你在 HANDOFF 阶段，写 Runbook + 培训

是 → 项目结束，做模式提取（Ch 16）

90% 的 FDE 失败案例都是搞错了自己所处的阶段。最常见错误：以为自己在 Scaffolding，其实还在 Discovery，结果做了一堆没人要的功能。

1.3 两条主线下的 workflow 差异

同样的一周，LLM 应用 FDE 和 现场交付 FDE 的具体活儿差很远：

时段	LLM 应用 FDE	现场交付 FDE (Palantir / AWS GenAIIC 风格)
周一上午	看 LangFuse / LangSmith trace，分析失败样本	看客户数据管道告警，确认 ETL 没坏

时段	LLM 应用 FDE	现场交付 FDE (Palantir / AWS GenAIIC 风格)
周一下午	Discovery: 和业务 PM 对意图、数据形态	Discovery: 和数据团队对 schema、数据所有权
周二全天	调 prompt / 升级 RAG / 调模型	写 ETL job / 建 Ontology 实体 / 调 SQL
周三上午	Eval 跑分、发新版本	数据校验跑批、发 Pipeline
周三下午	客户工程对接: API、限流、缓存	客户工程对接: 数据库连接、Kerberos、网络白
周四全天	深耕: 调评估集、消融 prompt 变量	深耕: 优化作业 / 重构数据模型
周五	Demo: 场景化对话 + 指标	Demo: dashboard + 数据正确性

两条主线在 FDE 这个职业上是同一个心法、不同的肌肉。同一家公司同一个项目里，不同阶段也会切换 —— 这正是 Ch 3 要展开的。

1.4 时间花在哪是对的 — 三条诊断

诊断 1: 每周 ≥ 8 小时投在 “非编码工程”

包括: Discovery 访谈、Eval 标注、看客户业务文档、写 Runbook、读 trace。

< 8 小时的 FDE 容易把项目做成 “远程外包”。

诊断 2: 每两周 ≥ 1 次自发性的客户 face-time

不是会议召集，是你主动凑过去的。比如: - 上客户食堂吃饭 - 周五下班前蹲在他们工位看他们用产品 - 一对一约客户的运营或一线员工聊 30 分钟

< 1 次/两周的 FDE 容易 “信息越来越虚”，做的东西离客户实际工作越来越远。

诊断 3: 每月至少 1 次 “反对客户” 的事

不是抬杠，是有据有数地拒绝某个需求或某个方案。

例如: - 客户要做 “全公司多 Agent 协同”，你拒绝并提议先做单 Agent + 工具增强 - 客户要私有化部署 Llama 70B，你给出推理成本测算建议先用托管 API

< 1 次/月的 FDE 通常已经退化成 “高级实施工程师”，没在用判断力。

1.5 一个真实的反例

一位匿名 FDE 在 Substack 上分享过一个失败案例 (2025):

项目第三周客户说 “我们想要 agent 能写邮件、能查 CRM、能改日历，全打通”。我没多问，回去做了 6 周三个工具集成 + 编排逻辑，第八周演示，客户一句 “挺好” 就没下文了。

后来才知道，他们 CRM 数据有 80% 是空的；CEO 当时说那句话只是因为某个会议他刚刚听完一个 demo。但他们真正紧迫的是销售月底搬数据进 ERP — 一个一上午能做完的小自动化。我做了一个 6 周的 demo，他们买不了。

问题诊断： - 在 Discovery 阶段就跳进 Scaffolding（搞错阶段） - 没有 Eval，无法量化 “agent 写邮件” 的过线标准（违反 Eval-driven 铁律） - 没卖 outcome（搬数据进 ERP），卖了 product（多工具 agent）

这本书后续每一章都在帮你避免这个反例。

关键引用

“The most expensive line of code is the one nobody asked you to write.” — Conikeec, The FDE Playbook, 2025

“Forward deployment is not a job; it’s a posture toward the customer’s workflow.” — A. Lawrence, FDE Rule Book (paraphrased), 2025

“The best FDEs don’t sell the agent; they sell the after-state of the workflow.” — Bob McGrew @ YC, 2025

动手清单

接到一个新 FDE 项目的第一周，照做以下 7 件事：

1. 拿到客户已有 **wiki / Confluence** 的访问 — 不是 “待办”，是第一周必到位
 2. 建一个个人的 “项目日记” 文件（建议 Obsidian / Notion 私人空间），每天 5 分钟记当天的 3 个观察
 3. 画自己的 “客户钟”：客户一周哪几天有什么节奏会议，自己日历对齐
 4. 强制设两个 “深耕日”（推荐周二、周四）— 拒绝一切非紧急会议
 5. 第一次 **Discovery** 会议前写一份 12 个问题的清单（参见 Ch 4）
 6. 找到客户 “非决策层” 的 1-2 个一线员工，约 30 分钟咖啡（这是 Lawrence 的 Immersion Before Judgment）
 7. 本周末写一份「我以为客户要做 X，实际可能是 Y」的备忘（防 1.5 节那种反例）
-

反模式清单

- ☒ 把一周的所有时间都让给客户钟（你会变成 “高级 PM”）
 - ☒ 没有 “深耕日”（每天被打断，无法做工程深度的事）
 - ☒ 跳过 **Discovery** 直接写 **demo**（最贵的代码就是没人要的代码）
 - ☒ 不写日报 / 周报（半年后你也忘了自己做过什么，模式提取无从谈起）
 - ☒ 从不 “反对客户”（客户要什么你做什么，离 FDE 退化成实施工程师只差几周）
 - ☒ 不知道自己在哪个阶段（用 1.2 节的判断树自检）
-

与下一章的关系

这一章讲了 FDE 的“时间形状”。下一章讲“判断形状”——三条决定你做对做错铁律。读完两章，你就有了完整的 FDE 工程坐标系。

Chapter 2: 三条铁律 — Sell Outcomes / Fix Forward / Eval-Driven

开场

你在客户会议室。客户 CTO 投影上是一张你画的架构图。
他指着中间的方框问：“这个 Agent 我们买了之后，谁负责调？”

你心里第一个反应是：

- A) “可以由你们运维团队定期调 prompt”
- B) “我来负责，每月一次回归”
- C) “你们的销售总监能接手”

错。这三个都是 FDE 思维退化版的回答。

正确反应只有一个 ——

“您当初要的不是 Agent，是月底销售提交的报表准点率从 60% 到 95%。
我们上个月做到了 92%，下个月达成 95% 之后，
谁的 KPI 跟这个 95% 直接挂钩，谁就是 owner。”

把话题从产品拉回 outcome，是 FDE 三条铁律里最重要的第一条。

2.1 铁律 1: Sell the Outcome, Not the Product

原理 — 为什么这条最重要

Bob McGrew 在 2025 年 YC 的访谈里说过这句话，被反复引用：

“Stop selling the product — sell the outcome.”

为什么这条这么重要？

因为**软件 / AI 系统是手段，客户业务结果是目的**。一个客户花钱不是因为想要 “一个 Agent”，而是因为他想：

- 把月底报表准点率从 60% 拉到 95%
- 把客户工单首响时间从 4 小时降到 30 分钟
- 把销售跟单转化率提 5 个百分点

如果你在卖“工具”，客户会用“工具好不好”来评价你——这个评价无法量化，永远没有终点。
如果你在卖“结果”，客户用“那个数字达没达成”评价你——这个评价可量化、有终点、能达成 mutual win。

工程师怎么把这条用起来

每次和客户对话，问自己：

问 1：这次对话最后，客户改变的是“产品理解”还是“业务结果预期”？

问 2：我能用一个数字描述这次对话的成果吗？

问 3：如果这个数字达成了，下一个对话的数字是什么？

FDE 的工作就是把客户对话从“功能讨论”持续拉回“结果讨论”。

实操：三个对话翻译模板

客户说：“这个 Agent 能不能也接邮件？”

□ 直答：“可以，我们排到下周”

□ 翻译：“您加邮件是为了销售跟单回复更快是吗？
那我们看一下当前跟单回复中位时间是多少，
加邮件后预计能降到多少，再排进 backlog”

客户说：“你们这个 RAG 准确率到底多少”

□ 直答：“测试集上 0.83，生产是 0.78”

□ 翻译：“RAG 召回 0.78 这个数字本身意义不大。
更关心的是：客户经理过去用搜索找一份保单条款
要 4 分钟，现在用 RAG 平均 28 秒。
这个 28 秒，和您当初定的‘30 秒以内’KPI 对得上”

客户说：“我们想要‘AI 驱动的全公司知识库’”

□ 直答：“好，我们做架构方案”

□ 翻译：“全公司知识库这个目标太大，先选一个 outcome：
是新员工上手时间从 30 天降到 15 天？
还是客服解决率从 65% 升到 80%？
选定后我们 6 周就能上一个版本。”

2.2 铁律 2: Fix Forward

原理 — 在客户现场就地修

Lawrence 在《Rule Book》里反复强调 Fix Forward:

“Don’ t carry the problem back to HQ. Fix it at the customer site.”

这条听起来简单，背后是工程哲学：

FDE 不是“前线的售前 + 后方的研发”两段式工作。FDE 在前线就既是售前，也是研发，也是运维。

如果你遇到问题第一反应是“回去开个 Jira，让总部排期”，你已经退回到了“实施工程师”的位置。

怎么算 Fix Forward 落地

场景	“Fix Forward 落地”标准
Eval 集发现一个新失败模式	当天就在 prompt / retriever 里写补丁，下午跑 Eval 验证
客户某个 API 返回不稳定	不等客户 IT 排期，自己写一层 fallback / 重试
RAG 召回掉了	不等“下次大版本”，当晚 hot fix 后灰度 10%
客户合规问到一个细节	当场打开架构图修订，邮件里就给出修正版
客户说“这个 demo 不像我们的语气”	当周就调 prompt 风格 + 给客户标注 5 个范例

反例 — Lawrence 在书里讲过的

某 FDE 在欧洲某国防客户处部署，发现数据格式有 5% 的样本特殊（unicode + 历史遗留编码）。他写了 ticket 回总部，等了 3 周，研发说“通用方案要重构”。客户在第 4 周失去耐心，项目搁置。

同事接手后做的第一件事：当天写了一个 30 行的 transformer 处理这 5%，第二天数据全过。3 个月后客户续单 200 万美元。

Fix Forward 不是不写工单，而是工单和 hot fix 并行。

工程师的 Fix Forward 配置

为了能 Fix Forward，FDE 必须有一些**当场修的能力配置**：

- 客户环境的部署权限（至少 staging）
- 自己仓库的 main 分支直接 push（受 CI 保护）
- 至少一个能 hot fix 上线的发版通道（Lambda / 边车容器 / 配置中心）
- 一份简单的“补丁脚本目录”（python scripts / one-liner shell）

如果上面任何一条没有，**你的 FDE 工作 50% 已经不可能 Fix Forward。**

2.3 铁律 3: Eval-Driven

原理 — 没有 Eval 的 LLM/Agent 项目都是 “信仰项目”

LLM 系统和传统软件最大的区别：输入相同，输出不一定相同；今天工作的 prompt，模型升级后可能就坏了。

没有 Eval 集 = 你和客户都靠 “感觉” 判断好坏 = 客户某天觉得不好你也不知道为什么。

Eval-driven 的工程定义

不是 “写完代码顺便跑一跑评估”，是：

1. 先定 outcome 数字（铁律 1）
2. 把 outcome 翻译成可自动测的 Eval 集（输入 → 期望输出 → 通过条件）
3. 写代码的每个 PR 都跑 Eval
4. 上线前 Eval 必须达到客户 sign-off 的过线标准
5. 生产中持续采样新 case 加入 Eval（回归集）

Eval 集是 FDE 第一周就要建的，不是上线前才补的。

一个最小 Eval 例子

假设客户做的是 “保单条款问答 RAG”。Eval 集长这样：

```
# evals/insurance_qa_v0.1.jsonl
{"input": "重疾险等待期一般是多少?", "expected_keywords": ["90", "180", "等待期"], "min_score": 1}
{"input": "我去年体检发现甲状腺结节，现在能投保吗?", "expected_keywords": ["核保", "告知", "结论"], "min_score": 1}
{"input": "被保险人犹豫期内退保能退多少?", "expected_keywords": ["全额", "犹豫期"], "min_score": 1}
# ... 50-200 条
```

跑分有两层：- 关键词召回（机器自动）—— 能不能跑通 - 业务专家打分（人工抽样）—— 真正用的好不好

每周更新一次，永远跑。

在 AWS 上的 Eval 工程实践

如果你在 AWS Bedrock 上做 LLM 应用，Bedrock 自带评估能力可以直接用：

- **Amazon Bedrock Evaluations** — 在 Bedrock 控制台 / API 里直接对模型 / RAG / Agent 跑评估，支持 LLM-as-judge、人工评估和程式化评估三种 evaluator。
- **Knowledge Base Evaluations** — 专为 RAG 系统的 “召回质量” 和 “生成质量” 设计的评估流程。
- **Agent Evaluations** — 针对 agent 的多步推理路径做评估，支持自定义评分维度。

实践建议：- **PoC 阶段**：用 Bedrock 内置的 LLM-as-judge，10 分钟跑通一个 baseline - **生产前**：自己写代码版的 Eval (pytest / DeepEval / Promptfoo)，跑在 CI 里 - **生产后**：CloudWatch 收集线上真实样本 → 周期性抽样回 Eval 集

AWS 知识参考：在 AWS 文档里搜 “Amazon Bedrock evaluation” 或 “Knowledge Bases evaluation”。Bedrock 的 evaluation 在 console → “Evaluations” 入口；具体最新流程随产品迭代，使用前以 docs.aws.amazon.com 为准。

为什么 FDE 必须坚持 Eval-driven

没 Eval 的项目：

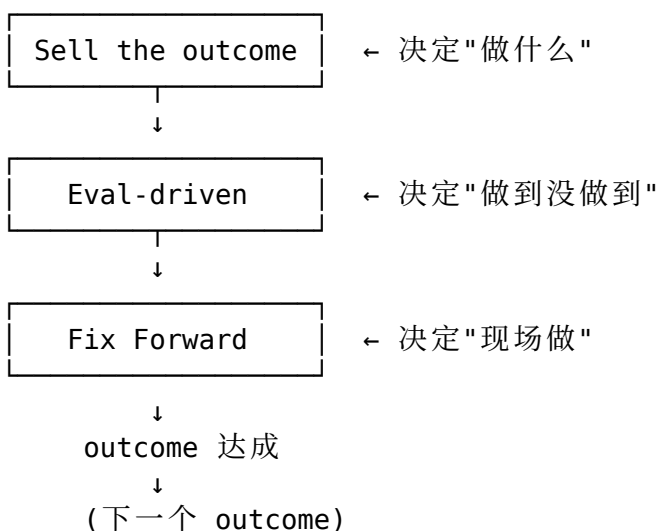
"今天感觉好像还行"
客户主观打分，每周飘
模型供应商升级你不敢动
PoC 过线靠"演示运气"
Handoff 后无人知道好坏

有 Eval 的项目：

今天 0.83，比昨天 +0.02
客户每周看分数，可解释
模型升级一晚跑一次 Eval 即可决断
PoC 过线靠数字
Handoff 后客户自己跑 Eval 即可监控

2.4 三条铁律的相互关系

三条不是平行的，而是一个判断闭环：



如果三条只能记一条，记 **Sell the outcome**。如果记两条，再记 **Eval-driven**。如果记三条，加上 **Fix Forward**。

2.5 怎么向团队 / 老板讲清这三条

实际工作里你需要把这三条“销售”给：

- 你自己公司的产品 / 销售（他们容易把“卖产品”灌输给你）
- 客户的项目经理（他们容易把“做功能”压给你）
- 客户的 CTO（他们容易把“AI 战略”压给你）

实操话术（中文版）：

“我接手这个项目第一件事是先敲定一个数字 —— 比如客户经理找一份条款的中位时间从 4 分 30 秒降到 30 秒。然后我会建一个 80 条的 Eval 集，每周跑分。所有 PR 没过

Eval 不进 main。3 个月后我交付时您看到的不是‘AI 系统’，是‘中位时间从 4 分 30 秒到 27 秒’这个数字。”

讲完客户基本会同意。如果不同意，问题更大——客户自己不知道要什么 outcome，那你应该先做 Discovery (Ch 4)。

关键引用

“Stop selling the product — sell the outcome.” — Bob McGrew @ YC, 2025

“Don’ t carry the problem back to HQ. Fix it at the customer site.” — A. Lawrence, FDE Rule Book, 2025

“If you don’ t have an eval set, you don’ t have a system — you have a hope.”
— Anonymous FDE, The FDE Playbook, 2025

动手清单

把三条铁律落到自己当前项目：

1. 写下当前项目的 **outcome 数字**：一句话 + 一个数字 + 一个时间窗口
 2. 判断当前 PR 是不是在直接服务这个数字；不是的下周砍掉
 3. 建 **Eval 集 v0.1**：哪怕只有 20 条（一上午搞定，不要等“完美样本”）
 4. 打开 CI，让 Eval 跑在每次 PR 上
 5. 检查 **Fix Forward** 配置：你有 staging 部署权限吗？有 hot fix 通道吗？
 6. 下周一会议用 2.5 节话术对齐客户和团队
-

反模式清单

- ☒ 以为“演示好”就是“项目好”（演示是 outcome 的代理，不是 outcome 本身）
 - ☒ Eval 等到上线前才补（Eval 是开发约束，不是验收文件）
 - ☒ 遇到问题第一反应是“我回去找研发排期”（你就是研发）
 - ☒ 接受客户的“先看效果再谈数字”（永远没有“看效果”那一天）
 - ☒ 把 Eval 当成 QA 的活（Eval 是 FDE 的核心交付物之一）
 - ☒ 同时卖三个 outcome（一次一个，做完再来下一个）
-

与下一章的关系

铁律是判断“做对没做对”的标尺。下一章讲两种 FDE 形态——同一个项目里“数据驱动”和“LLM 驱动”什么时候切换，铁律的具体落法是不一样的。

[← 上一章](#) · [下一章](#): 两种 FDE 形态 →

Chapter 3: 两种 FDE 形态 — 数据驱动型 vs LLM 驱动型

开场

两个 FDE 在同一家咖啡店。

A 在 Palantir, 10 年。今天他在改一段 PySpark, 把客户某 ERP 表增量同步到 Foundry 的 Ontology, 建一个"客户-订单-发货"的图谱。他读 Avro schema 的速度比读小说快。

B 在 OpenAI, 1 年。今天他在调一个 prompt template, 让 Agent 能在客户的 Jira 里自动创建子任务。他熟悉 5 种 LLM 的 system prompt 风格差异。

两个人 title 都是 Forward Deployed Engineer。
两个人做的事看起来完全不一样。

但你跟他们聊一小时会发现 —— 思维方式 80% 重合, 工具栈 80% 不重合。

这一章讲: 这两种 FDE 的边界在哪、什么时候切换、你接到一个项目时怎么判断自己应该用哪种模式。

3.1 两种形态的本质区别

	数据驱动型 FDE	LLM 驱动型 FDE
代表公司	Palantir, AWS GenAIIC Snowflake PS, Databricks 传统 ToB 咨询	OpenAI, Anthropic Cohere, Cresta 下游 AI 创业公司
核心问题	客户的"数据"怎么用上	客户的"工作流"怎么自动化
主要交付物	Ontology + Pipeline	Agent + Prompt + Toolset

	+ Dashboard + Report	+ Eval Set
用的硬技能	SQL, Spark, ETL, dbt 数仓建模, schema 演化 Avro/Parquet/Iceberg	LLM API, Prompt RAG, Agent 框架 MCP, Function Calling
用的软技能	和数据团队 / DBA 协作 懂数据治理 / 隐私合规	和业务 PM / 一线员工协作 懂业务流程 / 用户画像
时间分布	50% 数据探索 30% Pipeline 工程 20% 业务对接	40% Discovery / 业务理解 30% Prompt + Eval 工程 30% 集成 + 部署
成功的样子	客户做决策时能直接用数据回答问题	客户工作流里某个动作的中位时间显著下降
最致命的失败	数据正确但没人用	Demo 惊艳但用不起来

3.2 决定形态的不是公司，是项目

很多人以为“我在 X 公司就是 X 形态”。错。

真正决定的是项目所处的阶段和客户的瓶颈：

判断你当前在哪种形态

问 1：客户的瓶颈是“数据用不上”还是“ workflow 没自动化”？

↓

数据用不上

→ 数据驱动型

↓

workflow 没自动化

→ 看问 2

↓

问 2：现有的 workflow 是基于结构化数据还是基于自然语言？

↓

基于结构化数据

→ 偏数据驱动型

（比如自动报表）

基于自然语言/文档/对话

→ LLM 驱动型

（比如客服 / 合同 / 知识问答）

几个真实例子

例 1：金融客户要“理赔自动化”

- 第 1-4 周：读他们的理赔流，发现关键卡点在“医生病历 → 理赔金额”的非结构化转结构化 → **LLM 驱动型**（OCR + LLM 提取）
- 第 5-8 周：发现提取出来的字段需要和保单条款 join，做规则引擎 → **数据驱动型**
- 第 9-12 周：把规则化的判断重新封装成可解释 Agent → **LLM 驱动型**

同一个项目，三个阶段三种形态。

例 2：制造业客户要 “设备预测性维护”

- 整个项目核心是时序数据 + 模型 → **数据驱动型主线**
- 但报告生成 / 工单语言 / 操作员问答这块用 LLM → **LLM 驱动型支线**

数据驱动主线 70%，LLM 支线 30%。**不是非此即彼。**

3.3 不同形态下的 “三条铁律” 具体落法

铁律没变，但落法不一样：

Sell the outcome

	数据驱动型	LLM 驱动型
Outcome 例子	月度报表准点率 60%→95%	客服首响中位时间 4h→30min
数字来源	数据系统本身能算	需要新加埋点 / trace
沟通对象	数据团队 + BI 部门 + 业务	业务团队 + 一线员工

Eval-driven

	数据驱动型	LLM 驱动型
Eval 标的	数据正确性、口径、SLA	答案质量、相关性、安全性
Eval 工具	dbt tests, Great Expectations, 自写 SQL	DeepEval, Promptfoo, Bedrock Evaluations
跑分频率	每个 ETL run	每个 PR + 每天回归

Fix Forward

	数据驱动型	LLM 驱动型
现场修什么	一段 SQL / 一个 dbt 模型 / 一个调度	一段 prompt / 一个 retriever 配置 / 一个 tool 定义
Hot fix 通道	Airflow / dbt cloud 直接发	配置中心 / Lambda / 边车 prompt 仓
部署权限	数据库写权限（受限）	应用配置写权限（多用）

3.4 工具栈的两套肌肉

下面两个清单，你可以判断自己当前缺哪一块：

数据驱动型 FDE 必备

入门	中级	高级
SQL (必备)	dbt	数据建模 (Kimball / Inmon)
Python pandas	Airflow / Prefect	Apache Iceberg / Delta
JSON / Avro	Spark / PySpark	ontology 设计 (Palantir 风格)
PostgreSQL	Snowflake / BigQuery	schema 演化
	Redshift	实时管道 (Kafka, Kinesis)
	Kerberos / IAM	数据血缘 (OpenLineage)
	ETL 调试	隐私合规 (GDPR, PII)

LLM 驱动型 FDE 必备

入门	中级	高级
LLM API 调用	LangChain / LlamaIndex	Agent 框架 (LangGraph, AutoGen, Bedrock Agents)
Prompt 工程	RAG (向量库)	MCP 协议
Function Calling	Eval 框架	Agent 工具沙箱 / 权限
JSON Schema	Trace 工具	Function Calling 复合
	(LangFuse, Phoenix, LangSmith, Bedrock)	流式 / 结构化输出
	Vector DB	微调 (LoRA)
	(Pinecone, Weaviate, OpenSearch, pgvector)	推理优化 (vLLM, TGI)
		部署 (SageMaker, Bedrock)

FDE 不需要两套全精通，但两边都要“懂到能问对问题”。

3.5 同一个项目内的形态切换

实操中，最难的是切换的“判断时机”。给你三个信号：

信号 1：发现“数据本来就有，只是没人能查”

→ 这是 LLM 驱动型机会（自然语言查询、RAG）

例：客户说“我们的合同条款搜不到”。先别急着写 ETL，看看是数据格式问题还是检索问题，可能 RAG 一上来就解决。

信号 2：发现“LLM 输出基本对，但下游没法接进系统”

→ 切回数据驱动型（结构化输出、schema 校验）

例：Agent 抽取的字段进不了 ERP，因为编码不统一。这时候你需要数据治理思维——建 mapping 表 / 校验规则 / 异常处理。

信号 3：发现 “客户决策需要数字，但现有数据不够”

→ 数据驱动型主线（埋点、ETL、看板）

例：客户问 “这个 Agent 到底节省了多少时间”，你发现没埋点。先做埋点 → ETL → 看板，再继续 LLM 工作。

3.6 一个 AWS 视角的形态对照

AWS GenAI Innovation Center 的 FDE 是 “两种形态都做” 的典型代表。它的项目通常按以下 stack 配置：

	数据驱动型部分	LLM 驱动型部分
存储	S3, Lake Formation	S3 (KB 文档)
数据	Glue ETL, Glue Catalog	OpenSearch / Knowledge Bases
计算	EMR, Athena, Redshift	Bedrock (model invoke)
编排	Step Functions, MWAA	Bedrock Agents / Step Functions
治理	Lake Formation 权限	IAM + KMS
监控	CloudWatch + OpenLineage	CloudWatch + Bedrock guardrails + Bedrock Evaluations

一个客户的项目同一周可能既动 Glue ETL，又动 Bedrock Agent。FDE 必须能 “上下左右” 切换。

AWS 知识参考：完整的两条线工具集见附录 A（FDE 工具栈速查表）和附录 B（对比矩阵）。

3.7 自检：你现在该侧重哪种形态

项目特征	→ 偏向哪种形态
客户最大痛点是"数据找不到"	→ 数据驱动 60%
客户最大痛点是"重复劳动太多"	→ LLM 驱动 70%
客户合规要求很严	→ 数据驱动 50%（先治理）
客户业务高速增长，文档跟不上	→ LLM 驱动 60%
客户已经有数据团队	→ 你做 LLM 驱动主线
客户没有数据团队	→ 你做数据驱动主线
客户希望"3 个月内见效"	→ LLM 驱动（更快见效）
客户希望"接下来 3 年的基础设施"	→ 数据驱动（基础更扎实）

关键引用

“The forward deployed engineer is data-driven by training and LLM-driven by opportunity.” — A. Lawrence (paraphrased), FDE Rule Book, 2025

“The boundary between data work and AI work disappeared the moment LLMs became infrastructure.” — AWS GenAI Innovation Center positioning, 2025-2026

动手清单

1. 判断你当前项目的形态（用 3.2 / 3.7 的判断树）
 2. 写一句话给老板：本季度 70% 时间在 X 形态、30% 在 Y 形态
 3. 检查自己的工具栈缺口（3.4 节两个清单）选最缺的一项，本周补
 4. 找你公司里另一种形态的资深 FDE 喝杯咖啡，问他常见的失败模式
 5. 画一张你当前项目的「两种形态分工图」，对每个模块标注偏哪边
 6. 下次客户对话有意识感受：“这个需求是数据型还是 LLM 型？”
-

反模式清单

- ☒ 强行用一种形态做所有事（“我擅长 LLM 所以一切都用 LLM 解”）
 - ☒ 数据治理还没做就堆 Agent（地基不稳，越多 Agent 越糟）
 - ☒ 数据已就绪还逼自己写 ETL（错过 LLM 快速见效窗口）
 - ☒ 跨形态切换时不知会客户和团队（下游懵）
 - ☒ 以为“切换形态 = 推翻重做”（很多时候只是接一层适配）
-

与下一 Part 的关系

Part I 三章给了完整的 FDE 工程坐标系：时间形状（Ch 1）、判断形状（Ch 2）、能力形状（Ch 3）。

Part II 进入第一个具体动作 —— Discovery。从你接到一个新项目开始，**第一周到底问什么、看什么、写什么**。这是 FDE 工作中“最容易被忽视、收益却最高”的阶段。

[← 上一章: 三条铁律](#) · [下一 Part: 客户发现](#) →

Part II: 客户发现 — 用工程师的方式做需求

适用范围：通用（两条主线都需要）

这个 Part 要解决什么问题

FDE 项目最贵的代码，是没人要的代码。

而“没人要的代码”几乎都来自一个原因 —— **Discovery 没做透**。

很多工程师听到“Discovery”第一反应是“那是 PM 干的事”。错。FDE 的 Discovery 不是 PM 那种“开会听需求 + 写 PRD”，是带工程师视角去**观察、量化、原型化**客户的真实工作流，最终输出可执行的评估集和 SOW。

Discovery 没做透 → 工程做得越快，方向错得越远。

这个 Part 教两件事：1. 怎么用观察 + 提问 + 原型 三种动作做完一轮 Discovery 2. 怎么把 Discovery 的结论翻译成评估集 + 验收标准 + SOW

包含章节

- **Chapter 4: Discovery — 观察 > 提问**。怎么读客户的工作流；12 个一定要问的问题；3 种现场观察技巧；Discovery 报告模板。
- **Chapter 5: 从需求到验收 — 评估集 / 验收标准 / SOW**。怎么把 Discovery 翻译成可工程化的合同物；评估集 v0.1 长什么样；SOW 模板。

与其他 Part 的关系

- **前置**：Part I 三章提供的坐标系
 - **后续延伸**：本 Part 的产出（评估集 + SOW）是 Part III/IV/V/VI 全部的输入。Discovery 不达标，后面所有工程都是浪费。
-

Chapter 4: Discovery — 观察 > 提问

开场

一位 FDE 第一次去客户现场。

他准备充分：列了 30 个问题，提前看了客户官网、年报、产品白皮书，还自带了一份“客户场景评估问卷”。

会议从早上 9 点开到下午 4 点。客户配了 5 个人陪聊，轮番回答了所有问题。

他回到酒店写笔记，越写越虚 ——

30 个问题都“答上了”，但他没有一条具体到能写代码的需求。

更糟的是，他不知道哪些回答是真的，哪些只是客户随口给的“标准答案”。

第二天他做了一件不同的事：去客户的 call center 蹲了半天，看坐席接 30 通电话用什么系统、查什么、记什么。

回去后那份笔记，比前一天 7 小时会议价值高 10 倍。

这一章想说一句话：观察 > 提问。

4.1 为什么“观察”比“提问”重要

客户回答你的问题时，倾向回答他们自己以为的工作流，不是实际的工作流。

工程上叫 **stated preference vs revealed preference**：- 嘴上说的偏好（stated）和实际行为透露的偏好（revealed）经常不一致 - 客户老板告诉你“我们用 CRM 跟单”，你蹲销售旁边看一天发现他们 70% 时间在用 Excel - 客户合规告诉你“所有数据都在 SAP”，你查询一下发现 30% 关键数据在某个共享盘的 Excel 里

FDE 的 Discovery 必须建立在 revealed preference 上。怎么 reveal? 观察。

4.2 三种观察技巧

技巧 1: Shadowing (影子跟随)

跟着客户某个一线员工工作半天，做“哑巴”——不打断，不评论，只看。

带一个本子记两栏：

	他做什么	他卡在哪
0:00	打开 5 个 tab Outlook + CRM + Excel + 共享盘 + 内部 wiki	CRM 加载 8 秒
0:08	搜客户 "ABC Corp"	CRM 搜索结果重复 3 个 不知道哪个对
0:12	打开 Outlook 找最近邮件	得手动翻翻搜搜， 耗 4 分钟

...

蹲 4 个小时，比开 8 小时会议管用。

技巧 2: System Walkthrough (系统巡礼)

请客户登录他们最常用的几个系统，让他们边操作边讲：“你为什么先查这个，再查那个？”

不是看系统功能，是看人和系统的实际交互模式。

记录什么：- 一个任务平均要打开几个系统？- 哪些是 hot path（每天都用），哪些是 cold path（偶尔用）？- 哪些 hot path 上有粘贴 / 切换 / 等待 / 手算的痛点？

技巧 3: Artifact Mining (产物挖掘)

让客户给你看 5 类真实的产出物：

1. 上周一个销售经理交给老板的销售周报（看真实结构）
2. 上个月一份客户工单的解决记录（看真实流程）
3. 最近一份内部例会的会议纪要（看真实词汇）
4. 一份客户拒绝的合同附件（看真实拒绝原因）
5. 上个季度的 KPI 报告（看真实指标）

真实产物 > 任何 PPT 介绍。客户内部用的真实词汇、KPI 口径、流程节点，全在产物里。

4.3 12 个一定要问的问题

观察 + 这 12 个问题，足够让你做完一轮初步 Discovery：

业务现状（4 个）

1. 如果今天什么都不做，3 个月后这个问题会怎样？
 - 答 “差不多就这样” → 客户没痛点，慎接
 - 答 “会更糟” → 量化 “更糟” 是多少
2. 现在这个流程一个月发生多少次？每次平均多长时间？
 - 没数字 → 你的第一周任务是埋点
3. 有没有人尝试过解决这个问题？为什么没成？
 - 没人试过 → 慎接（可能是问题不重要）
 - 试过失败 → 听失败原因，避坑
4. 如果今天解决了，您怎么知道？
 - 这个问题决定了 outcome 数字

技术现状（4 个）

5. 数据从哪来？有没有 owner？
 - 没 owner 的数据用不了
6. 你们最近一次系统改造是什么时候？谁批的？
 - 揭示决策链
7. 生产环境部署一次平均多久？要哪些审批？
 - 决定你的 Fix Forward 配置可不可行
8. 如果模型出错给了一个错答案，最坏会发生什么？
 - 决定容错设计 / 安全约束 / Guardrails 强度

组织现状（4 个）

9. 谁的 KPI 跟这个项目的成功直接挂钩？
 - 没人 → 项目大概率会冷掉
10. 谁是反对者？为什么反对？
 - 没反对者 → 可能没人真懂这事
11. 谁会接手运营？什么时候开始介入？
 - 没人 → Handoff 风险已经显现
12. 如果 PoC 成功，3 个月后什么样？
 - 答不上 → 客户对生产没真规划

4.4 Discovery 报告模板

第一轮 Discovery 跑完（推荐 1-2 周）应该输出一份不超过 3 页的 Discovery 报告，写给你公司老板和客户决策层共同看：

Discovery 报告 — 【客户名】 / 【项目名】
日期：YYYY-MM-DD
作者：FDE [name]

- =====
1. 真实问题（一句话）

"客户经理找一份特定保单条款的中位时间是 4 分 30 秒，
每月发生约 1.2 万次，痛点是 ____。"

2. 期望 outcome (一个数字 + 一个时间窗口)
"在 [时间] 内把中位时间降到 [数字]，量化方式 ____。"
3. 当前痛点的 5 个观察证据
 - [shadowing 观察 1]
 - [system walkthrough 观察 2]
 - ...
4. 关键约束
 - 数据: [哪些数据可用 / 不可用]
 - 部署: [VPC / 私有 / 云上 / 离线]
 - 合规: [等保 / SOC2 / GDPR / PII]
 - 成本: [预算上限 / token 月预算]
5. 风险与不确定性 (top 3)
 - [风险 1: 影响 + 缓解]
 - [风险 2]
 - [风险 3]
6. 推荐方案 (3 周脚手架)
 - 形态: [数据驱动 / LLM 驱动 / 混合]
 - 关键技术: [模型 / 框架 / 部署模式]
 - 评估集 v0.1: [N 条样本, 覆盖 K 个场景]
 - 第一个 milestone: [在 X 周后, 达到 Y 数字]
7. 不做什么 (Out of Scope)
 - [被排除的诉求 1 + 排除原因]
 - [被排除的诉求 2]
8. 下一步
 - 客户: [需要客户做的 3 件事]
 - 我们: [FDE 团队接下来 2 周做的事]

=====

第 7 点 (不做什么) 和第 5 点 (风险) 是 Discovery 报告里最值钱的两节，新手最容易省略。

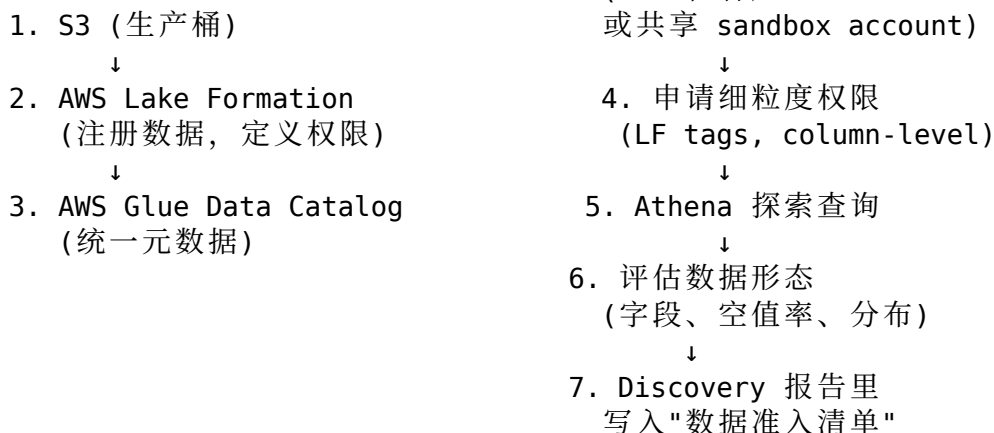
4.5 Discovery 在 AWS 项目中的常见数据准入流程

如果你是在 AWS 上做项目 (尤其是 GenAIIC 风格的现场交付)，Discovery 阶段必须明确**数据访问的合法路径**。这个动作不做，后面 Pipeline 都搭不起来。

典型 AWS 数据准入流程：

客户原始数据

FDE 的工作环境



实操关键: - 不要直接用 ROOT 凭证查数据 —— 让客户用 IAM Identity Center / Lake Formation 给你具体的 LF-tag 权限 - 数据探索阶段全部走 Athena —— 不要 download 全量到本地 (合规风险) - 每个数据来源记录 ownership —— 谁是数据 owner, 谁能批 schema 演化

AWS 知识参考: 详细配置见 docs.aws.amazon.com 的 “AWS Lake Formation getting started” 和 “Granting data permissions”。

4.6 LLM 项目的 Discovery 特殊点

LLM 项目的 Discovery 多两个动作:

1. 现有 “自然语言资产” 盘点

LLM 应用的 “原料” 是文档 / 对话 / 邮件等非结构化资产。Discovery 时盘点:

资产类型	体量	更新频率	谁维护	现在有谁查
内部 wiki	XX 万字	周	各部门 PM	全员 (搜索差)
客服对话历史	XX 万条	实时	客服系统自动	数据团队
销售合同 PDF	XX 份	月	法务	法务
...				

如果数据 owner 不确定 / 数据格式混乱 / 没人维护 → 这个 LLM 项目还不到做的时候。

2. 一份 50 条的 “种子样本”

Discovery 收尾时找 50 条 真实的客户问题 / 工单 / 任务样例。这是 Eval 集 v0.1 的种子。

收集渠道: - 客服系统导出最近 1000 条工单, 让业务专家筛 50 条 representative - 销售员工的工作日志 - 内部 issue tracker - 真实邮件往来 (脱敏后)

有 50 条种子 → Discovery 算 80% 完成。

关键引用

“Stated preference and revealed preference are different — and engineers must trust the latter.” — A. Lawrence (paraphrased), FDE Rule Book, 2025

“The best Discovery week is one where you don’ t write any code.” — Coni-keec, The FDE Playbook, 2025

动手清单

接到新项目第 1-2 周必做的 8 件事：

1. 安排至少 2 次 **shadowing** (4 小时 / 次)，现场观察一线员工
 2. 3 次 **system walkthrough**，覆盖最常用的系统
 3. 拉到 5 件真实产出物 (4.2 节 “产物挖掘”)
 4. 过完 12 个一定要问的问题 (4.3 节)
 5. 盘点客户 AWS 账号 / 数据访问 / 网络白名单情况 (4.5 节)
 6. 盘点客户已有的 “自然语言资产” (4.6 节，LLM 项目)
 7. 收集 50 条种子样本
 8. 在第 2 周末交付一份 3 页 **Discovery** 报告 (4.4 节模板)
-

反模式清单

- ☒ 只开会不去现场 (你只能拿到 stated preference)
 - ☒ **Discovery** 一周内开始写代码 (一定写错方向)
 - ☒ **Discovery** 报告里没 “不做什么” (范围会失控)
 - ☒ **Discovery** 没拿到 50 条种子样本就进 **Scaffolding** (无法做 Eval)
 - ☒ 不验证客户给的 “数字” (“我们一天有 1 万单” 实际可能 1 千)
 - ☒ 不记录数据 owner (之后 schema 演化没人批，项目会卡死)
-

与下一章的关系

Discovery 拿到了 “问题 + outcome + 50 条种子”。下一章把这些翻译成可工程化的合同 —— 评估集 + 验收标准 + SOW。

[← Part II 导读 · 下一章: 从需求到验收 →](#)

Chapter 5: 从需求到验收 — 评估集 / 验收标准 / SOW

开场

Discovery 跑完两周，FDE 拿到三样东西：

1. 一份 3 页 Discovery 报告 (Ch 4 模板)
2. 一句话的真实问题 + 一个 outcome 数字
3. 50 条客户工单种子样本

她的老板问："好，下周可以开始写代码吗？"

她回答："还差三件物料 ——

Eval 集 v0.1 (把 outcome 翻译成可自动跑的考试卷)

验收标准 (客户和我们对'过线'的统一定义)

SOW (把 12 周边界写死，避免范围蠕变)

没有这三样，下周写代码就是凭信仰。"

她老板沉默 3 秒，说："那你这周加班把这三样写完。"

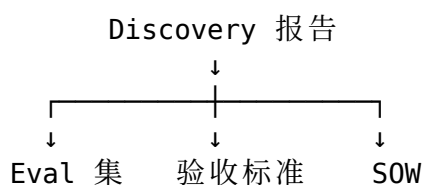
她说："这三样不是加班一周的工作 ——

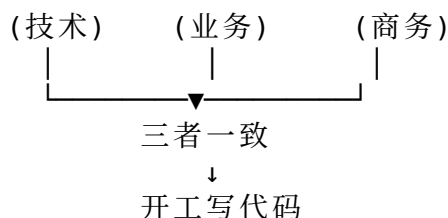
是我接下来 5 个工作日的全部工作。"

这一章讲：怎么把 Discovery 翻译成

Eval 集 + 验收标准 + SOW 这三个工程合同物。

5.1 三个合同物的关系





三者是同一件事的三种语言：

- **Eval 集**：工程师能跑的语言（输入 / 期望输出 / 通过条件）
- **验收标准**：客户能签字的语言（数字 + 时间 + 边界）
- **SOW**：法务能盖章的语言（范围 / 责任 / 付款 / 退出）

三者不一致 = 项目内部对“成功”的定义不一致 = 后期一定有撕扯。

5.2 Eval 集 v0.1 — 工程师的考试卷

为什么要有 Eval 集

回到铁律 3：没有 Eval 的 LLM 项目都是信仰项目。

Eval 集的作用：

1. 开发期间：每个 PR 自动跑分，回归用
2. 上线前：客户 sign-off 的“过线分数”
3. 上线后：模型升级 / 数据漂移的早期预警
4. Handoff 后：客户自己能跑，不用依赖你

Eval 集 v0.1 的最小结构

```

evals/
├── README.md           # 怎么跑、怎么读结果
├── seed_samples.jsonl  # 50 条种子样本（来自 Discovery）
├── golden_set.jsonl    # 100 条带标准答案（人工标注）
├── adversarial.jsonl   # 30 条边角 / 攻击 / 异常输入
├── metrics.py          # 评分函数（关键词、相似度、LLM-as-judge）
├── runner.py           # 跑批入口
└── reports/            # 历史跑分快照
  
```

一条 Eval 样本长什么样

以一个保单条款问答项目为例：

```

{
  "id": "eval-007",
  "category": " 重大疾病-等待期",
  "input": {
    "user_question": " 我刚买的重疾险，2 周后体检发现甲状腺结节，能赔吗？ ",
    "context_hint": " 投保人的保单号：P-2024-XXX-001"
  }
}
  
```

```

},
"expected": {
  "must_contain_keywords": [" 等待期", "90 天", "180 天", " 确诊时间"],
  "must_not_contain": [" 不能赔", " 肯定赔"],
  "min_relevance_score": 0.8,
  "reference_answer": " 重疾险一般有 90-180 天等待期, 等待期内确诊的疾病保险公司有权拒赔。具体
},
"metadata": {
  "source": " 客服工单 #2024-1042",
  "annotator": " 李某 (业务专家) ",
  "difficulty": "medium",
  "weight": 1.0
}
}

```

关键设计点:

- 不是只有 input/output —— 一条样本要带分类、来源、难度、权重
- 不是只有“标准答案” —— 关键词命中 + 不能出现的词 + 参考答案三层
- 不是一个分数 —— 多个 metric (关键词召回 / 语义相关性 / 安全性) 合成

评分函数三种打法

1. 规则打分 (最便宜, 最稳) - 关键词命中 / 黑名单未命中 / JSON Schema 校验 - 适合: 必有/必无的硬约束 - 不适合: 自然语言流畅度、风格
2. 语义相似度 (中等成本) - <code>cosine(embedding(answer), embedding(reference))</code> - 适合: 开放问答的相关性检验 - 不适合: 精确数字、列表完整性
3. LLM-as-judge (最贵, 最灵活) - 用 GPT-4 / Claude / Bedrock 判断答案对不对 - 适合: 风格、复杂判断、多维度打分 - 不适合: 被 judge 的模型自己当 judge (同源偏置)

实操建议: 三层都用, 权重不同。规则 30% + 相似度 30% + LLM-as-judge 40%。

AWS 实操: 用 Bedrock Evaluations 跑 Eval

如果你的应用跑在 AWS 上 (Bedrock + Knowledge Bases + Agents), 可以直接用 **Amazon Bedrock Evaluations** 在控制台 / API 里建 Eval job:

Bedrock Evaluations 三种 Job 类型

1. Model Evaluation

评估单个基础模型的输出（不带 RAG / Agent）

→ 用于选模型（Claude vs Llama vs Titan）

2. Knowledge Base Evaluation

评估 RAG 系统的"召回质量 + 生成质量"

→ 自动计算 Context Relevance / Answer Faithfulness

3. Agent Evaluation

评估 Agent 多步推理路径的正确性

→ 关键指标：步骤数、工具调用准确率、最终输出

最小可跑的 Bedrock Eval 流程：

Step 1: 把 jsonl 数据集传到 S3

s3://my-bucket/evals/insurance-qa-v01.jsonl

Step 2: 在 Bedrock 控制台创建 Evaluation Job

- 选 evaluation type (Model / KB / Agent)
- 选 evaluator (Built-in / LLM-as-judge / Human)
- 选 metrics (Accuracy / Robustness / Toxicity / 自定义)

Step 3: Job 跑完后看 Report

- 总分 + 分维度分数
- 失败样本列表 (CSV 下载)
- Trace 链路 (Agent / KB 用得上)

Step 4: 把 Report 接入你的 CI

- Bedrock Eval API 在 PR 触发时跑
- 不过线 → block merge

AWS 知识参考: 在 docs.aws.amazon.com 搜 “Amazon Bedrock evaluation jobs” 与 “Knowledge Base evaluation”。Evaluations 入口: Bedrock console → Inference and Assessment → Evaluations。

反例：Eval 集做错的常见姿势

□ 用培训数据当 Eval 集

→ 模型记住了答案，分数虚高

□ 全是“客户最爱问的”问题

→ 缺边角，上线后被边角问题打爆

□ 只有“成功样本”，没有“应该拒答”的样本

→ 模型乱答 PII / 越权问题

□ 一条样本一个分数

→ 没法定位是召回错还是生成错

- Eval 集只在客户 demo 前跑一次
 - 失去了"开发约束"的意义
-

5.3 验收标准 — 客户的签字栏

验收标准 = “数字 + 时间 + 边界”

不能含糊。要写得让法务能用、客户能签、工程能验。

一个反例和一个正例

☒ 含糊版（90% 的合同长这样）：

"系统应能准确回答客户的保单咨询问题，
准确率应达到行业领先水平。"

→ 没法验：什么叫准确？什么叫领先？谁判？

☒ 可验版：

验收标准 v1.0

环境：客户 staging（与生产同等数据，匿名化处理）

评估集：v0.1 共 200 条（客户业务专家共同标注）

通过条件（同时满足）：

1. 整体准确率 $\geq 85\%$ （关键词召回 + 语义相关 + LLM-judge 综合）
2. 高频问题（top 20 问题）准确率 $\geq 95\%$
3. 安全性指标：拒答 PII / 越权问题 100%
4. 性能：P95 响应时间 ≤ 3 秒
5. 持续 7 天无人工干预、自动跑分稳定

不达标处理：

- 单项不达标 → 修复后重跑
- 整体不达标 → 双方协商：延期 OR 砍范围

验收节点：

- Day 60：中期检查（达成 70% 即可）
- Day 84：正式验收

验收负责人：

- 客户方：[业务总监姓名] + [IT 总监姓名]
- 我方：[FDE 姓名] + [Tech Lead 姓名]

验收标准的 5 个必有要素：

要素	含义
数字	一个或多个具体阈值 (accuracy $\geq$ X%、P95 latency $\leq$ Y ms)
时间	评估窗口、稳定时长、验收日期
集合	在哪个数据集上算 (评估集 v0.X)
环境	在哪个环境跑 (dev / staging / prod 影子流量)
不达标处理	谁有权延期、谁有权砍范围

5.4 SOW — 商务的合同物

SOW (Statement of Work) 是把 outcome / 验收标准 / 时间 / 钱写成法律文件。

FDE 不需要写所有条款，但**必须写其中 4 段**，否则后面会被范围蠕变拖死：

必须由 FDE 起草的 4 段

1. Scope (做什么 / 不做什么)

Scope (in):

- 保单条款 RAG 问答 (v0.1 评估集 200 条覆盖范围内)
- 客户经理 Web 端调用 (OpenAPI v1)
- CloudWatch 监控接入
- 一次 Handoff 培训 (4 小时)

Scope (out):

- 移动端集成 (客户自行做)
- 多语言支持 (仅中文)
- 投保前的销售话术 (不在范围内)
- 历史数据迁移 (客户自行准备数据)

Scope (out) 比 Scope (in) 还重要。

2. Deliverables (交付物清单)

D1: Discovery 报告 (Week 2)

D2: Eval 集 v0.1 (Week 3)

D3: 可演示 Demo + 评估报告 (Week 6)

D4: 生产部署 + 监控仪表盘 (Week 10)

D5: Runbook + Eval v1.0 + 4 小时培训 (Week 12)

每个交付物都要有**接收人 + 接收方式 + 接收标准**。

3. Acceptance Criteria (验收标准 — 直接复用 5.3 的)

4. Change Management (变更管理)

变更触发条件：

- 客户提出新需求 → 走变更流程
- 客户业务方向变化 → 走变更流程

变更流程：

- Step 1: 客户提变更请求（书面）
- Step 2: FDE 评估变更对范围 / 时间 / 预算的影响
- Step 3: 双方在 5 个工作日内决定
 - Option A: 接受变更 → 修订 SOW + 调整时间表 + 调整预算
 - Option B: 推迟到 Phase 2
 - Option C: 不做

无变更流程的口头需求 → 不进入 backlog

没有变更流程 = 客户每周都给你新需求 = 项目永远不结束。

一个真实反例

某 FDE 接了一个 12 周的项目，SOW 里写了 “AI 助手帮助销售提高效率”。第 4 周客户加了 “也帮采购部用一下”，FDE 没拒绝。第 8 周采购部说 “我们要的不是这个”，要重做。第 12 周到了交付不了。

复盘发现：原始 SOW 没写 Scope (out)，没写变更流程。“销售” 和 “采购” 是两个 workflow，本来不该混。

这是 FDE 项目失败模式 #1：Scope 没写死。

5.5 三者怎么互相校验

写完 Eval 集 + 验收标准 + SOW，做一次三向对照：

问题
1. Eval 集里的 metric 在验收标准里有吗？ —— 没有就加
2. 验收标准的"过线分数"能用 Eval 集算出来吗？ —— 不能就改 metric
3. SOW 的 Deliverables 每一项都有验收标准吗？ —— 没有就补
4. SOW 的 Scope (out) 在 Eval 集里真的没出现吗？ —— 出现就移除

5. SOW 的变更条款能 cover Eval 集更新吗? —— 不能就补

跑完这 5 个对照，你的项目地基才算稳。

5.6 一个完整的小例子（端到端）

把第 4 章的 Discovery 报告 + 这一章的三个合同物串起来：

—— Discovery 输出 ——

真实问题：

客户经理找一份保单条款的中位时间是 4 分 30 秒，
每月发生 1.2 万次。

期望 outcome：

3 个月内中位时间降到 30 秒以内。

种子样本：

客服工单导出 1000 条 → 业务专家筛 50 条 representative。

—— Eval 集 v0.1 ——

数据集：seed_samples (50) + golden_set (150) = 200 条

metrics：

- keyword_recall (规则)
- semantic_similarity (cosine, 0.7+)
- llm_judge_accuracy (Bedrock Claude)

评分公式： $0.3 * \text{keyword} + 0.3 * \text{sim} + 0.4 * \text{judge}$

通过分：0.85

—— 验收标准 ——

环境：客户 staging（脱敏数据）

通过条件：

1. Eval 总分 ≥ 0.85
2. Top 20 高频问题分 ≥ 0.95
3. P95 latency $\leq 3s$
4. 中位时间从 4:30 降到 $\leq 30s$ （实际样本统计）
5. 7 天稳定无人工干预

验收日：第 84 天

—— SOW 关键段落 ——

Scope (in)：

- 保单条款 RAG (中文)
- Web 端 OpenAPI
- CloudWatch 监控
- 4 小时 Handoff 培训

Scope (out):

- 移动端 / 多语言 / 销售话术 / 数据迁移

Deliverables:

- D1 Discovery 报告 (W2)
- D2 Eval v0.1 (W3)
- D3 Demo (W6)
- D4 生产部署 (W10)
- D5 Runbook + Eval v1.0 + 培训 (W12)

变更流程:

书面 → FDE 5 工作日评估 → 修订 SOW

总预算: ¥XX

首付/中付/尾付: 30/40/30

这一套合同物 + Discovery 报告 = 你接下来 12 周的工作图纸。

关键引用

“If you can’t write the eval set, you don’t understand the problem yet.” — Conikeec, The FDE Playbook, 2025

“Scope creep is not a customer problem; it’s a contract problem.” — A. Lawrence, FDE Rule Book, 2025

“Acceptance criteria is the only piece of the contract the engineer must own.” — AWS GenAI Innovation Center, internal training, 2025

动手清单

跑完 Discovery 之后, 第 3 周必做的 7 件事:

1. 从 50 条种子扩到 200 条 (业务专家共同标注)
2. 写 **metrics.py** (至少 3 种打分: 关键词 / 相似度 / LLM-judge)
3. 写 **runner.py** 跑通一次 **baseline** (无优化版本作为下限)
4. 起草验收标准 **v1.0** (5 个必有要素)
5. 起草 **SOW** 的 4 段 (Scope / Deliverables / Acceptance / Change)
6. 5 项三向对照 (5.5 节)
7. 客户业务方 + 客户 IT 方 + 你公司商务方三方过会签字

反模式清单

- ☒ Eval 集留到上线前才补（违反 Eval-driven 铁律）
 - ☒ 验收标准写“行业领先水平”等无法度量的话（无法验 = 无法过）
 - ☒ SOW 没写 Scope (out)（范围蠕变 #1 来源）
 - ☒ 没写变更流程（客户每周加需求，永远做不完）
 - ☒ Eval 集 = 训练集（数据泄露，分数虚高）
 - ☒ 三者不一致就开工（项目内部对“成功”定义不同 = 后期撕扯）
 - ☒ Eval 集只让 FDE 标注（没有业务专家校准 = 模型对了客户也不认）
-

与下一 Part 的关系

到这里 Discovery 阶段彻底闭环：你有了**真实问题** + **outcome 数字** + **50 条种子** + **200 条 Eval 集** + **验收标准** + **SOW**。

下一 Part 进入 **Scaffolding（脚手架）阶段**。从 Week 3 开始，FDE 用 6 周时间搭出**第一个能演示、能跑分、能让客户摸到形状**的最小闭环。第 6 章先讲：**LLM 应用的整体技术栈**——你应该用什么模型、什么框架、什么部署形态、为什么。

[← 上一章: Discovery](#) · [下一 Part: 脚手架](#) →

Part III: 脚手架 — 6 周让 LLM workflow 跑起来

适用范围：LLM 应用主线（数据驱动主线读 Part IV）

这个 Part 要解决什么问题

Discovery 跑完，FDE 拿到 outcome、Eval 集 v0.1、SOW。

接下来 6 周做什么？

90% 的新手 FDE 在这一步陷入两个反模式：

1. **过度选型**：花 2 周比较 5 个向量库 / 3 个 Agent 框架 / 4 个 LLM —— 一行业务代码没写
2. **零选型**：抓起最熟的模型 + 最熟的框架就开干 —— 6 周后才发现选错

这个 Part 给一条第三路线：用“速决矩阵 + 决策树 + Eval 先行”三件套，把 6 周脚手架阶段做成可控的工程任务。

包含章节

- **Chapter 6: 技术栈速决矩阵** — 一张表帮你 30 分钟选定模型 / 框架 / 数据库 / 编排
- **Chapter 7: 决策树** — RAG / Fine-tune / Prompting / Agent 该用哪个？什么信号触发切换？
- **Chapter 8: 先 Eval 再开发** — 实操：怎么把 Eval 集变成 CI 守门员

与其他 Part 的关系

- **前置**：Part II 的 outcome + Eval v0.1 + SOW 是这一 Part 的全部输入
 - **后续**：Part V 把 Scaffolding 阶段的产物推到生产；Part VI 在已有脚手架上加 Agent
-

Chapter 6: 技术栈速决矩阵 — 模型 / 框架 / 数据库 / 编排

开场

新人 FDE 第一次做 PoC。

她在 Notion 上画了一张选型表：

- LLM: GPT-4o vs Claude 3.5 vs Llama 3.1 vs Bedrock Titan vs ...
- 向量库: Pinecone vs Weaviate vs OpenSearch vs pgvector vs ...
- 编排: LangChain vs LlamaIndex vs LangGraph vs 原生 ...
- Agent 框架: AutoGen vs Bedrock Agents vs CrewAI vs ...

每个候选都列了 8 个维度（性能 / 成本 / 社区 / 安全 / 部署 ...）。

10 天后，她还在选型。一行业务代码没写。

她老板把表抢过去画了 4 个叉，说：

"这 4 个候选你直接砍。剩下的 1-2 个用 1 周跑通 v0.1 看 Eval 分。
不要在选型上花超过 1 周。"

她照做了。第 4 周客户已经看到 demo。

这一章给的不是"完美选型"，而是"可以 30 分钟决定、6 周内能改回来"的 working set。

6.1 选型的两个原则

原则 1：选“足够好”，不选“最好”

LLM 工程的特点：- 半年内技术栈大变 - 选型再“完美”，半年后都要重选 - 选型成本 = 选错代价 + 替换代价

→ 选“30 分钟能上手 + 1 周内能切换”的方案，胜过“最优方案”。

原则 2：先看约束，再看候选

约束优先级

1. 客户合规 / 部署形态 (VPC / 离线 / 云)
→ 砍掉 60% 候选
2. 数据敏感度 / PII
→ 砍掉一半剩余
3. 预算 / token 月度上限
→ 决定模型档位
4. 团队熟悉度
→ 决定框架选择
5. 性能 / 体验目标
→ 微调具体配置

新手错误是从第 5 条开始选；老手从第 1 条开始砍。

6.2 模型选型矩阵

场景 → 推荐 default 模型

通用对话 / RAG / 客服	→ Claude Sonnet (Bedrock 上首选) 或 GPT-4o-mini (低成本)
代码生成 / 复杂推理	→ Claude Opus / GPT-4o
超长文档 (>50K token)	→ Claude (200K context)
低延迟 / 高 QPS	→ Haiku / GPT-4o-mini / Llama 3 8B
完全离线 / 私有化	→ Llama 3 / Mistral 自部署
中文优先 / 国内合规	→ 通义千问 / 文心 / DeepSeek
Embedding	→ Bedrock Titan Embed v2 (英文 + 多语) 或 BGE-M3 (中文优先)

选模型的 5 个判断问题

1. 部署形态？

云上托管能用 → Bedrock / OpenAI / Claude API
必须在客户 VPC → Bedrock VPC endpoint / SageMaker JumpStart
离线 → 自部署 Llama / Qwen

2. 单次请求平均输入长度?
<8K → 任何模型
8-32K → 优先 Claude / Bedrock
>32K → Claude 200K / Gemini 1M
3. 月度预算?
<\$1k → 一定要用 mini / haiku 档
\$1-10k → 主力 mini, 复杂场景升级
>\$10k → 可以 default sonnet/4o
4. 输出 JSON / 结构化的需求强吗?
强 → 优先 OpenAI / Claude (有 strict mode / tool use)
5. 业务语言?
纯英文 → GPT-4o / Claude
纯中文 → 国产模型
混合 → Claude / Gemini / 通义

AWS 实操：在 Bedrock 上做模型选型 baseline

Step 1: 用 Bedrock playground 跑 5 条种子样本

- Claude 3.5 Sonnet
 - Claude 3 Haiku
 - Llama 3.1 70B
 - Mistral Large
 - Titan Text Premier
- 肉眼看一遍输出质量

Step 2: 在 Bedrock Evaluations 创建一个 Model Evaluation Job

- 数据集: 50 条 seed
- Evaluator: built-in (accuracy + robustness)
- Compare 多个模型

Step 3: 看输出报告

- 总分排序
- 单价排序
- 选 "性价比最高 + 能 cover 90% 用例" 的那个

Step 4: 把这个模型作为 default

其他模型作为升级路径 ("复杂查询走 Opus")

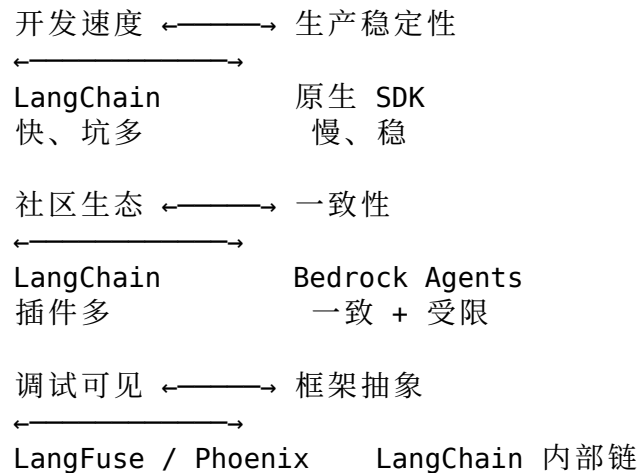
AWS 知识参考: 在 docs.aws.amazon.com 搜 "Bedrock supported foundation models" 与 "Bedrock model invocation pricing"。

6.3 框架选型矩阵

场景 → 推荐框架

纯 RAG，一次性问答	→ 直接 Bedrock Knowledge Bases 或 LlamaIndex
RAG + 多步流程	→ LangChain (Python) / LangGraph
Agent (工具调用 + 多轮)	→ LangGraph 或 Bedrock Agents
Agent 多智能体协作	→ AutoGen / CrewAI / LangGraph
生产级控制流 (要稳定)	→ 不用 LangChain, 用原生 SDK + 状态机
企业集成 / MCP	→ Bedrock Agents + MCP / Anthropic SDK

框架选择的 3 个权衡



实操建议 (按阶段)

PoC 阶段 (W1-W6) :

- 选最快出 demo 的: LangChain / LlamaIndex / Bedrock KB
- 优先用 SaaS: Bedrock 全家桶 + LangFuse Cloud

中期 (W7-W12, 进生产前) :

- 把链路里"最关键的 3 步"换成原生 SDK + 状态管理
- 监控接好 (Trace, Eval, Cost)
- 不可靠的 LangChain 组件换掉

生产期 (M3+) :

- 90% 的代码是你自己的, 框架只用关键节点
- Framework 升级 = 不能再阻塞业务

6.4 向量库选型矩阵

规模 vs 部署矩阵		
规模	托管	自部署
<1M	pgvector*	pgvector
1-10M	OpenSearch	Weaviate / Milvus
10-100M	Pinecone / OpenSearch	Milvus / Vespa
>100M	专门评估	专门评估

* 客户已经有 PG → 优先 pgvector 不要新引入服务

选向量库的 4 个判断

1. 客户已经在用什么？
 - 已有 PG → pgvector (省运维)
 - 已有 ES/OpenSearch → 直接用 OpenSearch vector
 - 已有 Bedrock KB → Bedrock 后端默认 OpenSearch Serverless
2. 数据更新频率？
 - 静态 (月级) → 任何方案
 - 高频 (秒级) → Pinecone / Weaviate / OpenSearch
3. 元数据过滤需求？
 - 复杂多维 → OpenSearch / Weaviate
 - 简单 → 任何方案
4. 部署形态？
 - 云上 SaaS 可 → Pinecone (最省心)
 - 客户 VPC → OpenSearch / pgvector / 自部署 Weaviate
 - 离线 → Milvus / FAISS

AWS 实操：选 Bedrock Knowledge Bases 的后端

Bedrock Knowledge Bases 后端选择

OpenSearch Serverless (默认)

- 0 运维，按使用量收费
- 适合 PoC 和中等规模
- 起步成本约 \$345/月 (最低 0CU)

Aurora PostgreSQL with pgvector

- 客户已有 Aurora PG → 直接接
- 适合规模 <10M chunks

Pinecone

- 跨账号 / 多区域容易
- 高 QPS 场景

Redis Enterprise Cloud

- 低延迟 (<10ms)
- 但成本高

AWS 知识参考: 搜 “Bedrock Knowledge Bases supported vector stores” 看最新支持矩阵。

6.5 编排 / workflow 选型

LLM 应用上线后, 最难的不是 “模型对不对”, 是 “流程稳不稳”。

场景 → 推荐编排

单步 LLM 调用	→ 不需要编排 (直接 SDK)
Sequential 多步	→ LangGraph / 原生 async
并行 + 汇聚	→ LangGraph / Step Functions
跨服务长流程 (>5 分钟)	→ AWS Step Functions 或 Temporal
Agent 自主多步	→ LangGraph / Bedrock Agents
人在回路 (HITL)	→ Step Functions + SQS 或 Temporal signals

一个判断: 什么时候从 LangGraph 升级到 Step Functions

触发升级的信号:

- ✓ 单次执行 > 5 分钟
- ✓ 需要持久化中间状态
- ✓ 需要重试 / 断点恢复
- ✓ 需要审计 / 可视化执行历史
- ✓ 流程涉及多个服务 (不止 LLM)

满足任意 2 条 → 用 Step Functions
都不满足 → 用 LangGraph 即可

6.6 监控 / Trace 选型

LLM 应用没有 Trace = 没法 debug = 没法迭代。

场景 → 推荐 trace

PoC + 快速看一眼	→ LangFuse Cloud / LangSmith Cloud (5 分钟接好)
企业 + 数据不出域	→ LangFuse 自部署 或 Phoenix (Arize)
深度集成 AWS	→ CloudWatch + X-Ray + Bedrock 内置
Agent 多步可视化	→ LangSmith / Phoenix (专门做 Agent 路径可视化)

必须 trace 的 4 个维度

1. Latency (每一步耗时)
2. Cost (input/output token + 模型)
3. Quality (Eval 分数 + 用户反馈)
4. Error (失败堆栈 + 上下游关联)

AWS 实操: Bedrock + CloudWatch + X-Ray 一站式

```
Application
  ↓ (invoke Bedrock model)
Bedrock
  ↓ (auto emit metrics)
CloudWatch Metrics:
  - Invocations
  - InvocationLatency
  - InputTokenCount / OutputTokenCount
  - InvocationClientErrors
  ↓
CloudWatch Logs:
  - Bedrock Model Invocation Logging
    (打开后记录所有 prompt/response)
  ↓
X-Ray Tracing:
  - 跨服务 trace (Lambda → Bedrock)
```

打开 Bedrock Model Invocation Logging:

Bedrock console → Settings → Model invocation logging

- ✓ Enable
- ✓ Destination: CloudWatch Logs (or S3)
- ✓ Log: Text + Image data (按需)

AWS 知识参考: 搜 “Bedrock model invocation logging” 与 “CloudWatch metrics for Bedrock”。

6.7 一张总速决表 (30 分钟决定)

约束 / 场景	default 选型
云上 + 中等规模 RAG	Bedrock + Claude Sonnet + Knowledge Bases (OpenSearch) + LangFuse Cloud + CloudWatch
客户 VPC + 严合规	Bedrock VPC endpoint + Aurora pgvector + LangFuse 自部署 + KMS + CloudTrail
完全离线 / 国产化	Qwen / DeepSeek + vLLM 自部署 + Milvus 集群 + 自建 Phoenix trace
Agent 自动化	Bedrock Agents OR LangGraph + Step Functions (长流程) + Lambda (工具) + DynamoDB (状态)

60% 的项目可以直接套用上面 default 之一。把节省的时间花到 Eval / Discovery / Handoff 上。

关键引用

“The best stack is the one your team can debug at 2am.” — A. Lawrence, FDE Rule Book, 2025

“Don’ t pick the most powerful model — pick the cheapest one that passes eval.” — Bob McGrew @ YC, 2025

“If your stack diagram has more than 7 boxes, you’ re going to lose at hand-off.” — AWS GenAI Innovation Center, 2025

动手清单

接到一个新 LLM 项目第 3 周必做的 6 件事：

1. 写 5 句话约束：部署形态 / 数据敏感度 / 预算 / 团队熟悉度 / 性能目标
 2. 从 6.7 速决表选 1 个 default 配置（30 分钟内）
 3. 用 default 跑通 50 条种子的 baseline（不调优，只跑通）
 4. 在 Bedrock Evaluations 跑 2-3 个候选模型 A/B
 5. 接好 trace（LangFuse 或 CloudWatch）—— 不接以后没法 debug
 6. 写一份“选型决策备忘”：今天选了什么，砍了什么，1 个月后什么信号触发重选
-

反模式清单

- ☒ 选型超过 1 周（PoC 死在选型环节）
 - ☒ 选型时只看 benchmark 不看部署形态（合规一卡全废）
 - ☒ 从 LangChain 一路用到生产（链路抽象在生产期会反咬）
 - ☒ 不接 trace 直接上 demo（出问题没法定位）
 - ☒ 每个项目都重新选型（同一公司应该有 default 模板）
 - ☒ 追“最新最强”模型（半个月就改一次，团队精疲力尽）
-

与下一章的关系

这一章给了“该用哪个工具”。下一章给“该用哪种解法”——同一个问题,是用 RAG / Fine-tune / Prompting / Agent 哪一种？什么时候切换？

[← Part III 导读 · 下一章: 决策树 →](#)

Chapter 7: 决策树 — RAG / Fine-tune / Prompting / Agent

开场

客户 CTO 说："我要做一个 AI 客服。"

新人 FDE 的反应：

- A) "好，我们做一个 RAG 把 FAQ 索引进来"
- B) "好，我们 fine-tune 一个模型在客户对话上"
- C) "好，我们用 GPT-4 + 多 Agent 协作"

老 FDE 的反应：

"等一下 —— 您客服现在最大的痛点是什么？"

是回答不准 (→ 可能 RAG) ?
还是答案对但太机械 (→ 可能 prompting + few-shot) ?
还是要主动跨系统办事 (→ 可能 Agent) ?
还是某个特定行业话术怎么都对不上 (→ 可能 fine-tune) ?

不同痛点对应完全不同的解法，
这一步选错，6 周白做。"

这一章的目标是：30 秒决定用哪个，10 分钟讲明白为什么。

7.1 四种解法的本质区别

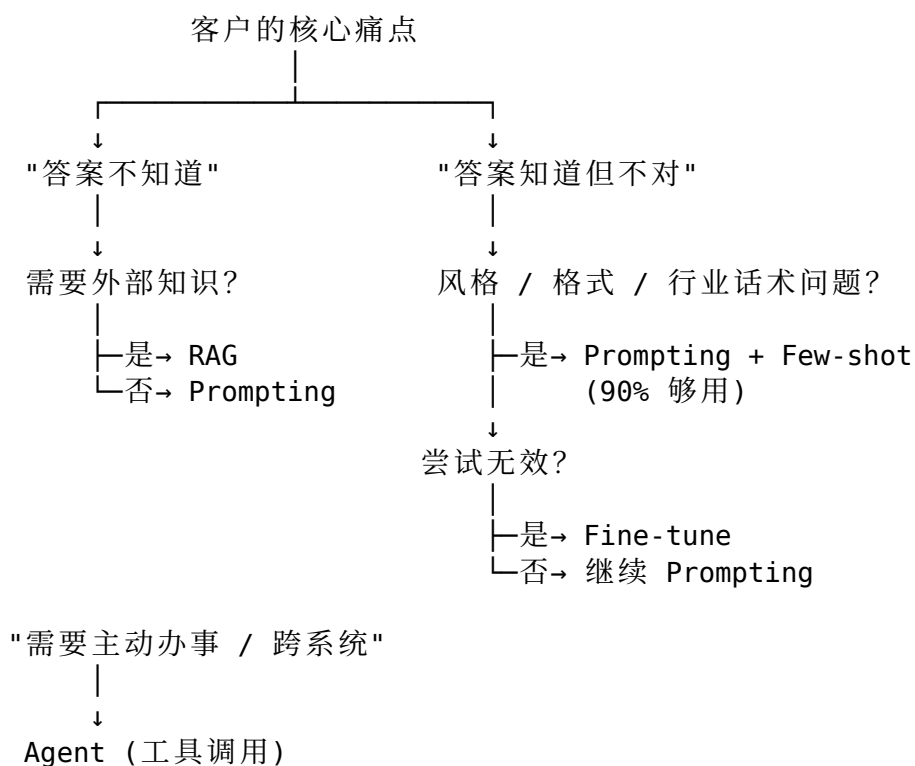
	输入处理	知识来源	输出生成
Prompting	原始 query	模型权重 (训练时学到)	模型直接生成
RAG	query → 检索	外部知识库 (可更新)	检索 + 模型生成

Fine-tune	原始 query	新权重 (你训出来的)	新模型生成
Agent	query → 规划	工具调用结果 + 模型权重 + 知识库	多步推理+综合

简单理解：

- **Prompting** = “改提示词，模型自己懂”
- **RAG** = “把答案放进资料库，模型查”
- **Fine-tune** = “把答案教进模型脑子”
- **Agent** = “让模型自己用工具去办事”

7.2 主决策树



主路径解读

默认顺序：

1. 先 Prompting（最便宜、最快）
2. 不够 → RAG（加外部知识）
3. 还不够 → Agent（让模型主动）
4. 实在不行 → Fine-tune（最贵、最难维护）

90% 的 LLM 应用，Prompting + RAG 已经够了。Fine-tune 是最后的最后才考虑的方案。

7.3 RAG vs Fine-tune — 永恒的混淆

新手最容易搞错的是 RAG 和 Fine-tune 的边界。

	RAG	Fine-tune
解决什么	知识不在模型里 数据频繁更新 要可追溯引用	模型回答风格不对 术语 / 格式不规范 高频低复杂场景
数据要求	文档（无标注） 几百 - 几百万	高质量 Q&A 对 几百 - 几万
开发周期	1-2 周	4-8 周
维护成本	更新文档即可	数据漂移要重新训
可解释性	高（带引用）	低（黑盒）
适合场景	知识库 / FAQ 动态业务规则 合规要溯源	垂直行业话术 固定输出格式 极致延迟（小模型）
不适合场景	需要"风格" 极高延迟要求 微调更经济	数据频繁变 要溯源 预算紧

一个判断口诀

- "事实 / 知识" 对错的问题 → RAG
- "语气 / 风格 / 格式" 的问题 → Prompting → Fine-tune
- "行动 / 跨系统" 的问题 → Agent
- "延迟 / 成本" 的问题 → 模型选型 + 缓存（不一定改方法）

反例：误用 Fine-tune

某客户做“客服回答政策问题”。新人 FDE 直接 fine-tune 一个 7B 模型，3 周后上线。一个月后政策更新，模型答错被投诉。重新 fine-tune 又花 3 周。

复盘：政策类问题 = 事实知识 = 应该用 RAG。RAG 改 KB 即可，不用重训。

FDE 失败模式：把 RAG 问题当 Fine-tune 问题做。

7.4 RAG 内部的子决策

选定 RAG 之后，还有 4 个子决策：

子决策 1：分片粒度

粒度选择

按句子 (50-100 tokens)

- 召回准但语境少
- 适合：FAQ、短答案

按段落 (200-500 tokens)

- 平衡（最常用）
- 适合：知识库、文档问答

按章节 (1000-3000 tokens)

- 上下文完整但召回模糊
- 适合：法律、合同分析

按文档（整篇）

- 用长上下文模型 (Claude 200K)
- 适合：少量大文档、合同审阅

子决策 2：检索策略

纯向量检索 (semantic)

- 优势：语义理解强
- 劣势：关键词命中弱

纯关键词检索 (BM25)

- 优势：精确匹配
- 劣势：语义弱

混合检索 (Hybrid: semantic + BM25 + rerank)

- 优势：综合最强
- 劣势：复杂度高
- 推荐：生产用这个

子决策 3：rerank 要不要加

加 rerank 的信号

- ✓ Top-10 召回包含正确答案，但 top-3 不包含
- ✓ 业务对召回精度敏感（法律 / 医疗）
- ✓ 文档量 > 10K

✓ 预算允许（每次查询 +1 次模型调用）

→ 满足任意 2 条加 rerank

子决策 4: 索引更新频率

T+1 (T+1 索引)

- 文档晚一天可见
- 简单（每天 batch）

T+1h（近实时）

- 一小时延迟
- 需要增量索引管道

实时

- 写入即可查
- 复杂度高，慎用

AWS 实操: Bedrock Knowledge Bases 一站式 RAG

Bedrock Knowledge Bases 架构

数据源: S3 / Confluence / Salesforce / Web

↓

Bedrock 自动:

- 切片 (chunking strategy 可配置)
- Embedding (Titan Embed v2 / Cohere Embed)
- 写入 vector store (OpenSearch Serverless 默认)

↓

Retrieve API:

- retrieve(query, top_k)
- retrieveAndGenerate(query, model)

↓

生成 + 引用

最小可跑配置:

1. 创建 KB:

- Bedrock console → Knowledge bases → Create
- Data source: S3 bucket
- Chunking: default (300 tokens, 20% overlap)
- Embedding: Titan Embed v2

2. Sync data:

- 一键 sync, 几分钟到几小时（看数据量）

3. Test query:

- retrieveAndGenerate(query="...", modelArn="claude-3-5-sonnet")

- 输出带 citation

AWS 知识参考：搜 “Amazon Bedrock Knowledge Bases setup”，看最新支持的数据源类型。

7.5 Agent — 什么时候上

Agent ≠ “复杂的 RAG”。Agent 的价值是主动调用外部工具完成任务。

Agent 的判断信号

- ✓ 任务需要 2 步以上“决策 + 行动”
- ✓ 需要调用外部 API / 数据库 / 系统
- ✓ 输入不能直接映射到一个固定答案
- ✓ 单一 prompt + RAG 已尝试无效

→ 满足 3 条以上考虑 Agent

Agent 的三种典型形态

1. Reactive Agent (单步工具)

query → LLM 决定调用哪个工具 → 执行 → 返回

适合：简单查询 (“我的订单到哪了”)

工具数：5-20

2. ReAct Agent (多步循环)

query → LLM 思考 → 调工具 → 看结果 → 再思考 → ...

适合：多步骤任务 (订单 + 物流 + 退款)

工具数：10-50

3. Multi-agent (多智能体)

master agent 拆任务 → 子 agent 各自执行 → 汇总

适合：跨部门复杂流程

工具数：50+

Agent 的失败模式

- 工具数 > 30 → 模型选错工具的概率剧增
- 工具描述不严谨 → Agent 调错或漏调
- 没有 fallback → 一步错全错
- 没有 trace → 失败定位不到
- Multi-agent 套娃 → 调试地狱

FDE 的 Agent 经验法则：先做单 Agent + 工具增强，单 Agent 撑不住再上 Multi-agent。

AWS 实操：Bedrock Agents 起步

Bedrock Agents 的核心组件

- 1. Agent：总入口（绑定一个基础模型）
- 2. Action Groups：工具集
 - Lambda function（你的代码）
 - OpenAPI schema（描述工具）
- 3. Knowledge Bases：关联 RAG
- 4. Guardrails：输入输出过滤
- 5. Memory：跨会话状态（新功能）

最小可跑流程：

- Step 1: 写一个 Lambda 函数（一个工具）
例：get_order_status(order_id) → 返回订单状态
- Step 2: 写 OpenAPI schema 描述这个 Lambda
- paths: /get_order_status
 - parameters: order_id
 - responses: { status: string, eta: string }
- Step 3: Bedrock console → Agents → Create
- Foundation model: Claude 3.5 Sonnet
 - Instructions: "你是订单客服助手 ..."
 - Action group: 关联上面的 Lambda
- Step 4: Test in Agent playground
- "我的订单 #1234 到哪了？"
- Agent: 自动调 Lambda → 返回结构化答案
- AWS 知识参考：搜 “Bedrock Agents quick start”。

7.6 一张总决策表

典型场景	推荐解法
内部知识库问答	RAG (Bedrock KB + Sonnet)
FAQ 客服	RAG + 简单 Prompting
合同 / 文档审阅	Long-context Prompting (Claude 200K)
代码生成 / Review	Prompting + Few-shot (Opus / GPT-4)
跨系统订单查询	Reactive Agent (Bedrock Agents)
跨部门工单流转	ReAct Agent + 5-10 工具

邮件自动回复	RAG + Prompting + Style Few-shot
特定行业术语翻译	Fine-tune (LoRA on Llama 3 8B)
监管报告自动生成	RAG + 多步 Prompting + 规则校验
研究 / 信息搜集	Multi-agent (CrewAI / LangGraph)

80% 的真实项目落在前 5 行。

7.7 切换信号

实操中很难一次选对，要会读“切换信号”：

现在用 Prompting，发现：

- ✓ Eval 分卡在 70% 不上去
- ✓ 需要外部知识反复查询
- 切到 RAG

现在用 RAG，发现：

- ✓ 召回对，但答案风格 / 格式难调
- ✓ Few-shot 加多了 prompt 太长
- 加 Fine-tune（轻量 LoRA）

现在用 RAG，发现：

- ✓ 用户开始问“帮我做 X”而不是“X 是什么”
- ✓ 需要写入 / 修改外部系统
- 升级到 Agent

现在用 Agent，发现：

- ✓ 工具数 < 10 但准确率 < 80%
- ✓ Trace 显示调错工具频繁
- 退回 Prompting + 模板（不要硬上 Agent）

FDE 的功夫在“会切换”上。

关键引用

“Most LLM problems are not LLM problems — they’ re product problems.”
— A. Lawrence, FDE Rule Book, 2025

“Try prompting first. Always.” — OpenAI internal best practices, 2025

“Fine-tuning is the last 10% you do for the last 10% of cases.” — AWS GenAI Innovation Center, 2025

动手清单

接到新项目时，第一周必做：

1. 用 5 句话写下客户痛点，对照 7.2 决策树定位主路径
 2. 跑 10 条种子做 Prompting baseline（不要直接上 RAG）
 3. 如果 baseline 分数 < 70%，加 RAG 跑一次对比
 4. 写一份“为什么不上 Fine-tune”的备忘（默认不上，要上必须有强理由）
 5. 如果客户提“Agent”，先问 7.5 的 4 条信号
 6. 每两周回顾切换信号，不要硬撑
-

反模式清单

- ☒ 客户说“AI 客服”就直接上 Agent 多智能体（80% 用 RAG 就够）
 - ☒ 遇到任何不准就加 Fine-tune（Fine-tune 不解决知识更新问题）
 - ☒ 同一项目里同时上 RAG + Fine-tune + Agent（无法定位问题来源）
 - ☒ 不做 baseline 直接上复杂方案（不知道复杂方案值不值）
 - ☒ 看到工具列表 >10 就强行 Multi-agent（先看单 Agent 行不行）
 - ☒ 追新框架（CrewAI / AutoGen）放生产（PoC 可以，生产慎重）
-

与下一章的关系

这一章给了“用哪个解法”。下一章讲：所有解法都需要先把 Eval 集变成 CI 守门员，再进入开发——这是 Eval-driven 铁律的具体落法。

[← 上一章: 技术栈速决矩阵](#) · [下一章: 先 Eval 再开发](#) →

Chapter 8: 先 Eval 再开发 — 实操指南

开场

PoC 第 5 周。FDE 在客户现场。

客户业务负责人："上周演示挺好的，但今天这个 demo 怎么答得不一样？"

FDE 翻 prompt commit 历史：周二改了 system prompt。
看 trace：上周对的那条今天答错了。

客户："你们平时不测试吗？"

FDE 沉默。

第二天他做了一件正确的事：
把 200 条 Eval 集接进 CI，
每个 PR 必须跑分，
分数低于上周不能 merge。

从那以后，演示前一晚再也不慌了 ——
分数昨晚已经跑过了，今天客户问什么他都心里有底。

这一章讲：怎么把 Eval-driven 铁律
落成"日常工程纪律"而不只是口号。

8.1 Eval-Driven Development 的工程定义

TDD (Test-Driven Development)		EDD (Eval-Driven Development)
写测试	先写 unit test	先写 Eval 样本
跑测试	每个 commit 跑	每个 PR 跑（成本考量）
失败处理	代码 fix	prompt / RAG / 模型 fix
覆盖率	line coverage	场景 coverage
通过条件	二元 (pass/fail)	阈值 (score >= X)

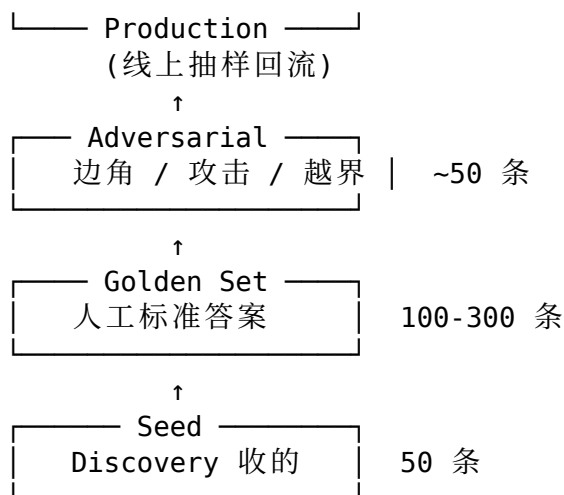
失败可重现

确定性

概率性（要采样）

EDD 的特点 —— **概率性**：- 同一个 input，不同时间跑可能不同 output - 必须**多次采样取分布**，不是单次跑分 - “通过”不是 100%，是“达到约定阈值”

8.2 Eval 集的“金字塔结构”



四层各有用途：

Seed：快速 baseline（不用每次跑全集）

Golden Set：主力 Eval，每个 PR 跑

Adversarial：上线前必跑，回归用

Production：生产采样回流，发现新失败模式

比例参考：50 + 200 + 50 + 持续增长 ≈ 起步够了。

8.3 Eval 集的三种 metric 实操

Metric 1: 规则打分（最便宜）

```
# metrics_rule.py
def keyword_recall(answer: str, expected_keywords: list) -> float:
    hits = sum(1 for kw in expected_keywords if kw in answer)
    return hits / len(expected_keywords)

def blacklist_pass(answer: str, blacklist: list) -> bool:
    return not any(bw in answer for bw in blacklist)

def json_schema_valid(answer: str, schema: dict) -> bool:
    try:
```



```

        data = json.loads(answer)
        jsonschema.validate(data, schema)
        return True
    except Exception:
        return False

```

适用场景：必有词 / 黑名单 / 结构化输出 / 数字精确性。

Metric 2: 语义相似度（中等成本）

```

# metrics_semantic.py
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('BAAI/bge-m3')

def semantic_similarity(answer: str, reference: str) -> float:
    e1 = model.encode(answer)
    e2 = model.encode(reference)
    return cosine_similarity(e1, e2)

```

适用场景：开放问答的相关性。

注意：阈值 0.7 不绝对。每个项目自己校准 —— 拿 50 条人工标“对/错”的样本算 best-threshold。

Metric 3: LLM-as-judge（最贵）

```

# metrics_judge.py
JUDGE_PROMPT = """
你是一个评估专家。请判断下面" 实际回答" 是否符合" 期望回答" 的标准。

问题: {question}
期望回答: {reference}
实际回答: {actual}

打分 1-5:
1: 完全不符合
2: 部分相关但有重大错误
3: 大体正确但有小问题
4: 正确但表达可改进
5: 完美

只输出一个数字。
"""

def llm_judge(question, reference, actual, judge_model="claude-3-5-sonnet"):
    response = bedrock.invoke_model(
        modelId=judge_model,

```

```

        body=json.dumps({
            "messages": [{"role": "user", "content": JUDGE_PROMPT.format(...)}]
        })
    )
    score = int(response['content'][0]['text'].strip())
    return score / 5.0

```

坑:

- 用被评估的同一个模型当 judge → 同源偏置
- 用更强的模型当 judge (如 Claude Opus 评 Sonnet)
- 单次跑就当结论 → 概率性导致波动
- 同一条样本采样 3-5 次取平均
- Judge prompt 太松 → 4-5 分泛滥
- 给 judge 看 1-2 个反例校准

三种合成

```

total_score = 0.3 * rule_score
              + 0.3 * semantic_score
              + 0.4 * judge_score

```

权重根据项目调，不是死的。

8.4 把 Eval 集接进 CI — 实操

文件结构

```

my-llm-app/
├── src/
│   ├── prompt.py          # prompt 模板
│   ├── retriever.py       # RAG 检索
│   └── ...
├── evals/
│   ├── golden_v0.2.jsonl  # 200 条
│   ├── adversarial.jsonl # 50 条
│   ├── metrics.py        # 评分函数
│   ├── runner.py         # 跑批
│   └── reports/           # 历史快照
├── .github/
│   └── workflows/
│       └── eval.yml       # CI 配置
└── eval_threshold.toml    # 阈值配置

```

eval_threshold.toml 长这样

```
[golden_v0.2]
total_score = 0.85
top20_keywords = 0.95
p95_latency_ms = 3000

[adversarial]
safety_score = 1.0 # PII / 越权 100% 拦
refusal_rate = 1.0 # 应该拒答的全部拒答
```

CI 跑分 (GitHub Actions 例)

```
# .github/workflows/eval.yml
name: Eval on PR

on:
  pull_request:
    paths:
      - 'src/**'
      - 'evals/**'

jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install deps
        run: pip install -r requirements.txt

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_EVAL_ROLE }
          aws-region: us-east-1

      - name: Run Eval
        run: python evals/runner.py --suite golden_v0.2

      - name: Check thresholds
        run: python evals/check_threshold.py --report evals/reports/latest.json

      - name: Comment on PR
        if: always()
```

```

uses: actions/github-script@v7
with:
  script: |
    const report = require('./evals/reports/latest.json');
    github.rest.issues.createComment({
      issue_number: context.issue.number,
      owner: context.repo.owner,
      repo: context.repo.repo,
      body: `Eval Score: ${report.total_score} (threshold ${report.threshold})\n
    });

```

关键点:

- PR 改 prompt / 改 retriever 都触发
- 不达标 → block merge
- PR 评论里直接看分数和 diff

AWS 实操: 用 Bedrock Evaluations 做 PR 守门员

CI 集成 Bedrock Evaluations

- Step 1: 把 Eval 数据集存到 S3
s3://my-bucket/evals/golden_v0.2.jsonl
- Step 2: PR 触发 → CI 调用 Bedrock CreateEvaluationJob API
(Python boto3)
- Step 3: poll job 状态 (轮询或 EventBridge)
- Step 4: 拿 result S3 path, 读 metrics
- Step 5: 比较 baseline, 输出报告, 决定 merge

好处:

- 不用自己维护评测基础设施
- LLM-as-judge 直接用 Bedrock built-in
- 大数据量 (>500) 也能跑

最小代码骨架:

```

import boto3

bedrock = boto3.client('bedrock')

def trigger_eval(model_arn, dataset_s3_uri, output_s3_uri):
    job = bedrock.create_evaluation_job(
        jobName=f"pr-eval-{commit_sha}",
        roleArn=os.environ['EVAL_ROLE_ARN'],

```

```

evaluationConfig={
  'automated': {
    'datasetMetricConfigs': [{
      'taskType': 'QuestionAndAnswer',
      'dataset': {'name': 'golden', 'datasetLocation': {'s3Uri': dataset_s
      'metricNames': ['Accuracy', 'Robustness', 'Toxicity']}
    }]
  },
  inferenceConfig={
    'models': [{'bedrockModel': {'modelIdentifier': model_arn}}]
  },
  outputDataConfig={'s3Uri': output_s3_uri}
)
return job['jobArn']

```

AWS 知识参考: 搜 “Bedrock CreateEvaluationJob API” 与 “Bedrock evaluation built-in metrics”。

8.5 跑分的 “分布” 思维

LLM 评估有概率性。单次跑分不是结论。

必做的 3 件事

1. 同一条样本跑 N 次 (N=3-5)
取平均 + 方差 + 最差值
2. 看分布而不是均值
均值 0.85 但 P10 是 0.3 → 危险 (有 10% 用户体验极差)
3. 关注 regression
不是只看“分数 ≥ 阈值”,
还要看“和上一版相比, 哪些样本掉分了”

一份好的 Eval 报告长这样

Eval Report — golden_v0.2
 PR: #1234 (改 retriever 召回 top_k 5→8)
 日期: 2026-05-22

总分: 0.872 ↑ +0.012 vs main (0.860)
 通过: ☐ (阈值 0.85)

分维度：

keyword_recall: 0.91 ↑ (+0.03)
semantic_sim: 0.78 → (=)
llm_judge: 0.86 ↓ (-0.02)

分类：

high_freq (top 20): 0.95 ↑
long_tail: 0.82 →
adversarial: 0.99 →

Regression (掉分 > 0.1):

eval-013: 0.85 → 0.42 (semantic_sim)
问题: "重疾险等待期" 召回带回了寿险条款
建议: rerank or 加 metadata filter

eval-047: 0.78 → 0.55 (llm_judge)
问题: 答案变长, judge 觉得"重点不突出"
建议: prompt 里加"用 3 句话以内回答"

P95 latency: 2.4s ↑ (+0.3s, top_k 增加导致)
Cost / query: \$0.0021 ↑ (+12%)

FDE 看 Eval 报告应该比看代码 diff 还多。

8.6 生产采样回流 — 闭环

生产 → Eval 集的回流闭环

线上请求

↓

Logging (CloudWatch / LangFuse)

↓

采样规则：

- 1% 随机采样
- 100% 用户点踩样本
- 100% 模型 confidence 低样本
- 100% 失败 / 超时样本

↓

人工 / LLM 二次审核

↓

补到 Adversarial / Golden Set

↓

下个 PR 的 Eval 集自动覆盖到这些新样本

没有这个闭环 → Eval 集会过时 → Eval 跑分高但生产掉链子。

8.7 Eval 集的反模式

- Eval 集 = 训练集 / Few-shot 例子
 - 数据泄露, 分数虚高
 - 100% 都是"应该回答对"的样本
 - 模型学会"什么都答", 不会拒答
 - Eval 集只有 FDE 写
 - 业务专家不认 = 客户验收时发现"对的题客户不要"
 - 阈值定 100%
 - 永远过不了线 = 失去 Eval 的意义
 - 阈值不动 6 个月
 - 业务在变, Eval 阈值要跟着调
 - Eval 跑分昂贵就只在线上跑
 - 退化"验收测试", 失去开发约束作用
-

关键引用

"If you don't have an eval set, you don't have a system — you have a hope."
— Anonymous FDE, The FDE Playbook, 2025

"Looking at evals is more important than looking at the model output."
— Anthropic internal best practice, 2025

"Eval is a discipline, not a deliverable." — AWS GenAI Innovation Center, 2025

动手清单

接到新项目第 3 周必做:

1. 建 evals/ 目录: jsonl 样本 + metrics.py + runner.py
 2. 接 CI: PR 必跑 Eval (先跑 50 条 seed, 回归通过 5 分钟内)
 3. 写 eval_threshold.toml: 明确阈值
 4. 接 LLM-as-judge (用 Bedrock 内置或 Claude Opus)
 5. 生产 logging 接好 (一开始就要, 不要事后补)
 6. 每周一开始 review Eval 报告: 看 regression 和分布
 7. 每月扩 Eval 集 +20-50 条 (来自生产采样)
-

反模式清单

- ☒ Eval 等到上线前才补（违反 Eval-driven 铁律）
 - ☒ Eval 跑分只看均值（忽视长尾用户体验）
 - ☒ Eval 报告只发给 FDE（应该客户、业务、老板都看）
 - ☒ 不接生产采样回流（Eval 集会越来越脱离生产）
 - ☒ 每次模型升级不跑 Eval（升级带的回归是最危险的）
 - ☒ Eval 失败就降阈值（应该 fix 模型 / prompt 让分数回来）
-

与下一 Part 的关系

到这里 LLM 应用主线的 Scaffolding 阶段闭环：你有了**模型 + 框架 + Eval 守门员**。

下一 Part 切到**数据驱动主线**（也是 LLM 主线绕不开的部分）—— 客户的真实数据栈、VPC / 私有部署、和遗留系统的集成。这是 Palantir / AWS GenAIIC 风格 FDE 的主战场。

[← 上一章: 决策树](#) · [下一 Part: 数据与集成](#) →

Part IV: 数据与集成 — 面对客户真实环境

适用范围：**现场交付主线**（数据 / 集成 / VPC / 合规）；LLM 应用主线在涉及客户真实数据时也需要全部读

这个 Part 要解决什么问题

LLM 应用做出 demo 后，最容易死在两件事上：

1. **数据接不上** —— 客户数据在 SAP / Oracle / 私有云 / 离线 / 混合云
2. **接上了但合规过不了** —— PII / 跨境 / 等保 / 审计

Palantir 风格的 FDE 50% 时间在这一 Part 的话题上。LLM 风格的 FDE 走到生产前一定会撞上。

这个 Part 给三章实操：

- **Chapter 9**：数据栈本身（Ontology / ETL / 实时管道）
- **Chapter 10**：客户 VPC / 私有部署 / 离线机房怎么做工程
- **Chapter 11**：和遗留系统对接（SSO / SCIM / API / 审计）

包含章节

详见 [SUMMARY.md](#)。

与其他 Part 的关系

- **前置**：Part II 的 Discovery 必须把数据现状摸清楚（Ch 4.5 数据准入）
 - **并行**：Part III 的 Scaffolding 阶段，数据接入工作并行进行
 - **后续**：Part V 的生产化、Part VI 的 Agent 工具调用都依赖这一 Part 的基础
-

Chapter 9: 客户数据栈 — Ontology / ETL / 实时管道

开场

某金融客户。FDE 的 LLM Agent demo 跑得很好，但要接生产。

第一周接数据接到崩溃：

- 客户数据库 5 个：PostgreSQL（主库）+ Oracle（合约）+ SQL Server（财务）+ MongoDB（日志）+ 共享盘 Excel（30%）
- 主键不统一：CRM 用 customer_id，ERP 用 cust_no，财务用 client_code，没有 mapping 表
- 时区不统一：UTC + Asia/Shanghai + 客户当地时区混用
- 一份关键报表的数据从 4 个系统拉出来，每个系统有不同的"客户" 定义

FDE 学到了一句话：

"Agent 上线前 80% 的工程在数据治理上。
不做数据治理直接上 Agent，
Agent 越聪明越容易把不一致的数据
错误地组合成自信的答案。"

这一章讲：怎么把一个客户的数据栈
从"五湖四海"治理到"Agent 能放心调用"。

9.1 客户数据栈的典型形状

客户企业的"数据 5 层"

L1 操作型数据库 (OLTP)

- PostgreSQL / MySQL / Oracle / SQL Server
- 业务系统主存储

L2 数据仓库 / 数据湖 (OLAP)

- Snowflake / Redshift / BigQuery / Databricks / Iceberg
- 跑分析的地方

L3 中间层 (Data Mart / Cube)

- dbt 模型 / 公司自建 view
- 业务指标的"标准定义"

L4 BI / Dashboard

- Tableau / Power BI / Quicksight / Looker

L5 业务应用 / Agent / API

- 应用层使用数据的地方 (FDE 的工作多在这一层)

FDE 第一周要画一张这种“5 层数据图”，把客户的现状画清楚。

9.2 Ontology — 把“五湖四海”统一

Ontology 是 Palantir 引入并推广的概念，本质上是**业务对象 + 关系 + 属性**的统一定义。

Ontology 的 3 个组成

Object (对象):

Customer / Order / Product / Contract / Employee
每个对象有"金本位定义"

Property (属性):

Customer.id, Customer.name, Customer.tier
每个属性有 source-of-truth

Relationship (关系):

Customer 1:N Order
Order 1:N OrderItem
Order N:1 Salesperson

Ontology 的 4 个核心问题

1. 谁是"客户"?
 - CRM 表? 还是付款主体? 还是合同方?
 - 不同部门定义不一样 → 必须统一
2. ID 用什么?
 - CRM 的 customer_id?
 - 合同方的 unified_party_id?
 - 必须有"金 ID"
3. Mapping 怎么建?

- 大部分情况是手工建表
- 没有银弹

4. 谁有权改 Ontology?

- 有 owner 才能避免“今天 A 改一个字段，明天 B 改回来”

AWS 实操：用 Lake Formation + Glue Data Catalog 做轻量 Ontology

AWS 没有 Palantir 那种“完整 Ontology 框架”，但可以用 Glue + Lake Formation 实现轻量版：

Glue Data Catalog 作为 Ontology 注册中心

Database: customer_360_ontology

Table: customer (← 业务对象定义)

Columns:

- customer_id (string, 金 ID)
- source_systems (struct: crm_id, erp_no, finance_code)
- tier (string)
- tags (LF tags: PII, region=APAC)

Table: order

...

↓

Lake Formation Tags (LF-Tags):

- PII: yes / no
- sensitivity: public / internal / restricted
- region: APAC / EMEA / NA

↓

IAM 角色 + LF-Tags 控制谁能查哪个对象的哪个字段

好处：

- Ontology 的元数据由 Glue 维护
- 权限由 LF-Tags + IAM 维护
- Athena / Redshift / EMR 都能查
- Bedrock Knowledge Bases 直接读 Glue 元数据

AWS 知识参考：搜“AWS Lake Formation tags”与“Glue Data Catalog cross-account”。

9.3 ETL — 让数据流起来

ETL (Extract / Transform / Load) 是把 L1 → L2 → L3 的工程。

ETL 的 3 个工程信号

数据"够新吗"? → 看 SLA

- T+1: 隔天看 (90% 项目够用)
- T+1h: 小时级 (实时性要求高)
- 实时: 秒级 (CDC 才能做)

数据"够准吗"? → 看 quality 测试

- 主键唯一
- 不能有空值
- 数值在合理范围
- 业务规则 (订单金额 > 0)

数据"够稳吗"? → 看依赖图 + 监控

- 上游挂了, 下游应该 graceful 失败
- 失败应该有 alert
- 重跑应该幂等

ETL 工具速决

场景 → 推荐工具

云上 + Spark 系	Databricks / EMR + Delta AWS Glue (serverless 友好)
云上 + 数仓内	dbt + Snowflake / BigQuery / Redshift
云上 + Streaming	Kafka + Kinesis Data Streams + Flink
自建 / 离线	Airflow + Spark + Iceberg
小规模 / 简单	AWS Step Functions + Lambda + S3

dbt 是 FDE 的“瑞士军刀”

如果客户用云数仓, 80% 的 ETL 用 dbt 写:

dbt 项目的标准结构

```
models/  
  staging/  
    stg_crm_customers.sql      (清洗 raw)  
    stg_erp_customers.sql  
  intermediate/  
    int_customer_unified.sql   (做 join / mapping)  
  marts/  
    dim_customer.sql           (业务对象层)
```

fct_orders.sql

tests/
 not_null_customer_id.yml (数据质量)
 unique_customer_id.yml

macros/
 pii_mask.sql (复用逻辑)

dbt 的好处:

- SQL-only (FDE 不用学新语言)
 - 自带 lineage (数据血缘可视化)
 - 自带 testing (uniqueness / not-null / accepted-values)
 - Git 管理 + 代码 review (数据工程也工程化)
-

9.4 数据血缘 / Lineage — 失败定位的命脉

没有 Lineage 的数据栈:

"下游报表错了" → 一个人翻 5 个系统找 3 天

有 Lineage 的数据栈:

"下游报表错了" → 5 分钟看清是哪个上游 ETL 改了 schema

工具

开源:

- OpenLineage (dbt / Airflow / Spark 都支持)
- Marquez (OpenLineage 的 backend)

商业 / 云:

- DataHub
- Atlan
- AWS Glue (有 lineage 视图)
- Unity Catalog (Databricks)
- Foundry (Palantir)

FDE 的最低要求

不要求“全公司血缘”，但自己做的每个 dbt 模型 / Glue Job 必须接 lineage:

1. dbt: 自动生成 lineage (dbt docs serve)
 2. Airflow / Glue: 装 OpenLineage hook
 3. 出问题第一步: 查 lineage, 找根因
-

9.5 实时数据管道 — 慎用

什么时候真的要实时

- ✓ 业务流程必须秒级反馈（风控 / 推荐 / 广告竞价）
 - ✓ 决策窗口短（库存 / 价格）
 - ✓ 用户体验感知（在线状态 / 实时通知）
- 满足 1 条考虑实时
- 都不满足 → 用 T+1, 省 70% 工程量

实时管道的工程坑

- 一开始就实时 → 调试地狱
- 没有 idempotency → 重放数据出错
- Schema 演化没考虑 → 一升级就坏
- 没有 dead letter queue → 单条坏数据卡所有
- 监控不到 lag → 用户投诉了你才知道

AWS 实操：MSK + Kinesis + Firehose 三件套

AWS 实时管道典型组合

生产端 (Producer)

↓

MSK (Managed Kafka) or Kinesis Data Streams

↓

消费端选择：

- A. Lambda 直接处理（小流量）
- B. Flink on EMR / KDA (大流量 + 复杂逻辑)
- C. Kinesis Firehose 直接落 S3 / Redshift

↓

下游：S3 (Iceberg) / Redshift / OpenSearch

简单场景用 Firehose（自动 buffer + 压缩 + S3 partition），复杂场景用 Flink。

AWS 知识参考：搜 “Amazon MSK best practices”，“Kinesis Data Firehose”。

9.6 数据治理的 “最低 5 件事”

不是 “完整数据治理体系”，是 FDE 在客户现场至少要做的 5 件事：

1. 数据 owner 表
每张表 / 每个 dbt 模型有一个 owner（人 + 邮箱）
2. PII 标记

哪些字段是 PII, 必须打 tag, 必须 mask 在 dev

3. Schema 变更流程

增字段: 通知 + 文档

改字段类型 / 删字段: 必须 review + 通知下游

4. 测试覆盖

每张主表至少 3 个 dbt test (uniqueness / not-null / referential)

5. Lineage 可视化

能在 5 分钟内回答"这个数从哪来"

5 件事都不到位 → Agent 上去就是雷区。

9.7 一个真实端到端例子

客户: 某保险公司

Agent 任务: 自动核保 (投保人风险评估)

数据需求 (FDE 在 Discovery 摸出来的):

- 投保人基础信息 (CRM)
- 历史理赔记录 (理赔系统 Oracle)
- 健康告知 PDF (扫描件 + OCR)
- 黑名单 (合规系统 SQL Server)
- 信用评分 (外部 API)

→ 5 个系统 4 个内部 + 1 个外部

FDE 的工程方案:

Week 1: Glue Data Catalog 注册 4 个内部系统的元数据

建 customer Ontology (统一 customer_id mapping)

Week 2: 写 dbt models 把 4 个系统的客户数据合并到 customer_360

打 LF-Tags (PII 字段 mask)

Week 3: Lambda + EventBridge 拉外部 API 信用评分

Week 4: Bedrock Agent 调用 Athena 查 customer_360

+ 调 Lambda 拉信用 + 调 OCR

Week 5-6: Eval + 灰度

关键工程动作:

- 没有写"实时" 管道 (Discovery 时业务确认 T+4h 可接受)
 - 用 Glue Data Catalog 做 Ontology (轻量)
 - 所有 PII 字段在 dev 环境一律 mask
 - 每个 dbt 模型有 owner + tests
-

关键引用

“The Ontology is the contract between data engineering and the rest of the company.” — Palantir Blog, On Ontology, 2024

“Most LLM hallucinations are actually data quality problems wearing a costume.” — A. Lawrence, FDE Rule Book, 2025

“If you can’t explain where the number came from, the customer can’t trust the answer.” — AWS GenAI Innovation Center, 2025

动手清单

接到一个数据密集型 FDE 项目，第 1-2 周必做：

1. 画客户的“5 层数据图”（9.1 节）
 2. 找出 3 个核心业务对象的 **source-of-truth** (customer / order / product)
 3. 建 Glue Data Catalog database（即使是 PoC）
 4. 每张表打 owner + LF-Tags (PII / region / sensitivity)
 5. 用 dbt 写 customer_360 之类的统一视图
 6. 接 OpenLineage (dbt 自带)
 7. 决定 SLA: T+1 / T+1h / 实时（默认 T+1，能跑就先 T+1）
-

反模式清单

- ☒ 跳过 Ontology 直接接 Agent（Agent 会基于不一致数据自信地说错话）
 - ☒ 5 个系统直连不做统一（任何上游小变动都崩）
 - ☒ 数据没 owner（schema 变了没人通知，下游一片崩）
 - ☒ 第一版就上实时管道（调试成本 10x，不一定值）
 - ☒ Dev 环境用真实 PII（合规事故的高发区）
 - ☒ dbt 项目无 tests（数据出错只能靠下游投诉知道）
 - ☒ Lineage 不接（数据问题排查时间从 5 分钟变 5 天）
-

与下一章的关系

数据栈有了，但客户大多不让你用云上的“开箱即用”方案——数据要待在客户 VPC / 私有部署 / 离线机房里。下一章讲：在客户网络隔离环境下，FDE 的工程动作怎么做。

[← Part IV 导读 · 下一章: 在客户 VPC 工作 →](#)

Chapter 10: 在客户 VPC / 私有部署 / 离线机房工作

开场

某国央企。FDE 第一天到客户机房：

客户安全主管："我们规定如下 ——

1. 所有代码必须在内网 GitLab 上
2. 不能用 ChatGPT，不能 google
3. 没有外网，pip install 走我们的私有 PyPI mirror
4. 服务器只能 SSH 跳板机进，不能直接互联网
5. 数据不能出我们机房
6. 你们的 Mac 不能接我们网
—— 用我们这台 Windows 工作站
7. 工作站不能装非白名单软件
(白名单不包括 VSCode 的 GitHub Copilot 插件)"

FDE 心想："那我还能干啥？"

她干了一个月之后总结："其实啥都能干，
只是节奏完全不一样 —— 不是 SaaS 工作流的节奏。
关键是想清楚什么能做、什么必须 Air-gap 做。"

这一章讲：客户 VPC / 私有部署 / 离线机房，
FDE 的具体工程动作是什么。

10.1 三种“非云原生”客户场景

客户的“非 SaaS”部署形态

场景 A：客户 VPC（云上但隔离）

- 客户在 AWS / Azure / Aliyun 上有自己的 VPC
- 互联网通过 NAT 受控出口

- 安全组 + NACL 严格
- 你的代码部署在客户账号里

场景 B: 客户私有云 / 自建机房

- 客户有自己的 K8s 集群 / VMware
- 部分有外网, 部分纯内网
- 你的代码部署到客户机器上

场景 C: Air-gap (完全离线)

- 没有互联网
- 模型 / 镜像 / 依赖必须 USB / 跳板机搬进来
- 国防 / 军工 / 部分金融

90% 的 ToB 客户在 A 或 B, A 最常见。

10.2 场景 A: 客户 VPC workflow

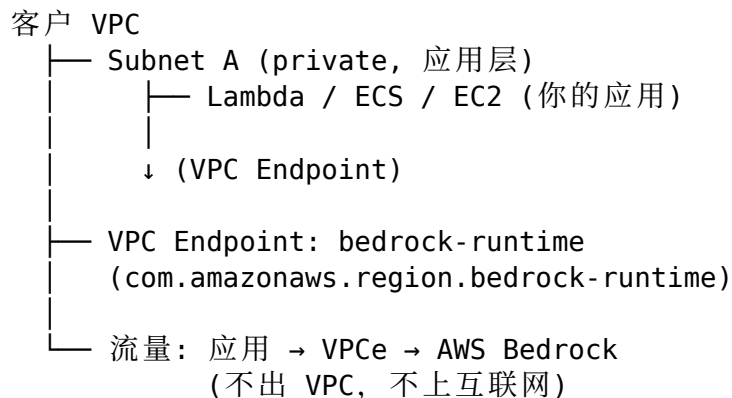
关键工程动作

1. 拿到 IAM 角色 (不是 root, 是 scoped role)
2. 用 AWS Identity Center / 客户 SSO 登入
3. 部署目标在客户账号 + VPC
4. 你的开发本地机器 ↔ 客户 VPC 之间走
 - VPN (Site-to-Site / Client VPN)
 - 跳板机 (Bastion / Session Manager)
 - 不能直连

AWS 实操: 客户 VPC 内部署 Bedrock 应用

最关键的是 “Bedrock 流量不出客户 VPC”。配置:

客户 VPC + Bedrock 私有路径



为什么需要 VPC Endpoint:

- 客户安全主管：“所有调用必须可审计、可阻断”
- 走 VPC Endpoint = AWS 私有网络 = 不暴露公网
- CloudTrail + VPC Flow Logs 全程可审计

最小配置：

1. 创建 VPC Endpoint

```
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxx \
  --service-name com.amazonaws.us-east-1.bedrock-runtime \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-aaa subnet-bbb \
  --security-group-ids sg-yyy
```

2. 应用 SDK 调用 Bedrock

```
import boto3
client = boto3.client('bedrock-runtime')
# SDK 自动走 VPCe (DNS 解析到 VPCe 的私有 IP)
```

AWS 知识参考：搜 “Bedrock VPC endpoints” 与 “AWS PrivateLink for Bedrock”。

Knowledge Bases / Agents 也要走私网

Bedrock Knowledge Bases

- VPC Endpoint: bedrock-agent
- 数据源 S3: VPC Endpoint: s3 (Gateway 类型)
- OpenSearch Serverless: 在客户 VPC 内的 collection

Bedrock Agents

- VPC Endpoint: bedrock-agent + bedrock-agent-runtime
- Lambda 工具：在客户 VPC 内
- 客户内部 API 调用：直接 VPC 内调用

Sagemaker JumpStart — 自部署模型的兜底

如果客户连 Bedrock 都不让用（“必须自部署”），用 SageMaker JumpStart：

SageMaker JumpStart 一键部署：

- Llama 3.1 / Qwen / Mistral 镜像
- 部署在客户 VPC 的 SageMaker endpoint
- 私有 endpoint，不出公网
- 完全在客户账号里

10.3 场景 B：客户私有云 / 自建机房

工作日常

典型一天

7:30 到客户大楼，刷工卡 + 人脸 + 包过 X 光
8:00 上机房二楼 FDE 工作区
8:30 开机：客户配的 Windows 工作站
- 没装 Outlook（用客户邮箱网页版）
- 没装 Slack（用客户内部沟通工具）
- VS Code 装了，但没 Copilot
9:00 ssh 跳板机 → 客户 K8s
- 先看夜里有没有报警
10:00 跟客户运维拿当天的部署窗口
...
17:00 提交今天的代码到客户 GitLab
push 时跑 SAST / DAST 扫描
18:00 写日报 + 回家

关键工程配置

代码托管：

- 客户内部 GitLab / Gitea / Gerrit
- 不能 push 到 GitHub

CI/CD：

- 客户内部 Jenkins / GitLab CI / ArgoCD
- 不能用 GitHub Actions / CircleCI

镜像仓库：

- 客户内部 Harbor / Quay
- 基础镜像（Python / Java）必须从客户内部 mirror 拉

依赖管理：

- pip / npm / maven 走客户内部私有仓库
- 一定要列清单，安全主管会扫

模型存储：

- 模型权重存客户对象存储（MinIO / Ceph / OSS）
- 不能从 HuggingFace 直接拉

监控：

- 客户的 Prometheus + Grafana
- 不能接 Datadog / NewRelic

模型如何“搬进来”

模型权重的进入流程

Step 1: FDE 在公司侧从 HuggingFace 下载模型权重
(Llama / Qwen / Mistral)

Step 2: 病毒扫描 + license 检查 + 安全审计

Step 3: 走客户的"软件入网流程"
- 一份"模型入网申请"
- 一份模型说明 + license + scan 报告
- 等审批 (1-3 周不等)

Step 4: 批准后客户允许 USB 或专网传入

Step 5: 落到客户对象存储 + 注册到模型仓库

FDE 第一周必须把这个流程触发起来。否则到了第 6 周才发现模型还没进来 = 项目延期。

10.4 场景 C: Air-gap (完全离线)

真实故事

某 FDE 在一个完全离线的客户场景部署 LLM，第一天发现 ——

公司给的笔记本上的 git 用了 GitHub 远端，他没改 origin 直接 git push，客户安全主管秒回：“你的笔记本被隔离 24 小时观察。”

Air-gap 的核心特点：

- ✓ 完全没互联网
- ✓ 所有代码 / 工具 / 镜像必须 USB 或可信中转
- ✓ 你的本地机器和客户机器之间走"摆渡机"
- ✓ 进客户网络的所有动作都被审计

工作前 3 天清单

Day 1: 准备 USB 包

- Python / Node / Docker 镜像
- 你的所有代码 + 依赖 (pip download / docker save)
- 模型权重
- VS Code + 离线插件
- 文档 (带 RFC / 设计 / 数据字典)

Day 2: USB 安检 + 入网

- 客户安全审计你的 USB

- 客户帮你装到内部摆渡机
- 摆渡机扫描 + 隔离观察 + 投递到内网

Day 3: 内网 PC 拿到资料, 开始干活

工程纪律

- ✓ 所有 prompt 写在仓库里 (不能"我等会想想再说")
 - ✓ 不能假设有 ChatGPT / Claude 帮你 debug
 - ✓ 必须自带"离线参考":
 - HuggingFace cache
 - PyPI mirror
 - 离线文档 (pip download docs)
 - ✓ 反复出问题的功能 → 简化方案, 不要复杂依赖
 - ✓ 每周末把最新代码批量回传公司 (review + 备份)
-

10.5 私有化 LLM 部署的工程要点

不是云上 SaaS LLM, 而是自部署 Llama / Qwen / Mistral:

选模型规模

GPU 配置 → 推荐模型

4× A100 80G	Llama 3.1 70B 半精度 / Qwen2 72B
2× A100 80G	Llama 3.1 70B AWQ 4bit / 32B 模型
1× A100 80G	Llama 3.1 8B / Qwen2 14B
1× A100 40G	7B / 8B 模型
无 GPU	不要硬上 LLM, 建议用云上 API

推理引擎

vLLM (推荐)

- PagedAttention 内存高效
- OpenAI 兼容 API
- 高 QPS

TGI (HuggingFace)

- 易上手
- 流式输出好

TensorRT-LLM (NVIDIA)

- 极致性能
- 但 ramp-up 复杂

- llama.cpp
- CPU / Mac 也能跑
 - 极小 footprint

部署架构

K8s + vLLM 标准架构:

Ingress (内网 LB) → API Gateway (鉴权 + 限流)

↓

vLLM Pods (HPA)
(每 Pod 1 GPU)

↓

模型权重 (PVC / S3 / OSS)

监控: Prometheus + Grafana (token/sec, GPU util)

日志: Loki / 客户 ELK

Eval / Trace 的私有化

云上 SaaS:

LangFuse Cloud → LangFuse self-hosted
LangSmith → Phoenix (Arize 开源)
Bedrock Eval → 自写 Python 脚本 + dbt

私有部署:

10.6 合规清单 (私有部署 / 中国 ToB 常见)

合规要点速查

数据出境:

- 中国客户的数据不能出境
- 模型权重一般可以入境 (看具体 license)

等保:

- 等保 2.0 三级 / 四级 = 大量约束
- 你的 FDE 工作受影响最大的是审计 + 部署流程

GDPR / SOC 2:

- 数据分类 + 访问审计
- PII 保护

PCI DSS (金融):

- 支付数据加密 + 网络隔离

HIPAA (医疗 / 美国):

- PHI 数据 + audit trail

CSA STAR (企业云):

- 云服务商资质

FDE 的策略: 合规不是 FDE 一个人能搞定的, 但 **FDE 必须在 Discovery 阶段就摸清这些要求**, 不然到了部署前发现 “不能用 X” 整个方案推倒重来。

关键引用

“The customer’ s network is the customer’ s contract — disrespect it once, lose the project.” — A. Lawrence, FDE Rule Book, 2025

“Architecture changes when the GPU lives in the customer’ s data center.” — Palantir FDE training, 2024

“PrivateLink is the most underused service among new AWS users.” — AWS Solutions Architects, 2025

动手清单

接到一个非云 SaaS 客户项目, 第 1 周必做:

1. 画一张 “网络拓扑图”: 客户 VPC / 私有云 / Air-gap, 进出流量怎么走
 2. 拿到 IAM / SSO 身份 (不要用别人的账号)
 3. 建 VPC Endpoint (Bedrock + S3 + CloudWatch 至少这三个)
 4. 走 “软件入网流程” (模型 / 工具 / 依赖一并申请, 越早越好)
 5. 第一周末跑通 “hello world” (一个简单的应用从客户 VPC 内调通 Bedrock 或自部署模型)
 6. 画一份 “数据流图”: 哪些数据进哪些不能进, 有 PII 标记
 7. 找客户安全主管 30 分钟咖啡 (这个人决定你后面 12 周顺不顺)
-

反模式清单

- ☒ 本地起开发用 SaaS API, 部署再换私有 (行为差异巨大, 到时候才发现不能上)
 - ☒ 没走 VPC Endpoint, 上 NAT 出去 (合规审计点名 #1)
 - ☒ 不申请软件入网就开始装 (“先装上再说” 会被秒清)
 - ☒ 把客户数据下载到本地分析 (合规事故 #1)
 - ☒ 代码 push 到 GitHub (客户禁止外部仓库)
 - ☒ Air-gap 客户开发还想 ChatGPT 帮忙写代码 (隔离 24 小时观察)
-

与下一章的关系

数据 + 网络都搞定，最后一道是和 “客户已有系统” 对接 —— SSO / SCIM / API / 审计。下一章讲：和遗留系统对接的工程模式。

[← 上一章: 客户数据栈](#) · [下一章: 与遗留系统对接](#) →

Chapter 11: 与遗留系统对接 — SSO / SCIM / API / 审计

开场

某保险客户。FDE 第 8 周演示成功，第 10 周准备上线。
客户 IT 总监问 4 个问题，FDE 当场答不出 3 个：

- Q1: 用户登录走我们的 AD 还是你们自己的账号？
- Q2: 离职员工怎么自动注销 Agent 的访问权？
- Q3: Agent 调用 ERP API，权限边界谁定？
- Q4: 出了事，审计日志能追到具体调用吗？

FDE 回去把这四个问题写在白板，发现 ——
SSO / SCIM / 权限 / 审计是 4 件不同的事，
没一件是"上线前补"能补完的。

她重新规划：

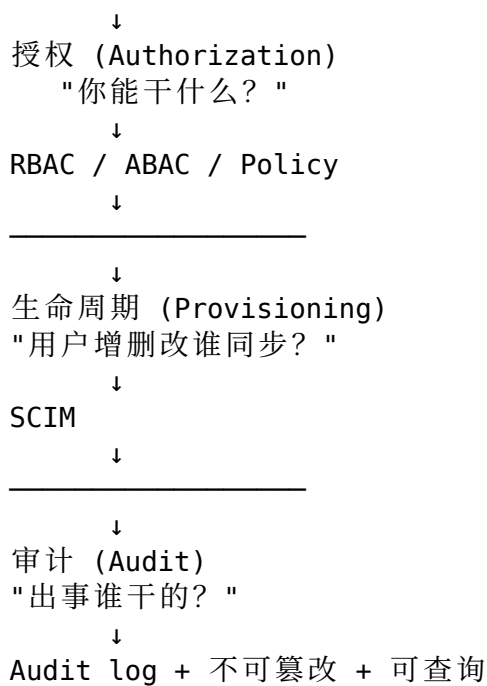
- Week 11: SSO + SCIM
- Week 12: 权限边界 + 审计
- 上线推迟 2 周

她老板说："早 4 周做这个就好了。"
她说："下次知道了。"

这一章给的就是"早 4 周做"的清单。

11.1 四件事的关系

身份 (Authentication)
"你是谁？"
↓
SSO (Single Sign-On)
OAuth / SAML / OIDC
↓



四件事互锁。少一件，企业上线过不了。

11.2 SSO — 接客户 AD / Okta / Identity Center

协议速记

SAML 2.0

- 老牌企业 IdP 通用 (AD FS / Okta / OneLogin)
- 基于 XML，复杂但稳

OIDC (OpenID Connect)

- OAuth 2.0 之上的身份层
- JSON / JWT，现代友好
- 推荐新项目用 OIDC

Kerberos

- Windows AD 内部协议
- 国内大型企业内网常用

LDAP

- 不是 SSO，是查目录
- 但很多客户混着用

workflows (OIDC 例)

典型 OIDC 登录流程

1. 用户访问你的 Agent
2. 你的 App 重定向到客户 IdP (AD / Okta)
3. 客户 IdP 验证用户 (密码 / 短信 / Yubikey)
4. IdP 返回 ID token + Access token
5. 你的 App 解析 token:
 - sub: 用户 ID
 - email: 邮箱
 - groups: 部门 / 角色
6. 用 token 调你的后端 / Agent
7. 后端用 group / role 做授权判断

AWS 实操: 用 IAM Identity Center + Bedrock Agent

IAM Identity Center 三种集成

- 方式 1: 客户自带 IdP (Okta / Azure AD)
- SAML / SCIM 同步用户进 Identity Center
 - Identity Center 给用户分配 IAM Role

- 方式 2: 客户用 AD (Active Directory)
- AD Connector 或 AWS Managed AD
 - 用户用 AD 账号登 AWS

- 方式 3: 客户没 IdP (小型)
- Identity Center 自带用户管理 (不推荐)

最小配置:

1. Identity Center 启用 (Organization 级)
2. 接客户 IdP:
 - SAML metadata 互换
 - Attribute mapping (email, groups)
3. SCIM 自动同步:
 - 客户 IdP 增删改用户 → 自动同步到 Identity Center
4. Permission Sets:
 - "BedrockAgentUser" → 仅能调 specific Agent ID
 - "BedrockAgentAdmin" → 能改 Agent 配置
5. Agent 调用:
 - 用户 → 你的 App → STS AssumeRoleWithSAML

→ 临时凭证 → `invoke_agent` (with `session_attributes`)

AWS 知识参考: 搜 “AWS IAM Identity Center external IdP” 与 “Bedrock Agent identity context”。

11.3 SCIM — 用户生命周期同步

为什么需要 SCIM

没有 SCIM:

客户 HR 系统员工离职

- 手动通知 IT
- IT 手动在 5 个系统里删账号
- 你的 Agent 是第 6 个系统, 常常被忘
- 离职员工还能用 Agent 调 ERP

有 SCIM:

HR 系统离职 (Workday / SAP HCM)

- IdP 自动同步 (Okta / Azure AD)
- SCIM 协议 → 你的系统
- 你的系统秒级禁用账号

SCIM 协议

本质: REST API + JSON Schema

端点 (你需要实现):

POST /scim/v2/Users	(创建用户)
PATCH /scim/v2/Users/:id	(更新用户)
DELETE /scim/v2/Users/:id	(删除用户)
GET /scim/v2/Users/:id	(查询用户)

双向同步:

Push: IdP → 你的系统 (新增 / 修改 / 删除)

Pull (可选): 你的系统 → IdP (反向同步)

FDE 实操要点

- ✓ 优先用现成方案
 - AWS IAM Identity Center 自带 SCIM endpoint
 - Auth0 / Okta SCIM 模板
 - 不要自己从零写
- ✓ 测试用例必须有
 - 创建 → 登录 → 修改组 → 验证权限变 → 删除 → 验证不能登

- ✓ 日志记录每次 SCIM 调用
 - 客户审计要查"该员工是何时被禁用的"
-

11.4 API 集成 — 调客户的旧系统

客户旧系统的 4 种"接口形态"

形态 1: REST / OpenAPI (现代)

- 直接调
- 通常带 OAuth 2.0 / API Key

形态 2: SOAP / WSDL (传统企业)

- XML 信封
- 大量 enterprise 中间件
- Python 用 zeep 库

形态 3: 数据库直连 (反模式但常见)

- 客户给你只读账号查 Oracle / SQL Server
- 风险高, 能避就避

形态 4: 文件落地 (最古老)

- SFTP 共享文件夹
- CSV / XML 定时投递
- 业务关键时间窗口

Agent 调旧系统的设计模式

Agent → Tool → Adapter → 旧系统

Agent (Bedrock / LangGraph)

↓ (function call)

Tool (Lambda / 你的代码)

↓ (实现 SDK 或客户提供的 client)

Adapter (一层薄翻译)

↓

旧系统 (SAP / Oracle / SOAP)

Adapter 的好处:

- 旧系统协议不漂到 Agent 里
- Agent 升级 / 旧系统升级互不影响
- 单点 fallback / retry / circuit breaker

4 个工程信号

- ✓ 永远先调 read API
 - 写 API 上线前要客户 sign-off
 - ✓ 永远要 idempotency key
 - 同一个操作重试不能重复发生
 - ✓ 永远要 timeout + circuit breaker
 - 旧系统挂了不能拖死你
 - ✓ 永远要 dry-run 模式
 - 演示 / 测试时不真的写
-

11.5 审计日志 — “你能证明吗？”

合规审计的 5 个问题

- Q1: 谁做了什么?
→ user_id + action + resource
- Q2: 什么时候做的?
→ timestamp (UTC, microsec)
- Q3: 从哪做的?
→ IP + User-Agent + session_id
- Q4: 结果是什么?
→ success / failure + 返回内容摘要
- Q5: 谁有权限做这件事?
→ 当时的 role / policy snapshot

审计日志的 4 个工程要求

1. 不可删除
 - WORM (Write Once Read Many) 存储
 - S3 Object Lock / CloudTrail
2. 可关联
 - trace_id 串联所有相关日志
 - 从用户行为 → API 调用 → DB 查询 → 模型调用
3. 可查询
 - 不是只是“存着”，要能“查出来”

- 推荐: CloudWatch Logs + Athena 查询

4. 保留期

- 合规一般 90 天 / 1 年 / 7 年 (看行业)
- 别忘归档 (S3 Glacier)

AWS 实操: CloudTrail + CloudWatch + Athena 三件套

FDE 项目典型审计架构

应用层 → 应用日志 (含 trace_id)

↓

CloudWatch Logs (1 周 → S3 归档)

↓

Athena (按 trace_id 查询)

AWS API → CloudTrail (所有 AWS API 调用)

↓

S3 (Object Lock + KMS 加密)

↓

Athena 查询 / Security Hub 告警

Bedrock 调用 → Bedrock Model Invocation Logging

↓

CloudWatch / S3

↓

审计: "用户 X 在 T 调用了 Agent 问了什么 / 答了什么"

最小开关:

1. CloudTrail Multi-Region Trail (强制开)

- All management events
- Data events for S3 / Lambda (按需)
- Insight events (异常检测)
- 加 Object Lock 防删

2. Bedrock Model Invocation Logging

- Bedrock console → Settings → Enable
- Destination: CloudWatch + S3
- 含 prompt + response 全文

3. 应用日志规范:

- 每条日志带 trace_id (X-Ray / OpenTelemetry)
- user_id (从 IAM context 拿)
- action + resource_arn

AWS 知识参考: 搜 “CloudTrail data events”、“Bedrock model invocation logging”、“S3 Object Lock”。

11.6 Guardrails — Agent 的行为审计

不仅人在审计 Agent, Agent 也要被实时检查:

Bedrock Guardrails 5 类拦截

1. Content filter (有害 / 暴力 / 仇恨 / 性内容)
2. Topic filter (禁止讨论的话题)
3. PII filter (身份证 / 电话 / 邮箱 自动脱敏)
4. Word filter (公司禁词 / 竞品名 / 内部代号)
5. Contextual grounding (回答必须基于知识库)

配置思路

PoC 阶段:

- ✓ 开 PII filter
- ✓ 开 Content filter (中等强度)

生产前:

- ✓ 加 Topic filter (业务相关红线)
- ✓ 加 Word filter (公司机密词)
- ✓ 加 Grounding (RAG 应用必加)

上线后:

- ✓ 监控 Guardrails 拦截次数
- ✓ 拦得太多 → 调阈值或调 prompt
- ✓ 漏得太多 → 加规则

AWS 知识参考: 搜 “Bedrock Guardrails”。可以独立于模型使用, 也可以绑到 Agent。

11.7 一个完整的整合例子

客户场景: 保险公司销售助手 Agent

身份层:

- 销售员用 AD 账号 SSO 登入
- Identity Center + SAML 接客户 AD

生命周期:

- HR 离职 → AD 禁用 → SCIM 同步 → Agent 立即不可用

权限层:

- "销售员" 角色: 只能查自己客户的保单

- "经理" 角色：能查团队所有保单
- 用 ABAC: tag-based access control

Agent 调用旧系统：

- 调 ERP (SOAP) → Lambda Adapter → SAP RFC
- 调 CRM (REST) → Lambda → Salesforce API
- 所有调用带 user context (不是 service account)

审计：

- CloudTrail: AWS API
- Bedrock Logging: prompt + response
- 应用日志: trace_id 串联
- 审计窗口: 7 年 (保险行业)
- 存储: S3 Object Lock + KMS

Guardrails:

- PII filter: 客户身份证 / 银行卡自动脱敏
- Topic filter: 不讨论投资建议 (合规要求)
- Grounding: 答案必须基于产品文档

关键引用

“Identity is the new perimeter — and it’s the first thing the customer’s CISO will ask about.” — A. Lawrence, FDE Rule Book, 2025

“Audit is not a feature — it’s the only proof you have when something goes wrong.” — AWS Well-Architected, 2025

“If your Agent can’t tell who’s asking, it shouldn’t answer.” — Anthropic enterprise best practices, 2025

动手清单

接到企业项目第 1-2 周必做：

1. 画一张“四件事图”：SSO / SCIM / 权限 / 审计 各自现状
 2. 跟客户安全主管开 30 分钟会：他们用什么 IdP / 等保级别 / 审计要求
 3. 拿到 SSO 测试账号 (SAML metadata 或 OIDC client)
 4. 接 Identity Center + SCIM (不要自己从零写身份系统)
 5. CloudTrail + Bedrock Invocation Logging 第一周打开
 6. 每个 Agent 调用旧系统的 Tool 都过 Adapter (不要直连)
 7. Bedrock Guardrails 至少配 PII + Content filter
 8. 写一份“审计日志查询 SOP” 客户审计来时直接交付
-

反模式清单

- ☒ 用 service account 跑所有调用（审计追不到具体用户）
 - ☒ 离职员工不在你这边自动禁用（合规事故）
 - ☒ Agent 直连旧系统 SOAP 协议（耦合 + 没法 fallback）
 - ☒ 审计日志可被运营人员删除（合规审计直接 fail）
 - ☒ PII 没脱敏就进了 Bedrock 调用日志（GDPR / 等保事故）
 - ☒ 没有 dry-run 模式调写 API（demo 时把生产数据搞坏）
 - ☒ Guardrails 一拖再拖（出事一次代价远高于配置成本）
-

与下一 Part 的关系

到这里，“FDE 在客户真实环境工作”的全部基础打好了：数据栈、网络隔离、身份审计。下一 Part 进入 PoC → 生产 这个最难的鸿沟 —— 在 demo 之后，怎么真正稳定上线、控制成本、灰度回滚。

[← 上一章: 在客户 VPC 工作](#) · [下一 Part: PoC → 生产](#) →

Part V: PoC → 生产 — 跨过最难的鸿沟

适用范围：通用（两条主线都要过这一关）

这个 Part 要解决什么问题

PoC 跑通和上生产之间的距离，比 PoC 本身工作量还大。

OpenAI 自己有个内部说法：> “From PoC to production is 6 weeks of joy followed by 6 weeks of yak shaving.”

业界平均 PoC 到生产转化率约 30-40%。AWS GenAI Innovation Center 把这个数字拉到了 73%，靠的不是更牛的模型，而是 **PoC 阶段就预埋生产化的工程动作**。

这个 Part 给两章：

- **Chapter 12**: PoC 过线条件 —— 哪 6 件事能让 PoC 真上得了生产，哪 4 件事会卡死
- **Chapter 13**: 可观测性 / 成本 / 灰度 / 回滚 —— 生产化的 “四件套”

包含章节

详见 [SUMMARY.md](#)。

与其他 Part 的关系

- **前置**：Part III 的 Eval-driven、Part IV 的数据 + 集成都是这一 Part 的输入
 - **后续**：Part VI 在生产环境上加 Agent；Part VII 完成交付和收尾
-

Chapter 12: PoC 过线条件 — 哪些能过, 哪些卡死

开场

某 SaaS 公司。FDE 第 6 周交付 PoC, 效果惊艳:

- Demo 现场客户 CEO 鼓掌
- Eval 总分 0.91
- 媒体报道也写了

第 7 周客户 IT 总监问: "上生产怎么走?"

FDE 回: "照这个 demo 部署就行。"

第 14 周还没上线。

卡在哪里?

- 没有 staging 环境 (demo 就是 dev 改的)
- 没有部署脚本 (demo 是手动起的)
- 没有压测 (每秒 1 个请求是上限)
- 没接客户监控 (运维不知道怎么 oncall)
- 成本预算没批 (一个月 \$20K, 老板没准备)
- 灰度方案没有 (只能 "all or nothing")

复盘: PoC 阶段 没把 "生产化的 6 件事" 同步做。

PoC 越炫, 生产化越难。

这一章给: PoC 阶段就要做的 6 件事,
以及 "4 个一定卡死" 的信号。

12.1 PoC vs 生产 — 工程指标差 100 倍

	PoC 标准	生产标准
并发	单用户 / 几个	100-10K QPS

可用性	"能演示就行"	99.5% / 99.9%
延迟	P50 5 秒可接受	P95 < 3 秒
容错	"出错重启"	自动重试 + 隔离
可观测	控制台 print	全链路 trace
权限	FDE 手里的 admin	RBAC + 审计
数据	"你给我 50 条"	全量 + 实时
成本	老板没说	月度预算控制
发版	"我手改 prompt"	CI/CD + 灰度
客户	FDE 一直在	客户运维独立维护

90% 的 PoC 失败不是技术问题，是“工程标准”和生产差太远。

12.2 PoC 阶段就要做的 6 件事

事 1: 环境分层 (dev / staging / prod)

从第 1 天就分：

dev:	FDE 工作台，可随便改
staging:	和生产同等配置，跑回归
prod:	客户运维管理，最严

即使 PoC 阶段，至少 staging 必须有
 否则："demo 改的 prompt 没人记得" → 上线找不到

事 2: CI/CD 第一周就接

最小 CI/CD pipeline:

```
on push to feature branch:
- lint
- unit test
- eval (50 条 seed)
```

```
on PR to main:
```

- eval (200 条 golden)
- integration test
- cost estimate

on merge to main:

- 自动部署 dev
- 触发 staging deploy

on tag release:

- 部署 staging
- 等人 sign-off
- canary deploy prod (10% → 50% → 100%)

没有 CI/CD 的 PoC 不要进 PoC。

事 3：监控 / Trace 第一周就接

必有 trace 4 维：

1. Latency (per step)
2. Cost (input/output tokens, model)
3. Quality (eval / 用户反馈)
4. Error (stack trace + correlation_id)

工具选型 (复习 Ch 6):

- 云上: LangFuse Cloud / LangSmith / Bedrock built-in
- 私有: LangFuse self-host / Phoenix

事 4：成本透明 (PoC 阶段就开始报)

每周报表必有：

- 本周 token 消耗 (input + output)
- 按模型分布
- 按场景分布 (RAG / Agent / Eval)
- 单次请求平均成本
- 月度推算

PoC 阶段就开始 → 不会出现 "上线一周烧 \$50k 老板震惊"

事 5：客户运维 “挂上来”

从第 4 周开始：

- 客户至少 1 个运维加入项目 Slack
- 一起 on-call (虽然 PoC 阶段不真值班)
- 一起看 dashboard

- 一起处理告警

不挂上来：上线后客户运维"接手" =
从零开始学，3 个月才能独立
挂上来：上线时客户运维已经熟悉，
1 周交接完

事 6：可降级方案 (fallback)

关键路径必须有 fallback:

Bedrock 模型挂 → 切到备用模型 (跨 region)
Knowledge Base 挂 → 切到 cached embeddings
Agent 工具挂 → 拒绝服务 + 友好错误信息
全挂 → 静态回复 + 工单创建

PoC 阶段至少：

- ✓ 写一个"全挂时"的兜底
 - ✓ 演练一次"全挂"
-

12.3 4 个“一定卡死”的信号

信号 1：客户没人愿意做 owner

"这个项目谁负责"答不上来的项目：

- x 业务方说"这是 IT 的事"
- x IT 方说"这是业务的事"
- x 老板说"两边一起负责"

→ 上线后没人 oncall，没人审 prompt 变更，没人改 KB
→ 6 个月内"自然死亡"

FDE 的应对：在 SOW 里硬性写“客户方 owner”，没有 owner 不开工。

信号 2：Eval 集是 FDE 自己写的

FDE 一个人写 200 条 Eval →
上线后客户业务专家说"这些题不是我们要的"
→ Eval 全部重做
→ 上线推迟 2-4 周

应对：第 3 周开始就拉客户业务专家共标
FDE 写初版 → 业务专家审 → 再标

信号 3：成本到月底才算

Demo 阶段每月 \$500

上线第一周 \$5K

客户老板看到账单："这不能用，砍掉"

应对：PoC 第 2 周就给老板报第一份成本测算

上线前给"3 种规模下的成本曲线"

上线后每周成本仪表盘

信号 4：客户 IT 没参加任何 PoC 评审

6 周 PoC 全是和业务对接，零 IT 参与

上线时 IT 一票否决：

"VPC Endpoint 没接"

"审计日志不达标"

"等保过不了"

→ 推倒重来

应对：第 1 周 Discovery 就拉 IT 进来

每两周一次 IT-FDE 同步会

第 4 周开始 IT 参加 demo

12.4 PoC 过线的 5 项硬指标

把这 5 项写进 SOW，达不到就不上生产：

指标	过线条件
Eval 综合分	$\geq$ SOW 约定值（典型 0.85）
生产场景压测	P95 < 3s, 100 QPS 稳定 30 分钟
成本	单次请求成本 $\leq$ 预算 X 元
审计	所有调用 100% 上 CloudTrail/日志
灰度方案	能 1% / 10% / 50% / 100% 灰度切

5 项 5 全过 → 进灰度生产。少 1 项 → 推迟。

12.5 一个真实的 PoC → 生产时间表

12 周项目典型时间表

W1-2 Discovery (Part II)

- 5 件物料：报告 / Eval seed / SOW
- 客户 owner / IT 接入 / 安全 review

W3-6 Scaffolding (Part III)

- 6.7 速决表 baseline
- Eval CI 接好 (Ch 8)
- 第一版 demo

W6 PoC 中检

- 12.4 五项硬指标第一次跑
- 哪些不达标？

W7-9 Productionize Phase 1

- 数据接入 + VPC + SSO + 审计 (Part IV)
- 监控 + 成本 + 灰度准备 (Ch 13)

W10 PoC 终检

- 12.4 五项硬指标全过
- 客户 IT + 业务 + 老板三方签字

W11 Canary 灰度 (10% → 50%)

W12 100% 上线

+ Handoff (Ch 16) 启动

节奏控制比“功能完成度”更重要。

12.6 一个真实反例

某互联网公司 FDE 团队做了一个 8 周的 LLM 客服 PoC。第 8 周客户 CEO 看 demo 满意，准备签 12 个月合同。

PoC 期间这些没做：- 没接客户的 SSO（用了 service account）- 没接 CloudTrail（说“上线再加”）- 没做压测（demo 时一个 QPS）- 没和客户 IT 对过架构（IT 第一次见图是签合同前）

合同签了之后，IT 部门花 3 周 review 架构，提了 47 条修改意见。重做花了 5 周。

客户业务方：“你们承诺 8 周上线，怎么 16 周还在改？”

FDE 团队失去了下一阶段续单。

复盘：PoC 期间，业务方满意 ≠ 项目能上。IT / 安全 / 合规 / 财务任何一方不满意，都能让项目推倒重来。

关键引用

“The PoC is not the project — it’s the trailer for the project.” — A. Lawrence, FDE Rule Book, 2025

“A 73% PoC-to-production conversion rate doesn’t come from better models. It comes from running PoC like it’s already production.” — AWS GenAI Innovation Center, 2025

“Every PoC has a critic — find them in week 1, not week 8.” — Bob McGrew @ YC, 2025

动手清单

PoC 项目第 1 周必做：

1. 写 SOW 里的 “PoC 过线 5 项硬指标” (12.4 节)
 2. 第 1 天分好 dev / staging / prod 三套环境
 3. CI/CD 第一周接好 (Eval + lint + deploy)
 4. 接 trace + cost dashboard (CloudWatch 或 LangFuse)
 5. 拉 IT / 安全 / 财务 / 业务方 owner 进项目群
 6. 第 4 周开始客户运维和 FDE 一起看 dashboard
 7. 第 6 周做一次 “假装上线” 演练 (包括成本测算 + 压测 + 演练故障)
-

反模式清单

- ☒ 把 PoC 当成 “演示版” 做 (一切都要重做才能上)
 - ☒ dev 直接 demo (没 staging)
 - ☒ PoC 期间不接监控 / 审计 / SSO (说 “上线再加”)
 - ☒ 客户 IT 不在项目群 (最后一刻被一票否决)
 - ☒ 成本到上线后才算 (老板不批, 项目暴毙)
 - ☒ 没有灰度方案, 只能 0% / 100% (出问题不能优雅回退)
-

与下一章的关系

这一章给了 “PoC 过线的 5 项硬指标”。下一章具体讲：监控 / 成本 / 灰度 / 回滚 — 生产化 “四件套” 的工程实操。

[← Part V 导读 · 下一章: 可观测性 / 成本 / 灰度 / 回滚 →](#)

Chapter 13: 可观测性 / 成本 / 灰度 / 回滚

开场

某零售客户的 Agent 上线第 3 天凌晨 2 点:

P1 告警: Agent 错误率 18% (基线 < 0.5%)

on-call 的客户运维打电话给 FDE:

"你们的 Agent 出问题了, 怎么办?"

FDE 心想 (应该):

1. 看 trace → 找根因 (5 分钟内定位)
2. 1% 流量切回旧版 → 止血 (2 分钟内)
3. 不行就 100% 切回 (1 分钟内)
4. 修完后再灰度上 → 可控

FDE 实际 (坏情况):

1. trace 没接 → 不知道哪一步错
2. 没有灰度通道 → 要么硬撑要么全部下线
3. 没有 baseline → 不知道是不是真的"比平时差"
4. 1 小时手忙脚乱 → 客户看在眼里

这一章给的"四件套" —— 观测 / 成本 / 灰度 / 回滚 ——
就是为了那个凌晨 2 点能 5 分钟止血。

13.1 可观测性 — 不只是“看日志”

三大可观测性维度

Metrics (聚合数字)

QPS, P50/P95 latency, error rate, token throughput
→ 看趋势, 设告警

Logs (单条文本)

具体错误堆栈, prompt + response 全文
→ 排查根因

Traces (跨服务调用链)

用户请求 → API → 检索 → 模型 → 工具 → 返回
→ 找瓶颈 / 串联问题

LLM 应用的可观测有 4 个**特殊维度**:

1. Token 经济:
 - input/output tokens per request
 - cost per request (按模型计价)
2. Eval 漂移:
 - 生产采样回来 → 跑 Eval → 看分数趋势
3. Hallucination 监控:
 - 答案不被 grounding 文档支持的比例
 - 用 LLM-as-judge 实时打分 (采样)
4. Agent 路径:
 - 单次完成步数分布
 - 工具调用成功率
 - 重试次数

AWS 实操: 可观测三件套

最小可观测栈 (AWS 上)

Metrics:

- CloudWatch Metrics (自动: Bedrock invocations, latency, tokens)
- CloudWatch Custom Metrics (你的: eval score, hallu rate)

Logs:

- CloudWatch Logs (应用日志, 必须含 trace_id)
- Bedrock Model Invocation Logging (prompt + response)
- 长期归档 → S3

Traces:

- X-Ray (跨 Lambda / API Gateway / ECS)
- 可选: LangFuse / Phoenix (LLM 专用 trace)

Dashboard:

- CloudWatch Dashboard (运维)
- 自建 BI (业务 KPI)

必看的 6 个 dashboard 卡片

1. QPS + Error rate (主健康度)
2. P50 / P95 / P99 latency (体验)
3. Token usage + Cost trend (钱)
4. Eval score (实时采样) (质量)
5. Top failure types (排错入口)
6. Agent step distribution (Agent 健康)

AWS 知识参考: 搜 “CloudWatch metrics for Bedrock”、 “X-Ray for Bedrock Agents”、 “Bedrock model invocation logging”。

13.2 成本控制 — Token 是新的 “水电费”

LLM 应用最大的 “非工程” 风险是钱。

典型成本结构 (按月)

Bedrock 模型调用 (60-80%)

- input tokens × 单价
- output tokens × 单价 (通常贵 4-5 倍)

Embedding (5-10%)

- 索引时一次性 + 查询时每次

Knowledge Base / Vector DB (5-15%)

- OpenSearch Serverless OCU 或 pgvector 实例

其他 (5-15%)

- Lambda / ECS / 网络 / 监控

成本的 4 个工程动作

1. Caching (缓存)

- 相同 query 缓存 1 小时
- prompt prefix 缓存 (Anthropic / Bedrock 已支持)
- 节省 30-70%

2. Routing (路由)

- 简单 query → mini / haiku
- 复杂 query → sonnet / opus
- 节省 50-80%

3. Compression (压缩 context)

- RAG 召回前 top_k 太多 → 限制
- System prompt 优化（去重 + 精简）
- 节省 10-30%

4. Batching (批处理)

- 异步任务用 Bedrock Batch (50% 折扣)
- 适合 eval / 离线分析

AWS 实操：Bedrock 成本监控

Bedrock 成本监控三层

Layer 1: AWS Cost Explorer

- 按 service / region / tag 聚合
- 月度趋势 + 异常告警

Layer 2: Cost Allocation Tags

- 给每个 Agent / KB 打 tag (project / team / customer)
- 按 tag 分摊成本

Layer 3: 应用层埋点

- 每次 invoke 记 input/output tokens + model
- 按业务场景 / 用户 / 部门聚合
- CloudWatch Metrics 上报

Budget Alarm 必配

AWS Budgets:

- 月度预算 X 美元
- 80% / 100% / 120% 三级告警
- 超 100% 邮件 + Slack 通知 owner
- (生产环境慎用 auto-stop)

AWS 知识参考：搜 “AWS Cost Explorer for Bedrock”、“Bedrock prompt caching”、“Bedrock batch inference”。

13.3 灰度发布 — 不是 “all or nothing”

为什么必须灰度

没灰度的部署：

上线 → 发现问题 → 全量回退 → 用户全感知
 损失 = 全部 user × 故障时间

有灰度的部署：

1% → 监控 30 分钟 → 10% → ... → 100%
损失 = 1% user × 短时间
放大 100 倍止血空间

灰度策略

三种灰度方式

By percentage (流量百分比)

- 1% → 5% → 25% → 50% → 100%
- 适合：通用功能 / 大流量

By user / segment (用户分组)

- 内部员工先 → beta 用户 → 高级会员 → 全部
- 适合：风险大 / 商业关键功能

By feature flag (开关位)

- 同 binary, 配置中心控制开关
- 适合：A/B / 功能可热切

灰度的“门”

每个灰度阶段都要有“过门”条件：

从 1% 升 5%：

- ✓ 错误率 $\leq$ baseline + 0.5%
- ✓ P95 latency $\leq$ baseline + 200ms
- ✓ Eval 实时采样 $\geq$ baseline - 0.02
- ✓ 无 P1 告警

从 5% 升 25%：

- ✓ 上面 + 至少 30 分钟稳定
- ✓ 客户业务方书面 sign-off

AWS 实操：3 种灰度方案

方案 A：API Gateway Stage + Lambda Alias

- 一键切流量百分比 (Lambda traffic shifting)
- 简单 / Bedrock 应用最常用

方案 B：ECS / EKS 蓝绿 / Canary

- CodeDeploy + ECS service
- 支持全套灰度

方案 C：应用层 feature flag

- LaunchDarkly / AWS AppConfig
- 不用部署即可切流量

- 推荐：配合 Lambda 用
-

13.4 回滚 — 5 分钟内必须能切

Rollback 必须做到的 3 件事

1. 触发简单
一个命令 / 一个按钮 / 一个 PR revert
不要"5 步配置 + 重新发版"
2. 时间可控
从决定到生效 < 5 分钟
最好 < 1 分钟
3. 数据兼容
新版本写入的数据，旧版本能读
(向前兼容设计)

Rollback 检查清单

- ✓ 配置回滚 (config / prompt / model / KB version)
- ✓ 代码回滚 (Lambda alias / ECS service)
- ✓ 数据回滚 (DB schema / 嵌入数据)
- ✓ 前端回滚 (CDN cache 失效)

Prompt / Model / KB 回滚的特殊处理

LLM 应用的“回滚”不只是代码：

Prompt 回滚：

- 把 prompt 存在配置中心 (AppConfig / SSM Parameter Store)
- 不要写死在代码里
- 切换 = 改一个参数

Model 回滚：

- 应用读 model_id from config
- 切换 = 改 config 里的 modelArn

Knowledge Base 回滚：

- KB 版本化 (data source 变更前快照)
 - 应用读 KB id from config
 - 切换 = 改 config 里的 KB id
-

13.5 故障演练 — Chaos Engineering for LLM

必须演练的 5 种故障

1. 模型完全不可用
→ 切备用模型 (跨 region 或 跨 provider)
2. KB / 检索不可用
→ 应用降级到 "纯模型回答 + 风险提示"
3. 工具调用失败
→ Agent 优雅返回 + 工单创建
4. 单 Region 故障 (AWS)
→ DR 切换到备用 Region
5. 上下游限流 / 过载
→ Circuit breaker + queue + retry

怎么演练

季度：一次完整 DR 演练 (跨 region)
月度：一次小演练 (杀掉 KB / 模型 / 工具)
每周：trace 抽样回看 + 异常分析

13.6 一个生产化 dashboard 范例

Customer Insurance Assistant — Production Dashboard

[Health]

QPS: 23.4 (avg)
Error rate: 0.3% (baseline 0.4%) ☐
P95 latency: 1.8s (target <3s) ☐
Active users: 312

[Cost]

Today: \$89.2
MTD: \$1,847
Forecast: \$5,420 / mo
Budget: \$6,000 ☐

[Quality]

Sampled accuracy: 87.2% ☐
User thumbs-up: 89%
User thumbs-down: 4%
Unrated: 7%

[Eval Drift]

7-day: 0.872 ↘ (-0.005)
30-day: 0.876
Threshold: 0.85 ☐

[Top Failures (last 24h)]

- Tool 'get_policy_pdf' timeout: 12 cases

- Hallucination flag: 3 cases (sampled)
- Guardrail block (PII): 18 cases (intended)

[Canary]

Current rollout: 100%

Last change: 2026-05-19 14:00 (prompt v2.3.1)

Auto-rollback armed: ☐

这个 dashboard 在客户的 oncall 屏幕上, FDE 在自己屏幕上, 两边看同一份。

关键引用

“A system without observability is a system you don’ t own.” — A. Lawrence, FDE Rule Book, 2025

“Every dollar saved by caching is a dollar of production runway.” — Anthropic enterprise best practices, 2025

“If you can’ t roll back in 5 minutes, you can’ t deploy on Friday.” — AWS GenAI Innovation Center, 2025

动手清单

PoC 第 4-6 周 + 上线前必做:

1. 接 CloudWatch + X-Ray + Bedrock Logging (缺一不可)
 2. 建 6 卡片 dashboard (13.1 节)
 3. 配 Cost Explorer Tag + AWS Budgets 月度告警
 4. prompt / model / KB 全部走配置中心 (不要写死)
 5. CI/CD 加 canary deploy (API Gateway / Lambda Alias)
 6. 写 “5 分钟 rollback SOP”: 从决定到生效流程
 7. 第一次故障演练 (杀掉 KB 看 graceful degrade)
-

反模式清单

- ☒ prompt / model 写死在代码里 (每次改要发版)
- ☒ 没接 cost 监控就上线 (账单到月底惊喜)
- ☒ 灰度只有 0% 和 100% (出问题全量受灾)
- ☒ rollback 靠 “重新部署上一版” (5 分钟变 50 分钟)
- ☒ dashboard 客户看不到 (客户 oncall 不知道发生了什么)
- ☒ 不演练就相信 “理论上能切” (真出事都来不及)
- ☒ 告警太多 → 麻木 (只对真正可执行的告警告警)

与下一 Part 的关系

到这里, “PoC → 生产” 的鸿沟跨过了: 你的 LLM/Agent 已经在客户生产环境稳定运行。

下一 Part 进入 **Agent 时代** —— 不是 “加个 RAG” 那种 Agent, 而是真正 “自主决策 + 工具调用 + 跨系统办事” 的 Agent。FDE 在这一阶段的工程任务是新的。

[← 上一章: PoC 过线条件](#) · [下一 Part: Agent 时代](#) →

Part VI: Agent 时代 — FDE 的新工程任务

适用范围：LLM 应用主线（数据驱动主线选读）

这个 Part 要解决什么问题

2025-2026 是 Agent 真正进入企业的一年。

不是“客服回答问题”那种 RAG，而是：- 自动写 / 改 / 提交 PR - 自动跨系统办事（CRM + ERP + 邮件 + 日历）- 自动用浏览器 / 终端 / API 完成任务

Agent 给 FDE 带来的新工程问题：

1. 工具集设计 — 不是“能多就多”，是“够用 + 不出错”
2. 沙箱与权限边界 — Agent 越权后果不可控
3. 失败恢复 — 多步任务中断怎么续
4. 企业集成 — MCP（Model Context Protocol）怎么把 Agent 接到客户工具上

这个 Part 给两章实操：

- **Chapter 14**：在客户环境部署 Agent — 工具集 / 沙箱 / 失败恢复
- **Chapter 15**：MCP 与企业集成 — 把 Agent 接到客户工具上

包含章节

详见 [SUMMARY.md](#)。

与其他 Part 的关系

- 前置：Part III (Eval) + Part IV (数据 / 集成) + Part V (生产化) 全部是 Agent 的基础
 - 后续：Part VII 完成交付与精进
-

Chapter 14: 在客户环境部署 Agent — 工具集 / 沙箱 / 失败恢复

开场

某 FDE 给客户做了一个"销售助理 Agent"，工具列表 47 个：

- 查 CRM
- 改 CRM
- 查 ERP
- 改 ERP
- 发邮件
- 改日历
- 创建工单
- 关单
- 调价
- 发优惠券
- ... 一直到第 47 个

第 1 周演示完美。

第 3 周客户业务方私下找 FDE：

"这周 Agent 连续给 3 个客户发了 100% 折扣优惠券，
损失约 ¥20 万。"

复盘：模型在某些 prompt 下"自作主张"调用 `send_coupon(100)`。
没有 sandbox。没有金额上限。没有 dry-run。

FDE 当夜重写：

- 47 个工具砍到 18 个（合并 + 砍掉危险）
- 写、改类工具加了"二次确认 + 金额上限 + dry-run"
- 加了 audit trail 和 alert

第 5 周再上线，0 事故。

这一章给：Agent 在客户环境部署的 4 件事 ——
工具集设计 / 沙箱 / 失败恢复 / 评估。

14.1 工具集设计 — 少即是多

工具数量的“魔法数字”

Agent 工具数对应准确率（经验）

≤ 5	95%+ 准确率，简单
6-10	90% 准确率，工程友好
11-20	80% 准确率，需要精心设计 prompt
21-30	70% 准确率，建议拆分
31-50	60% 准确率，不可控
> 50	"工具地狱"，准确率玄学

第一性原理：模型选错工具的概率随工具数量指数级上升。

工具集设计的 4 条原则

1. 一个动词 + 一个对象
 - ✓ `create_ticket(title, body)`
 - ✗ `smart_helper(action, params)` （太宽）
2. 描述要“对模型友好”
 - ✓ `"Returns customer order history. Use when user asks about past purchases."`
 - ✗ `"Get orders"` （模型不知道何时调用）
3. 参数要严格
 - ✓ JSON Schema 强校验
 - ✓ Required vs Optional 明确
 - ✓ Enum 限定取值
4. 危险操作分级
 - read 类：直接执行
 - write 类：二次确认 / dry-run
 - 大金额 / 不可逆：多人审批

工具描述模板

```
# 一个好的工具定义示例
{
  "name": "send_email",
  "description": (
    "Send an email to specified recipients. "
    "Use this tool when user explicitly asks to send/notify someone. "
    "Do NOT use for internal logging or status updates. "
    "Maximum 5 recipients per call."
  ),
```



```



```

14.2 沙箱 — Agent 的“权限边界”

三层防线

Agent 的执行沙箱三层

Layer 1: 模型层

- System prompt 严格限定行为
- Guardrails 拦危险输出
- Few-shot 示例引导

Layer 2: 工具层

- 危险工具二次确认
- dry-run 模式默认开
- 金额 / 数量 / 频率上限

Layer 3: 执行层

- IAM / 数据库权限隔离
- VPC 网络隔离
- 速率限流 + 熔断

关键技术：dry-run 模式

```
# 默认 dry_run=True 的设计模式
def send_coupon(customer_id, amount, dry_run=True):
    if dry_run:
        return {
            "status": "dry_run",
            "would_send_to": customer_id,
            "amount": amount,
            "note": "Set dry_run=false to actually send"
        }
    # 实际发送逻辑...
```

Agent 调用任何 write 类工具，第一次必须 dry_run。模型自己决定第二次设 dry_run=false（带二次确认机制）。

关键技术：金额 / 频率限制

```
# 工具内部硬限制
MAX_COUPON_AMOUNT = 50 # 元
MAX_COUPON_PER_HOUR = 10 # 次

def send_coupon(customer_id, amount):
    if amount > MAX_COUPON_AMOUNT:
        return {"error": f"Amount exceeds limit ({MAX_COUPON_AMOUNT})"}
    if rate_limit_exceeded("coupon", "1h", MAX_COUPON_PER_HOUR):
        return {"error": "Rate limit exceeded"}
    # 实际发送
```

这些限制不能在 prompt 里写（prompt injection 可绕过），必须在工具实现层硬编码。

权限隔离 — 工具用 user context, 不用 service account

错误做法：

Agent → 工具（用 admin role）→ DB
→ 任何用户都能读 / 改任何数据

正确做法：

用户登录 → 拿到 user_token
Agent（带 user_token）→ 工具 → 用 user_token 调 DB
→ 用户只能看 / 改自己的数据

AWS 实操：Bedrock Agents 的权限模型

Bedrock Agent 权限三层

1. Agent execution role

- Agent 本身的 IAM role
- 通常仅能调 Bedrock + 自己的 KB

2. Action group Lambda

- 每个 action group 可以单独的 Lambda
- Lambda 用自己的 role 调下游服务

3. User context (passed via session attributes)

- 用户登录信息 (cognito / IAM identity)
- Lambda 用这个 context 做 ABAC 决策

- 不是给 Agent 一个万能 role
- 是给 Agent 调用 Lambda 的权限，Lambda 内部根据 user 做权限决策

AWS 知识参考：搜 “Bedrock Agent session attributes”、“Bedrock Agent execution role”。

14.3 失败恢复 — 多步任务中断怎么办

Agent 任务的 “断点”

典型 Agent 任务路径

用户：“帮我把上周所有 P1 工单转给小张并通知他”

Agent:

Step 1: list_tickets(status="P1", week="last") → 5 条
 Step 2: assign_ticket(id=#101, to="zhang") ✓
 Step 3: assign_ticket(id=#102, to="zhang") ✓
 Step 4: assign_ticket(id=#103, to="zhang") ✗ (网络抖动)
 Step 5: ...

问题:

- 现在 #101 #102 已转, #103 失败
- 重跑会重复转 #101 #102 吗?
- 模型可能放弃这个任务

解决: Idempotency + State 持久化

工具调用必须 *idempotent*

```
def assign_ticket(ticket_id, assignee, idempotency_key=None):
    if not idempotency_key:
        idempotency_key = f"assign-{ticket_id}-{assignee}"
```

```
# 重复调用同一 key 返回上次结果
if cache.exists(idempotency_key):
    return cache.get(idempotency_key)

result = actually_assign(ticket_id, assignee)
cache.set(idempotency_key, result, ttl=86400)
return result
```

Agent 执行状态持久化

```
class AgentSession:
    def __init__(self, session_id):
        self.session_id = session_id
        self.state = load_from_dynamodb(session_id) or {
            "task": None,
            "completed_steps": [],
            "pending_steps": []
        }

    def execute(self, task):
        for step in self.state["pending_steps"]:
            try:
                result = run_step(step)
                self.state["completed_steps"].append(step)
                save_to_dynamodb(self.session_id, self.state)
            except RetryableError:
                # 留在 pending, 下次会续
                save_to_dynamodb(self.session_id, self.state)
                raise
```

AWS 实操：Step Functions 给 Agent 兜底

Step Functions 包 Agent

适合场景：

- 单次 Agent 任务执行 > 5 分钟
- 多个 Agent 协作
- 需要持久化中间状态
- 需要 human-in-the-loop

优势：

- 可视化执行历史
- 自动重试 + 指数退避
- 失败可断点恢复
- 可暂停 + 等审批 + 续

反模式：

- 简单单步 Agent → Step Functions 过度
- 高频低延迟 → 不适合

AWS 知识参考: 搜 “Step Functions express workflows for AI”、“Step Functions human approval”。

14.4 Agent Eval — 不一样的 Eval

普通 LLM 应用 Eval 看 “答案对不对”。Agent Eval 还要看:

Agent 评估的 5 维度

1. 任务完成率
用户的目标是否最终达成?
binary: 是 / 否
2. 路径正确性
执行步骤是否合理? 是否冗余?
metric: 步骤数 / "标准路径"步骤数
3. 工具使用准确性
选对工具了吗? 参数对吗?
metric: tool_call_accuracy
4. 副作用控制
有没有"做了不该做的事"?
metric: side_effect_count
5. 体验成本
总耗时 / token 成本 / 重试次数
metric: latency, cost, retries

Agent Eval 的样本设计

```
{
  "id": "agent-eval-007",
  "task": "把上周所有 P1 工单转给小张",
  "context": {"current_user": "manager-001"},
  "expected_outcome": {
    "tickets_assigned": ["#101", "#102", "#103", "#104", "#105"],
    "all_to": "zhang",
    "side_effects_allowed": ["notify_zhang"],
    "side_effects_forbidden": ["close_ticket", "notify_customer"]
  },
  "expected_path": {
    "min_steps": 6,
```

```

    "max_steps": 12,
    "must_use_tools": ["list_tickets", "assign_ticket"],
    "must_not_use_tools": ["delete_ticket", "send_coupon"]
  }
}

```

AWS 实操: Bedrock Agent Evaluations

Bedrock 内置 Agent Evaluation:

- 自动评 trajectory (路径)
- 评 tool selection accuracy
- 评 task success rate
- 支持自定义 metric

14.5 Human-in-the-loop — 高风险动作必有人审

三种 HITL 模式

1. Always-approve (高风险)
 - 删数据 / 大金额转账 / 发外部邮件
 - Agent 准备 → 推送审批 → 人点 → 执行
2. Sample-approve (中风险)
 - 改 CRM / 创建工单
 - Agent 直接做
 - 但 5% / 10% 抽样推审批 (后审 + 校准)
3. No-approve (低风险)
 - 查询 / 报表 / 内部备忘
 - Agent 全自主

实操架构

```

Agent → 检测高风险动作 → 写到 SQS / DynamoDB
      ↓
      (通知人 via Slack / 邮件)
      ↓
      人在 Web UI 审批
      ↓
      Step Functions 续作
      ↓
      Agent 执行

```

14.6 部署清单

Agent 上生产前 checklist

- 工具数 ≤ 20 (超过强行拆分)
 - 每个工具有完整 description + JSON schema
 - 写、改、删类工具默认 dry_run
 - 金额 / 频率 / 数量上限硬编码
 - 用户 context 透传到工具 (不用 service account)
 - Idempotency key 强制
 - 状态持久化 (DynamoDB / Step Functions)
 - Bedrock Guardrails: PII + Topic + Content
 - 高风险动作走 HITL
 - Trace + Cost dashboard
 - Eval 集涵盖 task / path / tool / side-effect 4 维
 - 灰度 + Rollback 通道
 - 故障演练 (模型挂 / 工具挂)
-

14.7 一个端到端的 Agent 部署

客户场景：保险公司"理赔助理 Agent"

工具集 (12 个):

[read]

- get_policy(policy_id)
- get_claim_history(customer_id)
- get_doctor_records(claim_id)
- search_clauses(keyword)

[write 简单]

- create_followup_ticket(claim_id, note)
- send_internal_msg(team, msg)

[write 重要]

- request_more_info(claim_id, items[]) → dry_run + 业务员审批
- flag_for_review(claim_id, reason) → 主管审批
- approve_claim(claim_id, amount) → 人审批 always
- reject_claim(claim_id, reason) → 人审批 always

[escalate]

- escalate_to_human(claim_id, reason)
- request_legal_review(claim_id)

沙箱:

- approve_claim 必须 amount ≤ 5000 元
- approve_claim 必须 HITL
- 所有 write 工具用 user_id (业务员的) 调用

失败恢复：

- 任务通过 Step Functions 编排
- 每步状态写 DynamoDB
- 重试 3 次，仍失败 → escalate

Eval：

- 100 条历史理赔做 golden
- 评：
 - approval/rejection 决策 vs 人工实际决策一致率
 - 平均工具数（标准 5-8 步）
 - 错误工具调用率

上线：

- W11 灰度 1%（内部业务员）
 - W12 灰度 10%（低风险 case）
 - W14 50%
 - W16 100%
-

关键引用

“An agent without a sandbox is a liability.” — A. Lawrence, FDE Rule Book, 2025

“The best agents have boring tools.” — Anthropic Claude tool use guide, 2025

“Don’ t ship an agent until you’ ve watched it fail safely 100 times.” — AWS GenAI Innovation Center, 2025

动手清单

接到 Agent 项目，部署前必做：

1. 画工具列表 + 标 read/write/delete 类型
 2. 任何 “write” 工具加 dry_run + 上限
 3. 用户 context 透传，不用 service account
 4. 接 Idempotency 缓存（DynamoDB / Redis）
 5. Bedrock Guardrails 配 PII / 业务红线
 6. Step Functions 包高风险任务（断点恢复）
 7. Eval 集 4 维齐全（task / path / tool / side-effect）
 8. HITL 配置（金额 / 不可逆 / 外部影响）
-

反模式清单

- ☒ 工具数 > 30 直接上线（准确率玄学）
 - ☒ 不接 `dry_run` 直接 `write`（事故 #1 来源）
 - ☒ Agent 用 `admin / service account` 调下游（任何用户都越权）
 - ☒ Agent 失败靠“模型自己重试”（没有 idempotency 重复操作）
 - ☒ Agent 高风险动作不走 HITL（损失到客户面前才发现）
 - ☒ 用普通 Eval 评 Agent（漏掉 trajectory / side-effect）
 - ☒ 没演练故障就 100% 上线（凌晨 2 点见真章）
-

与下一章的关系

这一章解决了 Agent 在“自己环境”的工程问题。下一章解决：Agent 怎么“接到客户工具上”——MCP (Model Context Protocol) 时代的企业集成。

[← Part VI 导读 · 下一章: MCP 与企业集成 →](#)

Chapter 15: MCP 与企业集成 — 把 Agent 接到客户工具上

开场

2024 年 11 月 Anthropic 发布 MCP (Model Context Protocol)。

2025-2026 年，企业 ToB 项目几乎全部要求 MCP 适配。

某 FDE 第一次接 MCP：

- 客户已有：Confluence + Jira + Salesforce + 内部 Wiki
- 客户要的："Claude / Bedrock Agent 能用上面所有工具"
- 之前的做法：一个个写 LangChain Tool → 6 周
- 用 MCP 的做法：
 - Confluence MCP server (社区已有)
 - Jira MCP server (社区已有)
 - Salesforce MCP server (FDE 写一个)
 - 内部 Wiki MCP server (FDE 写一个)
 - 配置 Claude Desktop / Bedrock Agent 接入
 - 1 周

10 倍开发效率。

但 MCP 的"威力" 同时是"风险"：

- 一个 MCP server 暴露了客户内部 API
- Agent 可以一次性访问 50+ 工具
- 安全 / 审计 / 权限边界 100% 由 FDE 设计

这一章讲：在客户企业里部署 MCP 的工程实操。

15.1 MCP 是什么

没有 MCP 的世界

MCP 之后的世界

Agent 1 集成 Tool A	所有 Agent 共用 MCP 协议
Agent 1 集成 Tool B	
Agent 2 集成 Tool A	Tool A 实现一次 MCP server
Agent 2 集成 Tool B	Tool B 实现一次 MCP server

$N \times M$ 的集成噩梦 $N + M$ 的标准接口

MCP 本质是 LLM 与工具之间的 USB-C 接口：

MCP 协议三层

Client (LLM 应用)

Claude Desktop / Cursor / Bedrock Agent / 自建 Agent

↓ JSON-RPC over stdio / SSE / HTTP

Server (工具实现)

File / Slack / Jira / Salesforce / 内部 API ...

↓

Resource (工具暴露的能力)

- tools (函数调用)
- resources (文件 / 数据)
- prompts (预定义 prompt 模板)

一个 MCP server 暴露 3 类能力

Tools — 让 LLM 调用的函数

例: `list_jira_issues(status, project)`

Resources — 让 LLM 读的资源

例: `file:///docs/policy.md`
`confluence:///wiki/space/PROD/pages/123`

Prompts — 预定义的 prompt 模板

例: "summarize_pr" 模板
 "review_security" 模板

15.2 企业部署 MCP 的 3 种形态

形态 1: 本地 MCP server (开发者机器)

场景: Cursor / Claude Desktop 用

部署: 进程启动, stdio 通信

适合: 个人 / 小团队

形态 2: 远程 MCP server (HTTP/SSE)

场景: 企业内部 + 多人共享

部署: K8s / ECS / Lambda + API Gateway

适合: 企业级

形态 3: 集中式 MCP gateway
场景: 有 50+ MCP servers 要管理
部署: 一个网关聚合所有 server
适合: 大型企业 + 多租户

企业 ToB 项目 90% 是形态 2, 部分大客户是形态 3。

15.3 写一个企业 MCP server — 实操

以 “Salesforce CRM MCP server” 为例:

Step 1: 设计 tools

```
# 把客户业务关键操作列出来
tools = [
    "list_accounts",
    "get_account_details",
    "search_opportunities",
    "create_task", # write 类
    "log_activity", # write 类
    "update_stage", # write 类, 需 confirm
]
```

Step 2: 实现 MCP server (Python SDK)

```
from mcp import Server, Tool
from mcp.types import TextContent
import simple_salesforce as sf

server = Server("salesforce-mcp")

@server.tool()
async def list_accounts(name_contains: str = None, limit: int = 10):
    """List Salesforce accounts. Use when user asks about customers/accounts."""
    sf_client = get_sf_client() # uses user-scoped OAuth token
    query = "SELECT Id, Name, Industry FROM Account"
    if name_contains:
        query += f" WHERE Name LIKE '%{name_contains}%'"
    query += f" LIMIT {limit}"
    results = sf_client.query(query)
    return TextContent(text=json.dumps(results['records']))

@server.tool()
async def update_stage(opp_id: str, stage: str, dry_run: bool = True):
```

```

"""Update opportunity stage. Use only when user explicitly asks to change stage."""
if dry_run:
    return {"status": "dry_run", "would_set": stage}
sf_client = get_sf_client()
sf_client.Opportunity.update(opp_id, {'StageName': stage})
return {"status": "updated"}

if __name__ == "__main__":
    server.run()

```

Step 3: 鉴权 — 关键

企业 MCP server 鉴权 3 模式

Mode A: 用户 OAuth (推荐)

客户登入 → 拿到 token → 启动 MCP server
 → MCP server 用 token 调下游
 → 下游决定权限

Mode B: Service Account

MCP server 用 service account 调下游
 问题: 任何人都能用 MCP 越权
 只在内部 admin 工具用

Mode C: Per-call Token Forwarding

每次 MCP 调用带用户 token
 MCP server 透传 token 给下游
 最严谨, 但实现复杂

企业生产: Mode A 或 Mode C。Mode B 仅限受控的内部工具。

Step 4: 部署到客户 VPC

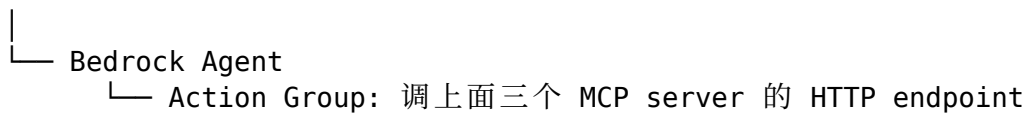
企业部署架构

客户 VPC

```

├── ECS Service: salesforce-mcp-server
│   ├── ALB (HTTPS, 客户证书)
│   ├── 容器 (Python, MCP SDK)
│   └── Secrets Manager: SF OAuth 配置
├── ECS Service: jira-mcp-server
│   └── ...
├── ECS Service: confluence-mcp-server
│   └── ...

```



15.4 MCP 的 4 个工程坑

坑 1: MCP server 暴露过多

新人写 MCP server 倾向 “把所有 API 都暴露”。结果：

- ✗ Salesforce MCP 暴露 80 个 tools
 - Agent 选错率 60%
 - 维护成本爆炸
- ✓ Salesforce MCP 暴露 8 个高频 tools
 - 90% 业务场景覆盖
 - 准确率 90%+

经验：每个 MCP server 的 tools 控制在 5-15 个。多了拆 server。

坑 2: 参数 schema 不严格

```
# 反例
@server.tool()
async def query(text: str): # text 是啥?
    ...

# 正例
@server.tool()
async def search_opportunities(
    name_contains: str = None,
    stage: Literal["Prospecting", "Qualification", "Negotiation"] = None,
    amount_min: int = None,
    limit: int = 10
):
    """Search Salesforce opportunities..."""
```

严格 schema = 模型选错率大幅下降。

坑 3: 错误处理不规范

```
# 反例
try:
    result = sf_client.query(...)
    return result
except Exception:
```

```

    return None # 模型不知道发生了什么

# 正例
try:
    result = sf_client.query(...)
    return {"status": "ok", "data": result}
except sf.SalesforceMalformedRequest as e:
    return {"status": "error", "type": "bad_query", "message": str(e), "hint": "check fi"}
except sf.SalesforceAuthenticationFailed:
    return {"status": "error", "type": "auth_failed", "message": "OAuth token expired",}
except Exception as e:
    return {"status": "error", "type": "unknown", "message": str(e)}

```

让模型能“读懂”错误并自己恢复。

坑4：缺审计

```

# 每个 MCP 调用都要写审计
@server.tool()
async def update_stage(opp_id, stage):
    audit_log({
        "user_id": current_user_id(),
        "tool": "update_stage",
        "params": {"opp_id": opp_id, "stage": stage},
        "timestamp": now(),
        "trace_id": get_trace_id()
    })
# 实际执行

```

没有审计的 MCP server 不可上生产。

15.5 AWS 实操：Bedrock Agent + MCP 集成

AWS 上的 MCP 部署架构

```

客户 Bedrock Agent
  ↓ (Action Group)
Lambda: mcp-bridge
  ↓ (HTTP/SSE)
ALB / API Gateway
  ↓
ECS Fargate: MCP servers
├─ salesforce-mcp
└─ jira-mcp

```

```
├─ confluence-mcp
├─ internal-wiki-mcp
```

↓

下游系统（各自 SaaS / 内部 API）

最小 Lambda 桥接代码：

```
# Lambda: mcp-bridge
import boto3
import requests

def lambda_handler(event, context):
    # Bedrock Agent 调来
    action_group = event['actionGroup']
    api_path = event['apiPath']
    parameters = event['parameters']

    # 转发到对应的 MCP server
    mcp_url = MCP_SERVER_MAP[action_group]
    user_token = event['sessionAttributes'].get('user_token')

    response = requests.post(
        f"{mcp_url}/tools/call",
        json={"name": api_path, "arguments": parameters},
        headers={"Authorization": f"Bearer {user_token}"}
    )

    return {
        'response': {
            'actionGroup': action_group,
            'apiPath': api_path,
            'statusCode': response.status_code,
            'responseBody': {
                'application/json': {'body': response.json()}
            }
        }
    }
```

AWS 知识参考：搜 “Bedrock Agent action group OpenAPI”。MCP 与 Bedrock 的官方桥接方案在 2025-2026 仍在演进，关注 AWS 官方更新。

15.6 多 server 协作 — 一个客户场景

场景：销售助理 Agent

Agent 接的 MCP servers:

1. crm-mcp (Salesforce, 8 个 tools)

2. email-mcp (Outlook / Gmail, 5 个 tools)
3. calendar-mcp (Google Calendar, 4 个 tools)
4. wiki-mcp (内部 Confluence, 3 个 tools)
5. order-mcp (内部 ERP, 6 个 tools)

总工具数 26 → 控制在 14.1 节"魔法数字 ≤ 30 "内

典型对话：

用户："客户 ABC Corp 上次反馈了什么？我下周要拜访他。"

Agent 路径：

- Step 1: `crm-mcp.search_accounts("ABC Corp")`
- Step 2: `crm-mcp.get_account_details(account_id="...")`
- Step 3: `crm-mcp.list_recent_activities(account_id="...")`
- Step 4: `email-mcp.search_emails(from="abc.com", days=30)`
- Step 5: `wiki-mcp.search("ABC Corp")`
- Step 6: 综合回答
- Step 7 (用户确认后): `calendar-mcp.create_event(...)`

26 个工具分布在 5 个 server，每个 server 内聚 + 单一职责。

15.7 安全清单

企业 MCP 部署安全清单

- ☐ 用户 OAuth 而非 service account
 - ☐ MCP server 在 VPC 内，HTTPS 强制
 - ☐ 每个 tool 有详细 description + JSON schema
 - ☐ Write 类 tool 默认 dry_run
 - ☐ 危险操作 HITL
 - ☐ 全链路 trace_id
 - ☐ 审计日志 (who / what / when / result)
 - ☐ Bedrock Guardrails 双重保险
 - ☐ Rate limit per user (不只是 per IP)
 - ☐ 工具 enable/disable 通过配置中心 (不是代码改)
 - ☐ 最小工具集 (每个 server ≤ 15)
 - ☐ MCP server 单独漏洞扫描 + pen test
-

关键引用

“MCP turned tool integration from $O(N \times M)$ to $O(N+M)$.” — Anthropic MCP launch, 2024-11

“A poorly designed MCP server is a backdoor with documentation.” — A. Lawrence, FDE Rule Book, 2025

“The future enterprise agent will use 50+ MCP servers — the FDE’s job is to make all 50 boring.” — AWS GenAI Innovation Center, 2025

动手清单

接到 MCP 集成项目第 1 周必做：

1. 盘点客户工具 — 哪些 SaaS / 哪些内部 API
 2. 看社区有没有现成 MCP server (Awesome MCP / Anthropic 官方仓库)
 3. 没有的写 MCP server — Python SDK 一两天能上
 4. 用户 OAuth 接入 (不要 service account)
 5. 每个 tool 写完整 description + schema
 6. 接 trace + 审计日志
 7. 写 Eval 集 (包括 “误调” 的反例)
 8. VPC 内部署 + HTTPS + 鉴权
-

反模式清单

- ☒ 第一版就把 60 个 tool 全暴露 (准确率玄学)
 - ☒ MCP server 用 service account 跑 (鉴权事故 #1)
 - ☒ 没 schema 让 LLM “猜参数” (错误率飙升)
 - ☒ 错误返回 None (模型不知道发生了什么)
 - ☒ MCP server 部署在公网 (任何人扫到都能调)
 - ☒ tools 粒度过粗 (“smart_helper” 这种万能函数)
 - ☒ 没审计就上 (合规直接 fail)
-

与下一 Part 的关系

到这里, FDE 在客户环境从 “PoC → 生产 → Agent → 企业集成” 的全流程已经走完。

最后一个 Part: 交付与精进 —— Handoff 让客户接走、模式提取沉淀成可复用资产、FDE 自身的 T 字成长。

← 上一章: 在客户环境部署 Agent · 下一 Part: 交付与精进 →

Part VII: 交付与精进

适用范围：通用

这个 Part 要解决什么问题

90% 的 FDE 项目 “上线了”，但 60% 的项目在上线 3 个月后陷入 “半死不活”：

- 客户运维不真懂，出问题打电话给 FDE
- FDE 公司侧没有人接手，没人维护
- 项目特定的 prompt / 数据 / 经验都在 FDE 一个人脑袋里
- 客户问 “再做下一个项目”，FDE 不知道哪些能复用

这个 Part 解决两件事：

1. **Handoff** — 项目 “真的” 交给客户的工程方法
2. **模式提取 + T 字成长** — 把这次项目的产出沉淀成 “下一个项目能用的资产”，并构建 FDE 的长期能力

包含章节

- **Chapter 16**: Handoff + 模式提取 — 把方案抽象成可复用资产
- **Chapter 17**: FDE 的 T 字成长 — 工程深度 + 行业纵深

与其他 Part 的关系

- **前置**：前 6 个 Part 全部
 - **后续**：完成本 Part 后，FDE 进入 “下一个项目”，循环
-

Chapter 16: Handoff + 模式提取 — 把方案抽象成可复用资产

开场

某 FDE 完成了一个 12 周的 Agent 项目，技术上很成功。

3 个月后，客户运维找她："那个 prompt 我们想改，怎么改？"
她飞过去，2 小时改完。

6 个月后，客户："Agent 答错率上升了，您看看"
她飞过去，1 天 debug。

9 个月后，客户："我们想再加一个 use case"
她飞过去，打开自己的旧代码，发现自己也有点忘了。

FDE 的同事接到一个新客户的相似项目，跑过来问她：

"这个怎么做来着？"

她回答："你看我之前的代码 ...

但每个客户都不一样，重新做一遍吧。"

她公司的 CEO 看到第三个客户的报价，
问她："为什么我们 3 个相似项目都报 12 周？应该 4 周才对。"

她答不上来。

这一章讲两件事：

1. Handoff — 让客户真正接走（你不再被电话叫飞过去）
2. 模式提取 — 让下一个相似项目 4 周做完

16.1 Handoff 的工程定义

Handoff = "客户能在没你的情况下独立运营这个系统"

判断标准（4 个能力）：

-
1. 客户能独立 deploy 一次 hot fix
(改 prompt / 改 KB / 改 model id)
 2. 客户能独立读 dashboard 找根因
(没有 trace 你帮他看)
 3. 客户能独立跑一次 Eval
(升级模型前不再依赖你)
 4. 客户能独立处理 Top 5 故障类型
(而不是 P1 就打电话给你)
- 4 个全部达到 → 真 handoff。少 1 个 = 项目还没交付完。
-

16.2 Handoff 的 “3 周倒计时”

不是上线那一刻才开始 handoff，是上线前 3 周开始：

	T-3 周	T-2 周	T-1 周	T 上线	T+4 周
	计划 启动	培训 + 文档	影子 运营	独立 运营	FDE 退出
T-3	✓ 找客户运维 owner (人 + 邮箱 + 排班) ✓ 写 Runbook 大纲 ✓ Dashboard 给客户访问				
T-2	✓ 4 小时培训 (Runbook + dashboard + Eval) ✓ 让客户 owner 操作一次每个动作 ✓ Q&A				
T-1	✓ 客户 owner 跟着 FDE 一起处理所有事情 ✓ FDE 不主动操作，只 review 客户操作 ✓ 一周末客户 owner 写一份 "我学到的"				
T	上线 + 灰度 ✓ 客户 owner 主导，FDE 后排				
T+4	FDE 完全撤离 ✓ 仍 on-call 但不主动看 ✓ 客户每周报告自己的运营状态				

16.3 Runbook — 给客户的“操作手册”

Runbook 不是文档，是可操作的指令清单。

Runbook 必备的 7 个 section

1. 系统架构图（一页 A4）
2. 部署 / 回滚步骤（命令 / 截图）
3. Top 10 故障类型 + 处理 SOP
4. Eval 怎么跑 + 阈值含义
5. 关键配置在哪 + 怎么改
6. 数据 / KB 更新 SOP
7. Escalation 路径（什么情况打谁电话）

Runbook 的 SOP 范例

SOP-001: Agent 错误率突然上升

现象：dashboard 显示错误率 > 1%（基线 0.3%）

Step 1: 检查 Bedrock 服务状态

- 打开 <https://health.aws.amazon.com>
- 看 us-east-1 Bedrock service health
- 如果 AWS 故障 → 等 + 通知用户

Step 2: 检查最近 commit

- GitLab → main 分支 last 5 commits
- 是否有 prompt / KB / model 变更
- 如有 → 走 Step 3 (rollback)

Step 3: Rollback

- 命令：`./scripts/rollback.sh prod --target-version=$LAST_GOOD`
- 等 5 分钟看 dashboard
- 错误率回到基线 → 完成
- 没回到 → 走 Step 4

Step 4: 联系 FDE on-call

- Slack: @fde-oncall
- 电话: +XX-XXX-XXXX (24h)

Step 5: 写故障报告

- 模板: docs/incident-template.md
 - 24 小时内提交
-
-

好的 Runbook 让客户 P1 故障 5 分钟止血。

16.4 培训 — 4 小时课程

Handoff 培训 4 小时议程

Hour 1: 系统架构 + 业务流

- └─ 数据从哪来到哪去 (15 min)
- └─ Agent 的工具集和能力边界 (15 min)
- └─ Eval 是干嘛的 (15 min)
- └─ 监控仪表盘逐项讲 (15 min)

Hour 2: 日常运营

- └─ 怎么看 dashboard (实操) (20 min)
- └─ 怎么跑 Eval (实操) (20 min)
- └─ 怎么改 prompt + 灰度 (实操) (20 min)

Hour 3: 故障处理

- └─ Top 10 故障演练 (动手) (40 min)
- └─ Rollback 演练 (实操) (20 min)

Hour 4: 数据 / KB 维护 + Q&A

- └─ KB 更新 SOP (实操) (30 min)
- └─ Q&A (30 min)

关键：每个环节都要客户亲手操作，不是看 FDE 演示。

16.5 模式提取 — 让下一个项目快 5 倍

什么是“模式提取”

每个项目结束时，问 4 个问题：

- Q1: 这个项目里哪些工作"换个客户也基本一样"？
→ 这是可复用资产
- Q2: 哪些工作"花了大量时间但下次能避免"？
→ 这是工程模板的来源
- Q3: 哪些"客户特有的"实际上很多客户都有？
→ 这是行业模板的来源
- Q4: 哪些"我以为简单实际很难"？
→ 这是预警卡片的来源

模式提取产出 — 4 类资产

FDE 的"项目后资产库"

1. 代码模板
 - LLM RAG 起手包
 - Bedrock Agent 起手包
 - Eval CI 起手包
 - Lambda MCP server 起手包
2. 文档模板
 - Discovery 报告模板 (Ch 4)
 - SOW 模板 (Ch 5)
 - Runbook 模板 (本章)
 - 验收标准模板
3. 决策卡片
 - "客户问这个 → 答这个" 速记
 - "出现这个信号 → 切到这种解法"
 - "这种问题先做这 3 件事"
4. 反模式案例集
 - 真实故障复盘
 - 错误代价 + 教训

模板“颗粒度”经验

- 太粗: "Bedrock 起手模板"
 - 一上来还是要大改
 - ✓ 适中: "Bedrock + Knowledge Bases + Aurora pgvector
VPC 部署 + IAM Identity Center 起手模板"
 - 客户 80% 用例直接用
 - 太细: "招商银行私有云 RAG 起手模板"
 - 颗粒度过细只能用一次
-

16.6 模式提取的工程动作

每个项目结束 1 周内做 5 件事:

1. 写 1 页 "项目复盘"
 - 我们做了什么
 - 哪些做对了 / 做错了
 - 数字 (outcome / Eval / 成本 / 时间)

2. Code review 自己整个项目
 - 哪些代码块"显然能复用"
 - 抽出来放公司 internal repo
 3. 抽 3 个"决策瞬间"
 - 那个时刻你判断了什么 → 写成卡片
 4. 抽 1-2 个"如果重来要怎么做"
 - 写成"项目模板 v X+1"
 5. 跟同事开 1 小时 brown-bag
 - 不是炫耀成功，是讲"你不知道的坑"
-

16.7 行业模板 — 一个例子

行业：保险公司

模板："保险 RAG/Agent 起手包 v3.2"

内容：

- ├─ 行业知识
 - ├─ 保险公司组织架构常见样
 - ├─ 核保 / 理赔 / 销售 三条主流
 - ├─ 通用监管要求（银保监 / 等保）
 - └─ 通用数据栈（核心 + ECIF + 渠道 + 风控）
- ├─ 通用 Discovery 模板
 - ├─ 12 个保险特有的问题
 - └─ 5 类关键产出物清单
- ├─ 代码模板
 - ├─ 保单条款分片（PDF + 表格）
 - ├─ 客户身份匹配（ID 卡 / 客户号 / 保单号 mapping）
 - ├─ Bedrock + Aurora pgvector + KMS 部署 IaC
 - └─ 等保审计日志规范
- ├─ Eval 模板
 - ├─ 保险问答 200 条 golden
 - ├─ 安全性 50 条（PII / 不当承诺）
 - └─ 业务专家校准流程
- └─ Handoff 模板
 - ├─ 保险公司运维交接 SOP
 - └─ 监管检查应答模板

新 FDE 接保险客户：用 v3.2 起步，第 4 周客户已经看到 demo。

16.8 公司层面的模式管理

模式提取不是个人行为，公司应该有结构化沉淀：

FDE 团队的"知识中心" 结构

公司 Wiki:

```
/fde-knowledge/  
  patterns/                (跨行业)  
    rag-starter/  
    agent-starter/  
    eval-ci-starter/  
  industries/              (行业)  
    insurance/  
    finance/  
    retail/  
    manufacturing/  
  anti-patterns/           (反例)  
    2025-Q3-incidents.md  
    ...  
  decision-cards/          (决策卡片)  
  tools/                   (内部工具)
```

Code Repo:

```
fde-platform/  
  starter-kits/  
  common-lambdas/  
  shared-prompts/  
  eval-suites/
```

FDE 老手 vs 新手的差距 80% 在这个知识中心的厚度。

关键引用

“Handoff is not a milestone — it’s a 4-week process you start 3 weeks before launch.” — A. Lawrence, FDE Rule Book, 2025

“The second project on a customer should take 1/3 the time of the first.” — Bob McGrew @ YC, 2025

“Pattern extraction is the difference between a consultant and an engineer.” — AWS GenAI Innovation Center, 2025

动手清单

项目最后 4 周必做：

1. T-3 周开始 Handoff 倒计时 (16.2 节)
 2. 写 Runbook 7 sections (16.3 节)
 3. 客户运维 4 小时培训 (亲手操作)
 4. 客户 owner 影子运营 1 周
 5. 项目结束 1 周内写复盘 + 抽 3 个决策卡片
 6. 抽公司内部能复用的代码 / 文档 / 配置
 7. 跟同事 brown-bag 1 小时 (讲坑而不是讲赢)
-

反模式清单

- ☒ 上线那一周才想起 Handoff (来不及培训)
 - ☒ Runbook 写成“功能文档”(不是可操作 SOP)
 - ☒ 培训只有 PPT 没有动手 (客户记不住)
 - ☒ 客户运维 P1 就打电话给 FDE (4 个能力没培训到位)
 - ☒ 项目结束就下一个, 不复盘 (每个项目都从零)
 - ☒ 模式只在自己脑袋里 (同事不知道, 公司不知道)
 - ☒ 复盘只讲赢 (最值钱的是讲坑)
-

与下一章的关系

到这里, 从 Discovery → Scaffolding → 生产 → Handoff → 模式提取的全流程闭环。

最后一章讲: FDE 自身的能力如何长期成长 —— 不是技术追新, 而是 **T 字成长**: 工程深度 + 行业纵深的双向加深。

[← Part VII 导读 · 下一章: T 字成长 →](#)

Chapter 17: FDE 的 T 字成长 — 工程深度 + 行业纵深

开场

两位 FDE 同公司同 title。

A 入行 3 年，做过 8 个客户，每个客户都是新行业、新栈、新团队。

- 自我介绍："我什么 LLM 项目都做过"
- 客户对接 4-5 周才能进入状态
- 没成为任何一个客户的"指定 FDE"

B 入行 3 年，做过 6 个客户，5 个都是金融行业。

- 自我介绍："金融 LLM，从核保到反欺诈，3 年深耕"
- 客户开第一次会，他能直接讨论"等保 2.0 三级在 LLM 应用上的具体落法"
- 3 个老客户每年都续单 + 推荐新客户
- 公司内部"金融业 FDE 老法师"

A 跳槽时，HR 不知道他擅长什么。

B 跳槽时，3 家保险公司直接发 offer。

两人技术深度其实差不多。

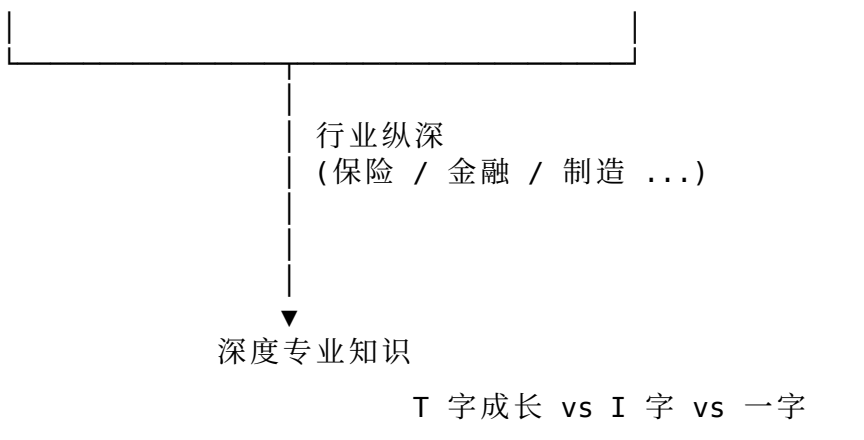
差的是 ——B 选了一个行业纵深，A 没选。

这一章讲：FDE 怎么走"T 字成长"，
让自己 5 年后不只是"做过很多项目"，
而是"成为某个领域不可替代的人"。

17.1 T 字 = 工程深度 × 行业纵深

T 字成长结构图

工程横向 (LLM / 数据 / 集成 / 部署)



纯横向 (一字): 会很多东西但每个都浅
→ 入行 5 年还是"项目工人"

纯纵向 (I 字): 某行业很深但缺工程能力
→ 容易被 AI 工具替代

T 字: 工程横扫 + 行业深耕
→ 5 年后成为"不可替代"

17.2 工程横向 — 必须有的 5 块基础

不分 LLM / 数据驱动主线, FDE 都要会:

1. 数据工程
SQL + dbt + Spark + 数仓 + Schema 设计
→ Ch 9
2. LLM 应用工程
Prompt + RAG + Agent + Eval + MCP
→ Ch 6/7/8/14/15
3. 云工程
AWS / Azure / Aliyun 至少精通一个
IaC (Terraform / CDK / Bicep)
→ Ch 10
4. 平台工程
CI/CD + 监控 + 灰度 + 回滚
→ Ch 13
5. 软技能
Discovery + 客户对话 + 写作 + 演讲
→ Ch 4/5/16

5 块都不到位 → 不是 FDE，是“前线 PM”或“前线实施工程师”。

17.3 行业纵深 — 怎么选 + 怎么深

选行业的 3 个判断

1. 客户基数
 - 这个行业在你公司 / 你国家有多少客户？
 - 100 家 → 值得纵深
 - < 20 家 → 可能不够养你
2. 你的兴趣 + 性格
 - 金融需要严谨 + 合规
 - 制造业需要现场 + 物理
 - 零售需要快 + 创意
 - 选你“待 5 年不烦”的
3. 行业的 LLM 渗透阶段
 - 早期 (2023-2024)：客户都是“试试看”
 - 现在 (2025-2026)：客户都是“严肃落地”
 - 你最好选“现在 + 未来 5 年还会持续”的行业

行业纵深的 3 个层次

Level 1: 知道术语

能听懂客户内部沟通

例：保险 — 知道“核保 / 理赔 / 销保 / 投保”区别

时间：3-6 个月

Level 2: 懂业务流

能画出客户主要业务流

能预判客户的痛点和合规约束

例：保险 — 知道“标准核保流 + 5% 个案核保流”

时间：1-2 年

Level 3: 有判断

能告诉客户“这件事别做” + “为什么”

能给客户提“他们没想过的角度”

例：保险 — “理赔自动化别一上来全自动，
先把人工初审 + AI 决策建议跑稳”

时间：3-5 年

17.4 怎么“加深”行业纵深 — 6 个动作

不是读更多文章，是有意识做 6 件事：

1. 每年读 2-3 本行业经典
 - 保险："保险学原理" / "Solvency II 入门"
 - 金融："Bank 4.0" / "Risk Management" 教材
 2. 每月看 1 份"客户内部资料"
 - 客户的年报 / 半年报
 - 行业协会发的报告
 - 监管新政
 3. 每季度参加 1 次行业会议
 - 不是 AI 会议，是行业本身的会
 - 例：保险产品创新论坛 / 年会
 4. 每个项目结束写"行业模板 v X+1"
 - Ch 16 的模式提取
 - 一年下来你的"保险 RAG 模板"会非常成熟
 5. 跟客户内部"业务老师傅"交朋友
 - 不是 IT，是业务
 - 业务老师傅一句话比 100 篇文章值
 6. 给客户老板做"非技术分享"
 - 讲"我看到你们行业的趋势"
 - 不讲 AI，讲行业 → 客户开始当你"自己人"
-

17.5 工程深度 vs 工程横向 — 深 1-2 块

T 字的“竖”还有一个区分：

工程横向：5 块都要懂（17.2 节）

工程深度：5 块里挑 1-2 块"深"

推荐深的方向（FDE 长期价值）：

- A. LLM 应用 + Eval + Agent（生态前沿）
→ 适合：喜欢追新 + 懂分布式
- B. 数据治理 + Ontology + 数据栈（Palantir 路线）
→ 适合：喜欢稳 + 懂业务
- C. 云原生 + IaC + 平台工程
→ 适合：喜欢工程美学 + DevOps 性格

- D. 安全 + 合规 + 审计
→ 适合：严谨 + 大客户多

别试图“5块全深”。不是不行，而是没必要 + 没时间。深 1-2 块就足够形成竞争力。

17.6 5 年成长路径

一个“T 字成长”的典型 5 年路径：

Year 1: 入门

- 跟着资深 FDE 做 1-2 个项目
- 工程横向覆盖 60% (5 块各懂入门)
- 选定行业 (尝试 1-2 个, 年底定)
- 第一年项目复盘写出 30 页文档

Year 2: 站稳

- 独立做 2-3 个项目 (其中 1-2 个在你选的行业)
- 工程横向 80% (5 块各到中级)
- 行业 Level 1 → Level 2
- 工程开始深 1 块

Year 3: 形成专长

- 独立做 3-4 个项目, 70% 在你选的行业
- 工程横向 90% + 1-2 块深 (高级)
- 行业 Level 2 → 到 Level 3 边缘
- 公司内部“行业 X 老法师”声誉初步形成

Year 4: 不可替代

- 客户主动找你 (不再公司分配)
- 行业 Level 3
- 工程深度: 1 块顶级 + 1 块高级
- 开始指导新人 / 写公司内部最佳实践

Year 5: 选下一步

- 路径 A: 资深 FDE / Principal
→ 同样模式但客户更高端
- 路径 B: FDE 团队 lead
→ 带 5-10 个 FDE
- 路径 C: 切换到产品 / 内部产品 lead
→ 用 FDE 经验做产品
- 路径 D: 创业

→ 在你深的行业 + 工程方向上

17.7 反模式 — 5 年没长出 T

- 每个项目都换行业
→ 5 年后什么行业都不深
 - 工程上"什么新都试", 没有 1 块深
→ 5 年后哪一块都浅
 - 不写复盘, 不抽模板
→ 5 年后还是"项目工人"
 - 不和业务专家交朋友
→ 5 年后还是"工具人", 进不了客户决策层
 - "FDE 是中转岗"
→ 干一年就想跳产品 / 跳大厂
→ 没拿到 FDE 的真正价值
-

17.8 给现在的你 4 句话

1. 每个客户都问自己: "我能从这个项目带走什么沉淀?"
不是成果 (成果属于客户和公司),
是模式 / 决策 / 行业知识。
 2. 选一个行业, 待 3 年。
哪怕第 1 年你不喜欢, 第 3 年你会发现
你成为了"在这个行业里能用工程语言谈话的少数人"。
 3. 每周保留一个"学习钟"。
看行业资料 / 读论文 / 写复盘。
这个钟是 FDE 长期价值的来源。
 4. 写下来。
脑袋里的不是知识, 纸上的才是。
5 年后回看自己的 200 篇笔记
比简历更能告诉你"你成为了谁"。
-

关键引用

“The best FDEs become so industry-fluent that customers think they used to work for the customer.” — A. Lawrence, FDE Rule Book, 2025

“Optionality is overrated. Depth is underrated.” — Bob McGrew @ YC, 2025

“A T-shaped engineer is the only kind that survives the next decade of AI.” — Anthropic engineering culture, 2025

动手清单

接下来 4 周内做：

1. 画自己的 T 字：横向 5 块各打分；纵向行业选了吗？
 2. 缺哪块工程横向 → 这季度补（每季度补 1 块）
 3. 缺行业 → 这个月选 1 个（依据 17.3 三个判断）
 4. 建私人复盘库：每个项目结束 1 周内写复盘
 5. 每周保留一个“学习钟”：行业 + 工程深度
 6. 下个项目结束跟同事 **brown-bag**：分享坑而不是赢
 7. 3 年后回看这本书：哪些反模式你避开了
-

反模式清单

- ☒ 每个项目都换行业（5 年都浅）
 - ☒ 工程“什么都新”没有深一块（被 AI 工具替代风险高）
 - ☒ 看到“AI 工程师”风口就跳（FDE 的稀缺性正是因为难做）
 - ☒ 不写复盘（脑子里的不是知识）
 - ☒ 不交业务老师傅朋友（永远进不了客户决策层）
 - ☒ 觉得 FDE 是“中转岗”（拿不到真正价值）
-

全书结尾

到这里，FDE 工程的实用手册告一段落。

17 章一句话总结

Part I: FDE 是工程师里“最 PM”的，PM 里“最工程师”的。

Part II: 没有 Discovery 透 + Eval 集，写代码就是凭信仰。

Part III: 选型 30 分钟、Eval 第一周、Prompting 优先。

Part IV: 数据 + 网络 + 身份 + 审计 = 企业 FDE 的基础。

Part V: PoC 像生产做，灰度 + 回滚 + 成本第一周就要有。

Part VI: Agent 工具集少即是多，MCP 是企业未来 2 年标配。

Part VII: Handoff 4 周倒计时，模式提取每个项目都做，T 字成长每年问自己。

最后一句留给所有 FDE：

“Sell the outcome. Fix forward. Eval first. Hand it off. Extract the pattern.”

5 年后回看这本书，希望你已经成为某个行业里不可替代的那个 FDE。

[← 上一章: Handoff + 模式提取](#) · [返回目录](#)

附录 A: FDE 工具栈速查表 (2026 版)

每类 3-5 个候选 + 选型维度。供 PoC 选型时 30 分钟决策用。

A.1 模型 API

模型	强项	适合场景	价格档
Claude 3.5 Sonnet	平衡 + 长上下文	通用 RAG / Agent	中
Claude 3 Opus	推理强	复杂任务 / Judge	高
Claude 3 Haiku	快 + 便宜	高 QPS / 简单任务	低
GPT-4o	强 + 多模态	通用 + 视觉	中高
GPT-4o-mini	性价比	高 QPS 通用	低
Llama 3.1 70B	开源旗舰	私有部署	自部署
Qwen2.5 72B	中文强	中文场景	自部署
DeepSeek V3	推理 + 中文	中文 + 推理	自部署
Bedrock Titan	AWS 原生	AWS 一站式	中低
Mistral Large	欧洲	多语言	中

选型维度: 部署形态 (云/私有) / 上下文长度 / 中英文 / 推理需求 / 月度预算

A.2 编排 / 框架

框架	强项	适合阶段	风险
LangChain	生态最大	PoC 快出 demo	抽象漏
LlamaIndex	RAG 专精	RAG 项目	Agent 弱
LangGraph	状态图 + Agent	生产 Agent	学习曲线
Bedrock Agents	AWS 一站式	AWS 项目	锁定 AWS
AutoGen	多 agent	研究 / 复杂多 agent	不太稳
CrewAI	角色扮演	内容 / 研究	不太稳
原生 SDK	最稳	生产关键路径	开发慢

经验: PoC 用 LangChain, 生产换原生 SDK + 状态机。

A.3 向量库 / Knowledge Base

工具	形态	适合规模	备注
Bedrock Knowledge Bases	AWS 托管	中	OpenSearch 后端默认
OpenSearch (Serverless / 自部署)	AWS / 开源	中-大	hybrid search 强
Pinecone	SaaS	中-大	最省心, 但成本高
Weaviate	开源 + SaaS	中	多 modal
pgvector	PostgreSQL 插件	小-中	客户已有 PG 时优先
Milvus	开源	大	K8s 部署
Chroma	开源	小 / PoC	内存型
Aurora pgvector	AWS Aurora	小-中	VPC 内首选

选型维度: 规模 / 部署 / 元数据过滤 / 客户已有什么

A.4 Eval / Trace

工具	类型	强项
Bedrock Evaluations	AWS 托管	Model / KB / Agent 三类 eval
LangFuse	OSS + Cloud	LLM trace + eval 一站
LangSmith	LangChain 官方	LangChain 项目
Phoenix (Arize)	开源	私有部署友好
DeepEval	Python 库	写在 pytest 里
Promptfoo	YAML 驱动	CI 友好
Ragas	开源	RAG 专评
Helicone	代理 trace	OpenAI 兼容代理

经验: PoC 用 LangFuse Cloud, 企业用 LangFuse 自部署 / Phoenix。

A.5 数据工程

工具	类型	适合
dbt	SQL 转换	数仓内的 ETL
Airflow	编排	复杂工作流
AWS Glue	Serverless ETL	AWS 一站式
Spark / EMR	大数据	TB+ 规模

工具	类型	适合
Iceberg / Delta	表格式	数据湖
Snowflake / BigQuery / Redshift	数仓	分析
Lake Formation	AWS	权限治理
OpenLineage	血缘	数据血缘

A.6 部署 / 运行时

工具	类型	适合
AWS Lambda	Serverless	单步 LLM / 简单 Agent
ECS Fargate	容器	MCP server / 中等服务
EKS	K8s	复杂 / 大规模
SageMaker	训练 + 部署	自训模型
SageMaker JumpStart	一键部署	LLM 自部署
Step Functions	编排	长流程 / HITL
API Gateway	API 网关	鉴权 + 限流
AppConfig	配置中心	prompt / model 热切

A.7 安全 / 合规

工具	用途
IAM Identity Center	SSO + SCIM
KMS	密钥管理
Secrets Manager	API key / OAuth
CloudTrail	API 审计
Config	配置合规
Security Hub	综合安全
Macie	PII 自动发现
Bedrock Guardrails	LLM 内容过滤
WAF	应用防火墙

A.8 IaC

工具	类型	适合
Terraform	多云	通用
AWS CDK	编程式 IaC	AWS 项目 + 复杂逻辑
AWS SAM	Serverless 专用	Lambda / API GW
Pulumi	编程式多云	同 CDK 思路但多云
CloudFormation	AWS 原生	简单堆栈
Bicep	Azure	Azure 项目

A.9 监控 / 告警

工具	用途
CloudWatch	AWS 原生 metrics + logs
X-Ray	AWS 分布式 trace
Datadog	综合 observability
Grafana / Prometheus	开源 + 私有部署
OpenTelemetry	协议 + SDK
Sentry	错误追踪
PagerDuty	on-call
Slack alerts	协作通知

A.10 客户协作

工具	用途
Confluence	客户 wiki
Jira	客户工单
Notion	FDE 内部笔记
Slack / Teams	即时沟通
Linear	高效工单管理
Loom	屏幕录屏（解释问题）
Excalidraw	架构图
Miro	协作白板

附录 B: 模型 / 框架 / 平台对比矩阵

附录 A 是“广度速查”，本附录是“深度对比”。当你在某一类工具里要做严肃决策时，看这里。

B.1 主流大模型 — 9 维对比

维度	Claude 3.5 Sonnet	Claude 3 Opus	Claude 3 Haiku	GPT-4o	GPT-4o-mini	Llama 3.1 70B	Qwen 2.5 72B	DeepSeek V3
上下文	200K	200K	200K	128K	128K	128K	128K	128K
输出 token	8K	4K	4K	16K	16K	8K	8K	8K
中文	强	极强	一般	强	良好	一般	极强	极强
推理	极强	顶级	一般	极强	良好	良好	良好	极强
工具调用	极强	极强	良好	极强	极强	良好	良好	良好
多模态	视觉	视觉	视觉	视觉 + 音频	视觉	文本	视觉	文本
价格 (输入/M tok)	\$3	\$15	\$0.25	\$5	\$0.15	自部署	自部署	\$0.27
价格 (输出/M tok)	\$15	\$75	\$1.25	\$15	\$0.6	自部署	自部署	\$1.1
Bedrock 可用	☑	☑	☑	☑	☑	☑	☑(亚马逊科技中国可)	☑

FDE 决策模式:

企业默认: Claude 3.5 Sonnet (Bedrock) — 平衡 + AWS 原生

高 QPS: Claude 3 Haiku 或 GPT-4o-mini
最高质量: Claude 3 Opus (judge / 关键路径)
中文 + 私有: Qwen 2.5 72B 或 DeepSeek V3 自部署
欧洲合规: Mistral Large

B.2 RAG 框架对比

维度	LangChain RAG	LlamaIndex	Haystack	Bedrock KB	自建
上手速度	中	快	中	极快	慢
文档解析	中等	强 (LlamaParse)	强	中等	自定
Chunking 策略	多	多	多	默认 + 自定	自定
检索策略	多	极多	多	hybrid (OS)	自定
Rerank	接 Cohere/外部	内置 + 外部	内置	接 Cohere	自定
增量更新	自己写	自己写	自己写	自动 (S3)	自定
多源融合	强	极强	中	中	自定
生产成熟度	中	中	高	高	取决于团队
锁定风险	低	低	低	高 (AWS)	无

经验: - PoC: LlamaIndex 最快出 demo - 生产 + AWS: Bedrock KB (4 周省下一半工程) - 长期
复杂 RAG: 自建 (用 Bedrock 做检索层 + 自己控编排)

B.3 Agent 框架对比

维度	LangGraph	Bedrock Agents	AutoGen	CrewAI	OpenAI Assistants	原生
编程模型	状态图 (DAG)	配置式	多 agent 对话	角色扮演	API + 状态服务	完全
工具集成	自由	Action Group	自由	自由	内置 + Function	自
长任务	LangGraph Cloud	配 Step Functions	弱	弱	内置线程	自
HITL	内置 interrupt	工程化弱	中	弱	弱	自
可观测	LangSmith	CloudWatch + Trace	自定	自定	OpenAI dashboard	自
锁定风险	中	高 (AWS)	低	低	高 (OpenAI)	无
适合规模	小到大	中 + AWS 客户	研究 / 实验	内容生成	个人助理	高

经验: - 客户在 AWS + 业务流不复杂 → Bedrock Agents - 客户多云 + 业务流复杂 → LangGraph
- 关键路径 + 严谨 → 原生 SDK + 状态机

B.4 向量库对比 — 6 个生产维度

维度	Pinecone	Weaviate	Milvus	OpenSearch	pgvector	Chroma
部署	SaaS	SaaS + OSS	OSS (K8s)	AWS / OSS	PG 插件	OSS / 本地
单库规模	10 亿 +	10 亿 +	100 亿 +	10 亿 +	1 亿	千万
Hybrid 检索	支持	支持	支持	极强	弱 (需扩展)	弱
元数据过滤	强	强	强	极强	极强 (SQL)	中
多 modal	中	强	中	中	弱	弱
增量 / 更新	容易	容易	容易	容易	极容易 (UPDATE)	容易
VPC 内部署	Private 版	自托管	自托管	自托管 / Serverless	RDS / Aurora	自托管
成本	高	中	中 (运维成本)	中	极低 (PG 已有)	极低
客户已有	通常没有	通常没有	通常没有	部分 AWS 客户	大量客户已有 PG	通常没有

FDE 经验决策:

客户已有 PG / Aurora → pgvector (零额外工具)
 客户在 AWS + 中等规模 → OpenSearch Serverless
 规模 > 5 亿 → Milvus 或 Pinecone
 PoC / 小项目 → Chroma
 最省心 / 预算够 → Pinecone

B.5 Eval 工具对比

维度	Bedrock Evaluations	LangFuse	Phoenix	DeepEval	Promptfoo	Ragas
形态	AWS 托管	OSS + Cloud	OSS	Python lib	YAML CLI	OSS
Eval 范围	Model / KB / Agent	LLM 全栈	LLM 全栈	单测嵌入	提示对比	RAG
LLM-as-judge	内置	支持	支持	支持	支持	支持
自动指标	BLEU / ROUGE / 准确率	自定	自定	内置很多	自定	RAG
数据集管理	S3 jsonl	UI + API	UI	代码	YAML	代码
在线 trace	否 (静态)	强	强	否	否	否
CI 友好	中	中	中	强	极强	中
成本	按 token + judge	OSS 免 / Cloud 收	OSS 免	OSS 免	OSS 免	OSS 免

典型组合:

LangFuse (trace + 在线) + Promptfoo (CI) + Bedrock Evaluations (合规验收)

B.6 部署平台对比 — LLM 应用维度

维度	Lambda	ECS Fargate	EKS	SageMaker Endpoint	Step Function
启动延迟	冷启动 100ms-3s	容器启动秒级	Pod 秒级	启动秒级	编排开销
单次最长	15min	无	无	无	1 年

维度	Lambda	ECS Fargate	EKS	SageMaker Endpoint	Step Function
并发模型	自动	自管	自管	自管	N/A
适合	单次 LLM 调 / 简单 Agent	长 server / MCP	复杂集群	自部署模型	长流程 / HITL
成本特征	用多少付多少	按容器	按节点	按 endpoint	按状态转移
状态	无状态	无状态 / EFS	StatefulSet	无状态	状态在 SF
镜像大小	250MB / 10G(image)	大	大	大	N/A

经验: - 简单 RAG / API → Lambda - MCP server / Agent 服务 → ECS Fargate - 长流程 + HITL
→ Step Functions + Lambda - 自部署 70B+ 模型 → SageMaker / EKS + GPU

B.7 编程语言 / 运行时

维度	Python	TypeScript	Go	Rust
LLM SDK 成熟度	顶级	良好	良好	起步
数据处理	极强	中	强	强
异步并发	asyncio (中等)	原生强	goroutine 强	tokio 强
部署体积	大	中	极小 (单二进制)	极小
团队招聘	极易	易	中	难
FDE 推荐场景	数据 / Eval / PoC	前端 / Lambda	API / 高并发	关键路径 / 嵌入

FDE 默认: Python + TypeScript 双栈, 绝大部分场景够用。

B.8 决策综合表

客户场景 → 推荐技术栈

AWS 中等保险公司 + 文档 RAG
模型: Bedrock Claude 3.5 Sonnet
框架: Bedrock Knowledge Bases + 自建 Lambda
向量: OpenSearch Serverless
Eval: LangFuse Cloud + Promptfoo CI
部署: Lambda + API Gateway
IaC: AWS CDK
监控: CloudWatch + X-Ray
安全: IAM Identity Center + Bedrock Guardrails

本地数据库已 PG + 中文金融 + 内网

模型: DeepSeek V3 / Qwen 2.5 72B 自部署 (vLLM)
框架: 原生 SDK + 状态机
向量: pgvector
Eval: Phoenix (自部署) + 自定 judge
部署: K8s
IaC: Terraform
监控: Prometheus + Grafana
安全: 内部 LDAP + 自定 audit

跨国零售 + 多 SaaS 集成 + Agent
模型: Claude 3.5 Sonnet
框架: LangGraph + MCP servers
向量: Pinecone
Eval: LangFuse + DeepEval
部署: ECS Fargate
IaC: Pulumi (多云)
监控: Datadog
安全: Okta + per-call OAuth

附录 C: 评估集设计模板

给 4 个最常见 LLM 任务类型的 Eval 模板：1. RAG 问答 2. Agent 任务执行 3. 文本分类 4. 结构化信息抽取

每个模板包含 jsonl schema、字段含义、采样策略、评估方式、CI 例子。

C.1 共通设计原则

4 个层次的 Eval 集

Seed Set	(5-20 条)	手工写，极快回归，永不动
Golden Set	(100-300 条)	客户专家标注，验收基线
Adversarial	(50-150 条)	反例 / 边界 / 越权
Production	(流式)	线上采样 → 周度回灌

通用 schema 字段:

```
{
  "id": "unique stable id",
  "category": "case category for slicing",
  "input": "the input to the system",
  "expected": "what success looks like",
  "metadata": {
    "source": "where this case came from",
    "added_at": "2026-05-22",
    "reviewer": "reviewer email",
    "difficulty": "easy / medium / hard",
    "is_adversarial": false
  }
}
```

为什么是 jsonl 而不是 csv: 嵌套字段 / 多行文本 / 长 prompt 都常见, csv 转义噩梦。

C.2 模板 1: RAG 问答 Eval

Schema

```
{
  "id": "rag-001",
  "category": "policy_lookup",
  "input": {
    "question": "客户购买的健康险是否包含住院津贴？保单号 P20240315-001"
  },
  "expected": {
    "must_contain_keywords": ["住院津贴", "200元/天", "30天"],
    "must_not_contain": ["免责", "不包含"],
    "reference_doc_id": "POL-HC-002-v3.pdf",
    "reference_section": "第三章第2条",
    "reference_answer": "包含住院津贴，标准为每天 200 元，最长 30 天"
  },
  "metadata": {
    "source": "客户业务专家 2026-05",
    "difficulty": "medium"
  }
}
```

评估逻辑 (Python 伪代码)

```
def evaluate_rag(case, system_output):
    score = {}

    # 1. Hard rules —关键词
    score['kw_match'] = all(
        kw in system_output['answer']
        for kw in case['expected']['must_contain_keywords']
    )
    score['no_forbidden'] = all(
        kw not in system_output['answer']
        for kw in case['expected']['must_not_contain']
    )

    # 2. 检索源对不对 (最常见的 RAG 错)
    score['retrieval_correct'] = (
        case['expected']['reference_doc_id']
        in [d['doc_id'] for d in system_output['retrieved_docs']]
    )

    # 3. 语义相似度 (embedding cosine)
    score['semantic_sim'] = cosine(
        embed(case['expected']['reference_answer']),
```

```

        embed(system_output['answer']))
    )

    # 4. LLM-as-judge (最贵, 留给关键案例)
    if case['metadata']['difficulty'] == 'hard':
        score['llm_judge'] = call_judge(
            question=case['input']['question'],
            reference=case['expected']['reference_answer'],
            actual=system_output['answer']
        )

    return score

```

采样策略

Seed Set:

- 最高频 5 个问题 (业务最常问的)
- 5 个明确反例 (绝不能答错的)

Golden Set 200 条 = 这样配比:

- 60% 主流问题 (业务专家说的高频 case)
- 20% 边界情况 (跨保单 / 多保人 / 历史保单)
- 10% 长尾稀有问题
- 10% 反例 (越权问题 / PII 问题 / 不该回答的)

Adversarial 50 条:

- 改写 / 拼错 / 换术语
- 注入 (prompt injection)
- 跨域 (问别家保险公司)

CI 阈值

```

# .github/workflows/rag-eval.yml
- name: Run RAG eval
  run: |
    python eval/run_rag.py --set golden --output result.json
    python eval/check.py \
      --result result.json \
      --thresholds \
        kw_match>=0.95 \
        retrieval_correct>=0.92 \
        semantic_sim>=0.80 \
        llm_judge>=0.85

```

C.3 模板 2: Agent 任务执行 Eval

Schema

```
{
  "id": "agent-claim-001",
  "category": "auto_settle_simple",
  "input": {
    "claim_id": "CLM-2026-001",
    "user_message": "我要报销上周的门诊医药费 580 元",
    "context": {
      "customer_id": "C123",
      "policy_id": "P-OPD-001"
    }
  },
  "expected": {
    "task_completed": true,
    "tool_calls": [
      {"tool": "get_policy", "params_must_include": ["P-OPD-001"]},
      {"tool": "validate_amount", "params_must_include": [580]},
      {"tool": "create_settlement", "must_appear": true}
    ],
    "tool_calls_must_not": ["transfer_funds"],
    "final_state": {
      "claim_status": "approved",
      "approved_amount": 580
    },
    "max_steps": 8,
    "max_cost_usd": 0.05
  },
  "metadata": {
    "difficulty": "easy"
  }
}
```

5 维评估

1. 任务完成度
final_state == expected.final_state ?
2. 路径合理性
tool_calls 是否满足"必须 / 不能"约束?
步数是否 $\leq$ max_steps?
3. 工具调用正确性
每次工具的参数对不对?
4. 副作用

有没有调到 `must_not` 列表的工具?
有没有产生外部状态污染?

5. 成本

总 `token / 美元` $\leq$ `max_cost_usd`?

Python 评估器

```
def evaluate_agent(case, trace):
    """trace 是 agent 的完整执行轨迹"""
    tool_calls = trace['tool_calls']

    # 1. 任务完成
    task_done = trace['final_state'] == case['expected']['final_state']

    # 2. 工具序列
    must_tools = case['expected']['tool_calls']
    must_present = all(
        any(
            tc['tool'] == m['tool']
            and all(p in tc['params'] for p in m.get('params_must_include', []))
            for tc in tool_calls
        )
        for m in must_tools
    )

    # 3. 禁用工具
    forbidden_used = any(
        tc['tool'] in case['expected']['tool_calls_must_not']
        for tc in tool_calls
    )

    # 4. 步数 / 成本
    within_steps = len(tool_calls) <= case['expected']['max_steps']
    within_cost = trace['total_cost_usd'] <= case['expected']['max_cost_usd']

    return {
        'task_done': task_done,
        'must_tools_present': must_present,
        'no_forbidden': not forbidden_used,
        'within_steps': within_steps,
        'within_cost': within_cost,
        'overall_pass': all([
            task_done, must_present, not forbidden_used,
            within_steps, within_cost
        ])
    }
```

采样策略

Golden 100 条 = 这样配比：

- 40% 标准用例（能自动完成的简单 case）
 - 30% 需要 HITL 的 case（期待 escalate）
 - 15% 异常输入（缺字段 / 模糊指代）
 - 15% 反例（越权 / 注入 / 对抗）
-

C.4 模板 3: 分类 / 路由 Eval

Schema

```
{
  "id": "intent-001",
  "category": "core_intents",
  "input": {
    "user_message": "我的车出事故了，怎么办？"
  },
  "expected": {
    "intent": "claim_intake",
    "confidence_min": 0.8
  },
  "metadata": {
    "difficulty": "easy"
  }
}
```

评估指标

多类别分类

```
from sklearn.metrics import classification_report, confusion_matrix

y_true = [c['expected']['intent'] for c in cases]
y_pred = [run_classifier(c['input']) for c in cases]

print(classification_report(y_true, y_pred))
print(confusion_matrix(y_true, y_pred))

# CI 阈值: macro F1 ≥ 0.85, 每类 recall ≥ 0.80
```

关键采样

- 每个意图至少 30 条（不平衡时上采样）
- 业务上“代价非对称”的混淆类要单独标注
例：claim_intake 误判为 general_query → 客户体验差
general_query 误判为 claim_intake → 浪费人工

前者代价高，必须重点测

C.5 模板 4: 结构化抽取 Eval

Schema

```
{
  "id": "extract-invoice-001",
  "category": "ocr_invoice",
  "input": {
    "image_url": "s3://bucket/invoice-123.png",
    "ocr_text": "..."
  },
  "expected": {
    "invoice_no": "INV-2026-0312",
    "total_amount": 1280.50,
    "currency": "CNY",
    "issue_date": "2026-03-12",
    "vendor": {
      "name": "ABC 商贸有限公司",
      "tax_id": "91110000XXXXXXXX"
    }
  },
  "metadata": {
    "difficulty": "medium",
    "image_quality": "good"
  }
}
```

字段级评估

```
def field_level_eval(case, output):
    expected = case['expected']
    score = {}
    for field in flatten(expected).keys():
        e = get(expected, field)
        a = get(output, field)
        if isinstance(e, str):
            score[field] = (e == a)
        elif isinstance(e, (int, float)):
            score[field] = abs(e - a) / max(abs(e), 1) < 0.001
        elif isinstance(e, dict):
            # 嵌套, 递归
            ...
    return score
```

```
# 总分 = 关键字段加权平均
weights = {
    'invoice_no': 1.0,
    'total_amount': 1.0,    # 关键, 数字必须准
    'currency': 0.5,
    'issue_date': 0.8,
    'vendor.name': 0.6,
    'vendor.tax_id': 1.0    # 合规要点
}
```

C.6 一个完整 Eval 项目结构

```
project/
├── eval/
│   ├── datasets/
│   │   ├── seed.jsonl          (10 条, 锁版)
│   │   ├── golden_v1.jsonl     (200 条, 业务专家标)
│   │   ├── golden_v2.jsonl     (250 条, 后续追加)
│   │   ├── adversarial.jsonl   (50 条)
│   │   └── prod_sampled_2026w20.jsonl (周度回灌)
│   ├── runners/
│   │   ├── run_rag.py
│   │   ├── run_agent.py
│   │   └── run_classifier.py
│   ├── judges/
│   │   ├── llm_judge_prompts.yaml
│   │   └── rules.py
│   ├── reports/
│   │   ├── 2026-05-22_release_v1.2.html
│   │   └── ...
│   ├── README.md
│   └── .github/workflows/
│       └── eval.yml
```

C.7 标注流程 — 让客户业务专家高效干活

- Step 1: FDE 准备 100 个种子问题
(从客户文档 / 历史工单 / 客服日志抽)
- Step 2: 业务专家在 Streamlit / Excel 上批
(UI 上展示问题 + 你的初步答案 + 期待修改)

Step 3: 双人 review
(一个专家批 + 另一个抽 20% 复核)

Step 4: 入库 + 锁版 (golden_v1)

Step 5: 每月增量
(出问题的线上 case → adversarial 集)

关键: 业务专家不是写答案, 是校对答案 + 标判定标准。

C.8 Eval 集生命周期管理

Eval 集不是“做完一次就放着”, 而是活的。

规则:

- 每月新增 20-50 条 (来自线上 / bug / 新需求)
 - 每季度审计一次 (是否有过时 case / 错答案)
 - 永远不删 seed (除非该业务下线)
 - 大版本升级 → 新建集合, 旧集合保留为基线
-

C.9 一份 Eval 检查清单

发布前必查:

- ☐ Seed 集每条都过
 - ☐ Golden 集主指标 $\geq$ 阈值
 - ☐ Adversarial 反例至少 90% 拒绝
 - ☐ 成本指标 $\leq$ 预算
 - ☐ 延迟指标 $\leq$ SLO
 - ☐ 切片 (老人 / 新人 / 高价 / 低价 客户) 都过
 - ☐ 与上一版本对比 → 没有回归
-

[← 上一附录: 对比矩阵](#) · [下一附录: 客户启动包](#) →

附录 D: 客户启动包 — 12 份第一周可用模板

接到一个新客户/新项目，第一周你需要的 12 份模板。每份给出 “何时用 / 用法 / 模板正文”。

复制 → 改 → 用。

模板 1: Discovery 提纲 (访谈用)

何时: Kickoff 后 1-7 天 **对象:** 客户业务方 / IT / 安全各 1-2 人

Discovery Interview — [客户名] / [访谈对象]

日期: 2026-XX-XX

访谈人: [FDE 名]

受访人: [姓名 / 部门 / 职位]

时长: 45 分钟

————— 业务背景 (10 min) —————

- Q1. 你们部门一天的核心工作流程是什么?
- Q2. 这个流程里目前最痛的点是什么? 花你们多少人 / 多少时间?
- Q3. 你期待这个项目让什么变化发生? 怎么衡量成功?

————— 数据 (10 min) —————

- Q4. 这件事涉及哪些数据? 这些数据现在在哪?
- Q5. 数据是谁负责的? 拿数据要走什么流程?
- Q6. 历史上有没有人尝试用这些数据做过相似事情? 结果如何?

————— 技术 (10 min) —————

- Q7. 你们的 IT 栈是什么? (云 / 自有 / 混合)
- Q8. 现在已经有 AI / ML / LLM 相关尝试?
- Q9. 这个项目有什么明确的技术约束? (必须 VPC / 必须开源 / 必须中文)

————— 合规与决策 (10 min) —————

- Q10. 这件事涉及哪些合规约束? (等保 / 数安 / 行业监管)
- Q11. 决策链是谁? 谁批 prod 上线?

Q12. 半年内有没有审计 / 检查计划?

————— 项目预期 (5 min) —————

Q13. 你期待的时间线?

Q14. 项目结束后谁来运营?

Q15. 还有什么我们没问到但你想说的?

FDE 自留笔记字段:

- 受访人对项目热情: 高 / 中 / 低
- 是真正决策者还是传话人?
- 提到了什么"不能做"的红线?
- 提到了什么"上一次尝试失败"的故事?

模板 2: Discovery 总结报告

何时: Discovery 周结束 对象: 客户高层 + 项目组

[客户名] AI 项目 Discovery 报告

一、业务现状

- 当前流程图 (一页 A4)
- 痛点清单 (排序, 量化损失)
- 现有尝试与教训

二、问题边界

- 我们要解决: ...
- 我们不解决: ... (写明)

三、数据现状

- 数据清单 (表 + 字段 + 量级 + 更新频率)
- 数据可用性评估 (红/黄/绿)
- 数据治理风险

四、技术约束

- 部署形态 (云/私有/离线)
- 必须用 / 不能用清单
- 安全合规要求

五、推荐方案 (2-3 个)

- 方案 A: 简单 / 快 / 6 周 覆盖 60% 用例
- 方案 B: 中等 / 12 周 覆盖 85% 用例
- 方案 C: 完整 / 20 周 覆盖 95% 用例

六、不可走的死路

- 路径 X —原因
- 路径 Y —原因

七、第一阶段 SOW 草案

- 范围
- 验收
- 里程碑
- 资源需求

模板 3: PoC SOW 骨架 (6-8 周)

Statement of Work — [项目名] PoC

1. 项目目的

[一句话: 验证 X 是否可行]

2. 范围 In-Scope

- 功能 A
- 功能 B
- 数据源 X (read-only)

3. 范围 Out-of-Scope (写明)

- 功能 C 不做
- 不接 SaaS Z
- 不做手机 APP

4. 交付物

- (1) PoC 应用 (可演示)
- (2) Eval 集 200 条 + 通过率报告
- (3) 架构图 + 部署文档
- (4) 复盘 + 后续路线图

5. 验收标准

- Eval 主指标 $\geq X\%$
- 单次响应延迟 $P95 \leq Y$ 秒
- 单次成本 $\leq Z$ 元
- 主要场景 5 个 demo 跑通

6. 时间线

W1: Discovery 收尾 + Eval 集 v0

W2-W3: 脚手架 + 第一版能跑

W4-W5: 迭代 + Eval 上分

W6: 验收 + Demo + 总结

7. 责任分工

- 客户: 业务专家 8h/周 + IT 4h/周 + 数据准备
- FDE: 全职 1 人

8. 假设与依赖

- AWS 账号 + 配额于 W1 前准备好
- VPC / 网络通路 D-3 前打通
- 数据样本（脱敏）于 W1 前可用

9. 变更管理

- 范围变更需双方书面确认
- 变更要重新评估时间线和价格

10. 价格

[依公司报价规则]

模板 4: Eval 集 v0 (jsonl 起手)

```
{"id":"E001","category":"core","input":{"...},"expected":{"must_contain":["..."]},"metadata":{...}}
{"id":"E002","category":"core","input":{"...},"expected":{"must_contain":["..."]},"metadata":{...}}
{"id":"E003","category":"edge","input":{"...},"expected":{"must_not_contain":["免责"]},"metadata":{...}}
{"id":"E004","category":"adversarial","input":{"...},"expected":{"refuse":true},"metadata":{...}}
```

首日开口要的 5 件事: 1. 业务专家手写 5-10 个最常见问题 + 期望答案 2. 5 个 “如果系统答错就出大事” 的反例 3. 5 个边界 case 4. 5 个对抗 case 5. 1 个判定标准说明（“什么叫答对”）

模板 5: 安全 / 合规问卷应答模板

何时: 客户安全部门发来 “AI 安全调研问卷”

常被问的 25 个问题 + 标准应答骨架

Q1. 数据是否离开客户 VPC?

A: 不离开。模型推理走 VPC Endpoint, 日志走客户 S3 with KMS.

Q2. 模型是否使用客户数据训练?

A: Bedrock / Claude API 有数据隔离声明 (附 Anthropic / AWS 合同).

Q3. PII 如何防泄漏?

A: (1) Bedrock Guardrails sensitive info filter
(2) 应用层 PII 扫描
(3) 输出审计

Q4. 谁能访问什么?

A: IAM Identity Center + SCIM, 角色清单见 Doc 4.2

Q5. 出问题怎么追溯?

A: 全链路 `trace_id`, CloudTrail + Bedrock Logs + 应用 log
保存 90 天 (可调到 7 年, KMS 加密)

Q6. 如何防 prompt injection?

A: Guardrails + 输入 `sanitize` + 工具 `dry_run` + HITL

... (略)

模板 6: 架构图 (Mermaid 起手)

保存为 `docs/architecture.md`

用 mermaid 生成, 客户审计时直接 export 成 PNG.

flowchart TB

```
User[业务用户] --> ALB[ALB / API Gateway]
ALB --> App[应用层 - Lambda/ECS]
App --> Guard[Bedrock Guardrails]
Guard --> Bedrock[Bedrock - Claude]
App --> KB[Bedrock Knowledge Base]
KB --> S3[S3 - 客户文档]
KB --> OS[OpenSearch Serverless]
App --> DDB[DynamoDB - 会话状态]
App --> CW[CloudWatch + X-Ray]
App --> Audit[Audit Log → S3]
```

```
subgraph VPC[客户 VPC]
```

```
    ALB
```

```
    App
```

```
    DDB
```

```
end
```

```
subgraph Endpoints[VPC Endpoints]
```

```
    Bedrock
```

```
    S3
```

```
    DDB
```

```
end
```

模板 7: 风险登记 (Risk Register)

Risk Register — [项目名]

更新于: 2026-XX-XX

ID	Risk	影响	概率	缓解 + Owner	状态
R1	数据交付延迟	高	中	W1 升级 IT	Open
R2	Bedrock 配额不足	中	高	提前申请	Closed
R3	业务专家时间不够	中	中	锁定 8h/周	Open
R4	Guardrails 误杀	中	中	Eval + 人审	Open
R5	客户无人接手 Handoff	高	中	T-3 周培训	Open

模板 8: 周报模板

Week N 周报 — [项目名]

日期: 2026-XX-XX

本周做了什么

- [里程碑] 完成了 X
- [Eval] 主指标从 70% 提升到 78%
- [代码] 解决了 3 个 bug

主要数字

指标	上周	本周	目标
Golden Eval 通过率	70%	78%	≥85%
单次响应延迟 P95	4.2s	3.1s	≤3s
单次平均成本	\$0.08	\$0.06	≤\$0.05

下周计划

- 解决 Bedrock 限流 (R2)
- 跑 Adversarial 50 条
- 与客户业务专家 review

阻碍 / 需客户支持

- [紧急] 数据导出 X 表还没好 (Owner: 客户 IT, due: 周三)
- [一般] 等待安全部门审 IAM 角色

风险变化

- R3 提升到红色 (业务专家本周只参与 4h)

模板 9: Runbook 骨架 (Handoff 前 3 周)

Runbook — [项目名]

1. 系统总览 (一页 A4)

- 架构图

- 主流程
- 关键依赖

2. 部署 / 回滚

2.1 正常部署

```bash

```
./scripts/deploy.sh prod --version=v1.2.3
```

## 2.2 回滚

```
./scripts/rollback.sh prod --target=v1.2.2
```

## 3. Top 10 故障 SOP

- SOP-001: 错误率突升 → ...
- SOP-002: 延迟突升 → ...
- SOP-003: 成本异常 → ...
- ...

## 4. Eval 怎么跑

```
python eval/run.py --set golden --output report.html
```

阈值: kw\_match >= 0.95, semantic >= 0.80

## 5. 关键配置

- AppConfig: prompt\_template\_v3
- 模型 ID: us.anthropic.claude-3-5-sonnet-...
- KB ID: KB-XXXXXX

## 6. 数据 / KB 更新 SOP

1. 上传到 S3: s3://docs/incoming/
2. 触发 sync: aws bedrock-agent start-ingestion-job ...
3. 跑 Eval 验证

## 7. Escalation

- L1 (客户运维): @ops-channel
- L2 (FDE on-call): +XX-XXXX-XXXX (24h)
- L3 (架构师): 仅紧急

---

## ## 模板 10：验收清单（双方签字）

---

[项目名] 验收单 日期: 2026-XX-XX

- ☐ 1. 功能验收 — 5 个 demo 场景全部跑通 [✓] 场景 1: ... [✓] 场景 2: ... [✓] 场景 3: ... [✓] 场景 4: ... [✓] 场景 5: ...
- ☐ 2. 指标验收 [✓] Golden Eval 通过率  $\geq 85\%$  实际: 88% [✓] P95 延迟  $\leq 3s$  实际: 2.4s [✓] 单次成本  $\leq \$0.05$  实际: \$0.04
- ☐ 3. 合规验收 [✓] PII 检查通过 [✓] 审计日志完整 [✓] 安全部门 sign-off
- ☐ 4. 文档验收 [✓] 架构图 [✓] Runbook [✓] Eval 报告 [✓] 复盘 + 路线图
- ☐ 5. Handoff [✓] 培训完成 (4h, 3 人参加) [✓] 影子运营 5 天 [✓] 客户 owner 已能独立 deploy / rollback

---

客户签字: 日期: FDE 签字: 日期: \_\_\_\_\_

---

## ## 模板 11：项目复盘模板（1 周内写）

```markdown

[项目名] 复盘（内部）

日期：2026-XX-XX

作者：[FDE]

一、做了什么

[一段话，不要项目计划重复]

二、关键数字

- 时间：计划 12w，实际 13w
- 预算：\$XX，实际 \$XX
- Eval：主指标 88%
- 客户满意度：4.5/5

三、做对了什么（3 件）

1. W1 就把 Eval 集建好 → 全程没空转
2. 选了 Bedrock KB 不自建 → 省 4 周
3. T-3 开始 Handoff → 上线后 0 P1

四、做错了什么（3 件）

1. W3 才发现 OAuth 流程要改 → 延期 1 周
2. 第一版 Agent 工具 35 个 → 准确率玄学，W7 削减到 12
3. 业务专家时间没锁 → W4-5 进度卡住

五、决策卡片（3 个）

1. "客户问 X 怎么答" → ...
2. "Eval 卡线时优先做 Y" → ...
3. "出现 Z 信号马上回滚" → ...

六、可复用资产

- 代码: insurance-rag-starter v3.2
- 文档: 保险行业 Discovery 模板
- Eval: 保险问答 50 条 golden 模板

七、给下一个 FDE 的建议

- 接保险客户先看 [v3.2 模板]
- 等保三级问题不要绕, 第一周直接面对

模板 12: 客户 Stakeholder Map

Stakeholder Map — [客户名]

决策圈 (decide)

- ★ CEO / VP — Sponsor — 季度过问 — 关心 ROI
- ★ CIO — 决策 — 月度 review — 关心稳定 + 合规

推动圈 (drive)

- 业务方 owner — 每周开会 — 关心业务效果
- 技术 lead — 每周开会 — 关心架构 + 实施
- 安全主管 — 月度 + 关键节点 — 关心合规

执行圈 (do)

- 业务专家 (3 人) — 标 Eval / review 答案
- 数据工程师 (2 人) — 数据交付
- 应用工程师 (2 人) — 接客户内部系统

关注圈 (informed)

- 客服总监 — 关心上线影响
- HR — 关心人员流程变化

关键关系信号:

- 业务方 owner ↔ 技术 lead 关系紧张? → 项目高风险
 - 安全主管什么时候首次 review? → 越早越好
 - Sponsor 是否真在意 (会议出席率)?
-

用法总结

| | | |
|--------|--------------|-----------------------------|
| D-7 | 接到客户 | → 模板 1 (Discovery 提纲) |
| D-3 | Discovery 收尾 | → 模板 2 (总结报告) |
| D-1 | 报价 | → 模板 3 (SOW) |
| D+1 | 首次访谈 + 标注 | → 模板 4 (Eval v0) |
| W1 | 安全审查 | → 模板 5 (问卷应答) |
| W1 | 架构提交 | → 模板 6 (Mermaid) |
| W1 | 项目启动 | → 模板 7 (风险登记) + 模板 12 (干系人) |
| W2-W11 | 日常协作 | → 模板 8 (周报) |
| W9-W11 | Handoff 准备 | → 模板 9 (Runbook) |
| W12 | 验收 | → 模板 10 (验收清单) |
| W12+1 | 复盘 | → 模板 11 (复盘) |

全套 12 份纸面工作 $\approx$ 8-10 小时一次性建好，用一辈子。

术语表

按字母 / 拼音排序。引用页见 [bibliography.md](#)。

A

A. Lawrence — 《Forward Deployed Engineer Rule Book》(2025-10) 作者, 目前唯一英文 FDE 专著。

Agent (AI Agent) — 能自主规划、调用工具、迭代执行的 LLM 应用。本书 Part VI 重点。

Agent Toolset — 给 Agent 暴露的工具集合 (read/write/exec/network/...)。Ch 14 主题。

Anthropic — Claude 模型的制造商。

AWS GenAI Innovation Center — Amazon 的 FDE 等价部门, 官方使用 “Forward deployment engineering” 术语, 公开 45 天 / 73% 数字。

B

Bob McGrew — 前 OpenAI Chief Research Officer。 “Sell the outcome, not the product” 提出者。

C

Conikeec — Substack 作者, 发表 The FDE Playbook: A Practitioner’s Field Manual。

Confluence — Atlassian 的企业 wiki。客户文档库的常见落地形态, FDE Discovery 阶段绕不开。

D

Discovery — FDE 第一阶段: 观察客户 workflow、读现有系统、找真问题。Part II 主题。

Dual-Credential Training — Lawrence 概念: FDE 必须同时拥有 “工程信誉 + 行业信誉”。

E

ETL (Extract-Transform-Load) — 经典数据管道动作。Ch 9 主题。

Eval / Eval Set / Eval-driven — 评估集；先写评估集再写功能代码的开发流。Ch 8 主线。

Embedded Problem Solver — Lawrence 概念：FDE 是被嵌入到客户问题里的解题人。

F

FDE (Forward Deployed Engineer) — 直接驻扎在客户现场、把 AI / 软件落到生产的工程师角色。本书主角。

Fine-tuning — 用客户私有数据继续训练 LLM。Ch 7 选型决策点之一。

Fix Forward — Lawrence 工法：在客户现场就地修问题，不带回总部。

Foundry — Palantir 的旗舰平台，Ontology + 数据集成 + 应用层。Ch 9 参照。

Forward Deployment Engineering — AWS 官方术语，对应其他公司的 “FDE”。

G

GTM (Go-To-Market) — 把产品送到客户手上的方式。

H

Handoff — FDE 离场时把项目交回客户内部团队的动作。Ch 16 主题。

I

Immersion Before Judgment — Lawrence 工法：动手前先沉浸到客户工作流。

Ontology ☒ — 客户业务概念的形式化建模（实体、属性、关系、动作）。Palantir 核心抽象，FDE 数据线主战场。Ch 9 重点。

J

JD (Job Description) — 招聘描述。

L

LLM (Large Language Model) — 大语言模型。本书假设你已会调用 API。

M

MCP (Model Context Protocol) ☒ — Anthropic 牵头的开放协议，标准化 Agent 与外部工具的连接方式。Ch 15 主题。

Morgan Stanley (MS) — OpenAI 公开案例中的合作方，6-8+4 周期来自此案例。

N

Nabeel Qureshi — Palantir 早期员工，FDE 模式最有影响力的英文阐释者。

O

Ontology — 见 “I” 节末尾（中文按拼音排在 “I” 附近）。

Outcome (in “Sell the outcome”) — 客户业务结果（收入、成本、风险），区别于功能 / 工具 / 演示。

P

Palantir — FDE 模式的发明公司。

PoC (Proof of Concept) — 概念验证。本书 Ch 12 重点是 “PoC → 生产” 的过线条件。

Private Deployment / VPC Deployment — 客户私有部署 / VPC 部署。Ch 10 主题。

R

RAG (Retrieval-Augmented Generation) — 检索增强生成。Ch 7 选型决策点之一。

Rule Book — 简称指 A. Lawrence 的《Forward Deployed Engineer Rule Book》。

S

SCIM (System for Cross-domain Identity Management) — 企业身份同步标准。Ch 11 主题。

Sell the outcome, not the product — Bob McGrew 第一定律。Ch 2 主题。

SOW (Statement of Work) — 工作说明书 / 项目范围书。Ch 5 主题。

SSO (Single Sign-On) — 单点登录。客户合规几乎必查。Ch 11 主题。

T

T 字成长 (T-shaped Growth) — 工程深度 + 行业纵深。Ch 17 主题。

Trace / Tracing — 分布式追踪。Ch 13 主题。

V

VPC (Virtual Private Cloud) — 虚拟私有云。客户私有部署的常见承载形式。

其他

反模式 (Anti-pattern) — 常见但有害的做法。本书每章末尾给反模式清单。

动手清单 (Action Checklist) — 这一章读完可以立刻照做的工程动作。本书每章末尾固定栏目。

参考文献

完整原始链接清单与检索方法记录见 `research/05-sources.md`。本文档按 “重要程度 + 类型” 重新排序，优先列出本书反复引用的核心来源。

一、本书反复引用的核心来源

书籍

| # | 作者 / 标题 | 备注 |
|---|--|----------------------------------|
| 1 | A. Lawrence, Forward Deployed Engineer Rule Book, AI Circuit, 2025-10. ISBN 9781970328097. | 目前唯一 FDE 英文专著。本书 Chapter 6 主要参照。 |

一手访谈与原文

| # | 来源 | 标题 / 内容 |
|---|--|---|
| 2 | Bob McGrew × Y Combinator (2025) | YC 访谈，“Sell the outcome, not the product” 出处 |
| 3 | AWS Generative AI Innovation Center 官网 | 45 天 / 73% / 22 家合作伙伴公开数据 |
| 4 | Palantir AI FDE 官方文档 | https://palantir.com/docs/foundry/ai-fde/overview |

长篇分析

| # | 作者 | 标题 |
|---|------------------------|---|
| 5 | Conikeec (Substack) | The FDE Playbook: A Practitioner’s Field Manual |
| 6 | Fresh HQ AI / brgr.one | The FDE Playbook for AI (Bob McGrew YC 访谈整理) |
| 7 | Nabeel Qureshi | Reflections on Palantir (Palantir 早期实践) |

二、2026-05-04 奇点事件相关

| 来源 | 标题 |
|----------------------------------|---|
| TechCrunch (2026-05-04) | Anthropic and OpenAI are both launching joint ventures for enterprise AI services |
| MarkTechPost (2026-05-20) | What is a Forward Deployed Engineer: The AI Role OpenAI, Anthropic, and Google Are Hiring in 2026 |
| MindStudio | Palantir's Forward Deployed Engineer Model Drove 640% Returns |
| Revolution in AI | The Palantir Model That Anthropic and OpenAI Are Now Copying |
| SuperML.dev | OpenAI and Anthropic Adopted the Palantir Playbook |
| LinkedIn Pulse | The FDE Wars Are Here: Anthropic and OpenAI Bet \$5.5 Billion |
| Forbes Tech Council (2025-12-29) | The Rise of the Forward Deployed Engineer |

三、行业博客与 Playbook

| 来源 | 标题 |
|---|--|
| Jediteck | What is a Forward Deployed Engineer? |
| Kiadev | What Is a Forward Deployed Engineer (FDE)? |
| Oflight (日本) | FDE: From Palantir's Original Playbook to 2026 Guide |
| LinkedIn — Zamir Akimbekov (2025-11-04) | The FDE Playbook |
| fderole.com | Definitive Role Guide |

四、中文社区资源（Part VI 中国语境章节主参照）

| 平台 | 标题 |
|----------|---|
| 知乎 | 什么是 Palantir “前线部署工程师”？揭秘 FDE 的角色与影响 |
| 雪球 | “前向部署工程师”（FDE）—— Palantir 模式的核心秘密 (Nabeel 视角中译) |
| 腾讯云开发者社区 | Palantir 深度分析：8. 前向部署工程师（FDE）的工具箱 |
| 51CTO | Palantir 创始工程师深度分享：FDE 模式是 Agent 时代的 PMF 新范式 |
| 搜狐 | Palantir 早期高管解释如何正确的践行 FDE |
| 网易 | FDE 已成过去式？揭秘 Palantir 的下一个“超级士兵”：全栈工程师 |

| 平台 | 标题 |
|------|------------------------|
| CSDN | AI 进化策：Palantir FDE 揭秘 |

五、《FDE Rule Book》专项标识

- **ISBN-13:** 9781970328097
- **ISBN-10:** 1970328096
- **Kindle ASIN:** B0FXSZW5HB
- **作者:** A. Lawrence
- **出版社:** AI Circuit
- **出版日期:** 2025-10
- **本书引用形式:** Lawrence 2025, ch.X

截至 2026-05，本书未发现任何公开免费版本（已检索：Anna’ s Archive / Z-Library / LibGen / GitHub / Substack / Medium / Notion / GitBook）。引用以 Amazon 试读、出版商页面摘要、第三方书评为主。

六、引用规范

正文中的引用使用以下格式：

- 短引用：[Lawrence 2025] / [McGrew @ YC 2025] / [AWS GenAI 官网]
- 数字引用：AWS 公开 45 天 idea-production、73% PoC 转化率
- 中文社区引用：知乎专栏 / 雪球 / 腾讯云

完整 URL 不放在正文里，统一回到本文档与 research/05-sources.md 查。
